РАЗБОР ОФИЦИАЛЬНОГО ОБРАЗЦА

Демонстрационный экзамен 2027

Информационная система мебельного производства

WPF + PostgreSQL



ФИО преподавателя _____

Должность _____

Кафедра / лаборатория _____

Как устроен экзамен

3 ч 30 мин
210 минут на выполнение

Базовый
Уровень ГИА ДЭ

75
Максимум баллов

Разбираем официальный образец КИМ 09.02.07-5-2027. Предприятие выпускает мебель по спецификациям. Нужно связать исходные документы, базу данных, программу для пользователей и сервис.

Шесть заданий и время

Задание	Время
1. Проектирование ER-диаграммы	20 мин
2. Создание базы и импорт	30 мин
3. Запрос стоимости заказа	20 мин
4. Информационная система	60 мин
5. API и тестовая коллекция	60 мин
6. Документация API	20 мин

Содержание

Раздел	Слайд
Подготовка	6
1. Проектирование ER	23
2. Создание БД и импорт	52
3. Расчёт стоимости	75
4. Формы и WPF	97
5. API и проверки	192
6. Документация API	234
Готовый пример	262

Что открыть перед занятием

1. Откройте КИМ на страницах 30–32 и скачайте приложения к официальному образцу. Проверьте, что открываются Excel, JSON, PDF и шаблон DOCX.
2. Проверьте Visual Studio с поддержкой .NET, PostgreSQL с pgAdmin, draw.io и Postman.
3. Распакуйте готовый пример отдельно от рабочей папки. В Visual Studio откройте решение или файл проекта. Перед входом подготовьте базу и учебные записи по инструкции запуска.

ЧТО ПРОВЕРИТЬ

Исходные документы и готовый пример открываются с одного учебного компьютера.

КИМ · PDF Исходные файлы · ZIP Готовый пример · ZIP

ПЕРЕД НАЧАЛОМ

Документы и задача

Перед вами пример про мебельное производство. Мы начнём с заказа на два стола «Самобранка» и дойдём до приложения, сервиса и документации.

ЧТО ПРОВЕРИТЬ

На рабочем столе есть папка **Surname + Subject Area**. Исходные файлы остаются отдельно от результатов.

ШАГ 1 ИЗ 9 · 1/2

Проверьте Windows и программы

Проверка среды заранее помогает понять, можно ли на этом компьютере собрать приложение и проверить API.

1. Откройте рабочий стол **Windows**. Если вы работаете на **Mac** через **Parallels**, сначала запустите свою виртуальную машину **Windows**.
2. Откройте «Пуск», введите **Visual Studio** и запустите программу.
3. В стартовом окне нажмите **Create a new project** («Создать проект»). Найдите шаблон **WPF Application** для **C#** на современном **.NET**. Перейдите по кнопке «Далее» до выбора платформы. Если в списке есть **.NET 10**, закройте мастер крестиком. На этом шаге достаточно увидеть платформу в списке.

ШАГ 1 ИЗ 9 · 2/2

Проверьте Windows и программы

1. В меню «Пуск» откройте **pgAdmin**. Слева раскройте **Servers**, дважды нажмите на настроенный учебный сервер **PostgreSQL** и введите пароль, если появится запрос. Раскройте **Databases**. Если сервера в списке нет или подключение не удалось, уточните параметры у преподавателя.
2. Откройте **Postman** через меню «Пуск». Если не удалось открыть программу или найти нужный шаблон или платформу, сообщите преподавателю, на каком действии возникла проблема.

ЧТО ПРОВЕРИТЬ

В **Visual Studio** доступен нужный шаблон **C#** и **.NET 10**. **Postman** открывается. Сервер базы данных отвечает в **phpMyAdmin** или **pgAdmin**, в зависимости от выбранного стека.

ШАГ 2 ИЗ 9 · 1/2

Определите задачу предприятия

Предприятие хранит заказ, состав изделия, цены и план выпуска в отдельных документах. Информационная система связывает эти сведения и поддерживает работу пользователей.

1. Откройте «КИМ · стр. 30» по ссылке в начале шага. Прочитайте задания на страницах 30–32.
2. Найдите в условии задания 1, что производит предприятие и на основании каких документов предстоит спроектировать систему.
3. Для каждого из шести заданий назовите, какой результат должен получиться. Сверьтесь со списком ниже.

ЧТО ПРОВЕРИТЬ

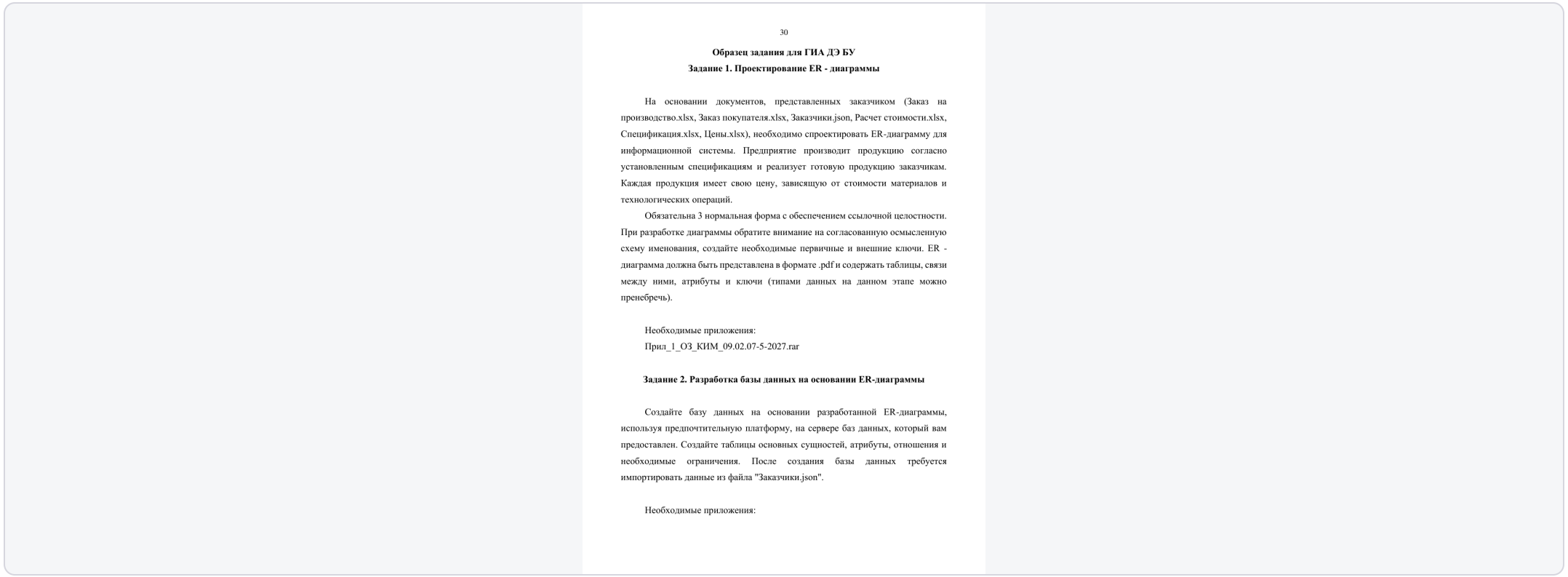
1 — ER-диаграмма в PDF. 2 — база данных с импортированными заказчиками. 3 — запрос расчёта стоимости заказа. 4 — настольное приложение с авторизацией и ролями. 5 — API и коллекция запросов с тестами. 6 — документация API по шаблону.

КИМ, стр. 30–32

ШАГ 2 ИЗ 9 · 2/2

Определите задачу предприятия

Покажите на экране



30

Образец задания для ГИА ДЭ БУ

Задание 1. Проектирование ER - диаграммы

На основании документов, представленных заказчиком (Заказ на производство.xlsx, Заказ покупателя.xlsx, Заказчики.json, Расчет стоимости.xlsx, Спецификация.xlsx, Цены.xlsx), необходимо спроектировать ER-диаграмму для информационной системы. Предприятие производит продукцию согласно установленным спецификациям и реализует готовую продукцию заказчикам. Каждая продукция имеет свою цену, зависящую от стоимости материалов и технологических операций.

Обязательна 3 нормальная форма с обеспечением ссылочной целостности. При разработке диаграммы обратите внимание на согласованную осмысленную схему именования, создайте необходимые первичные и внешние ключи. ER - диаграмма должна быть представлена в формате .pdf и содержать таблицы, связи между ними, атрибуты и ключи (типами данных на данном этапе можно пренебречь).

Необходимые приложения:
Прил_1_О3_КИМ_09.02.07-5-2027.rar

Задание 2. Разработка базы данных на основании ER-диаграммы

Создайте базу данных на основании разработанной ER-диаграммы, используя предпочтительную платформу, на сервере баз данных, который вам предоставлен. Создайте таблицы основных сущностей, атрибуты, отношения и необходимые ограничения. После создания базы данных требуется импортировать данные из файла "Заказчики.json".

Необходимые приложения:

КИМ 09.02.07-5-2027, страница 30: начало заданий ГИА БУ. Изображение оригинальной страницы PDF.

КИМ, стр. 30–32

ШАГ 3 ИЗ 9

Подготовьте собственное место работы

В одной рабочей папке проще найти все файлы, которые вы создадите на следующих этапах.

1. Перейдите на рабочий стол **Windows**. Если открыты окна, нажмите **Win+D**.
2. Нажмите правой кнопкой на свободном месте рабочего стола → «Создать» → «Папку».
3. Введите фамилию латиницей, знак «+» и название предметной области на английском. Например, **Ivanov + Furniture Manufacturing**. Нажмите **Enter**.
4. Откройте созданную папку двойным щелчком.

ЧТО ПРОВЕРИТЬ

На рабочем столе **Windows** есть папка, названная по шаблону **Surname + Subject Area**.

ШАГ 4 ИЗ 9

Сохраните исходные документы

Эти документы понадобятся для схемы, импорта заказчиков и расчёта стоимости.

1. Откройте свою рабочую папку на рабочем столе Windows. Нажмите Ctrl+Shift+N, введите «Исходные данные» и нажмите Enter.
2. Нажмите «Приложения · ZIP» в начале этого шага.
3. В Проводнике откройте папку «Загрузки» или папку, куда сохранили архив. Нажмите правой кнопкой по basic-inputs.zip и выберите «Извлечь всё».
4. Нажмите «Обзор», выберите созданную папку «Исходные данные» и нажмите «Извлечь». Откройте эту папку в Проводнике.

ЧТО ПРОВЕРИТЬ

В папке «Исходные данные» видны папки «Задание 1» — «Задание 6». Внутри «Задание 1» открывается «Заказ покупателя.xlsx».

Откройте: archive

ШАГ 5 ИЗ 9

Подготовьте папку для результатов

Готовые файлы будет проще найти, если хранить их отдельно от исходных документов.

1. Откройте свою рабочую папку на рабочем столе Windows.
2. Нажмите Ctrl+Shift+N, введите «Результаты» и нажмите Enter. Сюда будете сохранять диаграмму, SQL-файлы и другие результаты.
3. Убедитесь, что в рабочей папке рядом видны «Исходные данные» и «Результаты». Откройте «Результаты». Папка пока пуста.

ЧТО ПРОВЕРИТЬ

В рабочей папке рядом лежат «Исходные данные» с материалами КИМ и пустая папка «Результаты».

ШАГ 6 ИЗ 9 · 1/2

Прочитайте заказ покупателя

Номер, дата и покупатель относятся ко всему заказу. Наименование изделия, количество и цена относятся к отдельной строке товара.

1. Откройте «Заказ покупателя.xlsx» по ссылке в начале шага. Ищите значения по заголовкам столбцов. Если навести указатель на ячейку, появится её адрес.
2. В верхней части документа найдите номер и дату заказа, исполнителя и заказчика.
3. Найдите строку со столом «Самобранка». Прочитайте значения в столбцах «Код», «Кол-во», «Цена», «Скидка» и «Сумма». Если часть столбцов скрыта, прокрутите таблицу вправо.

ЧТО ПРОВЕРИТЬ

В заказе два стола по **14 120**. Скидка на позицию — **1 412**, итог к оплате — **26 828**.

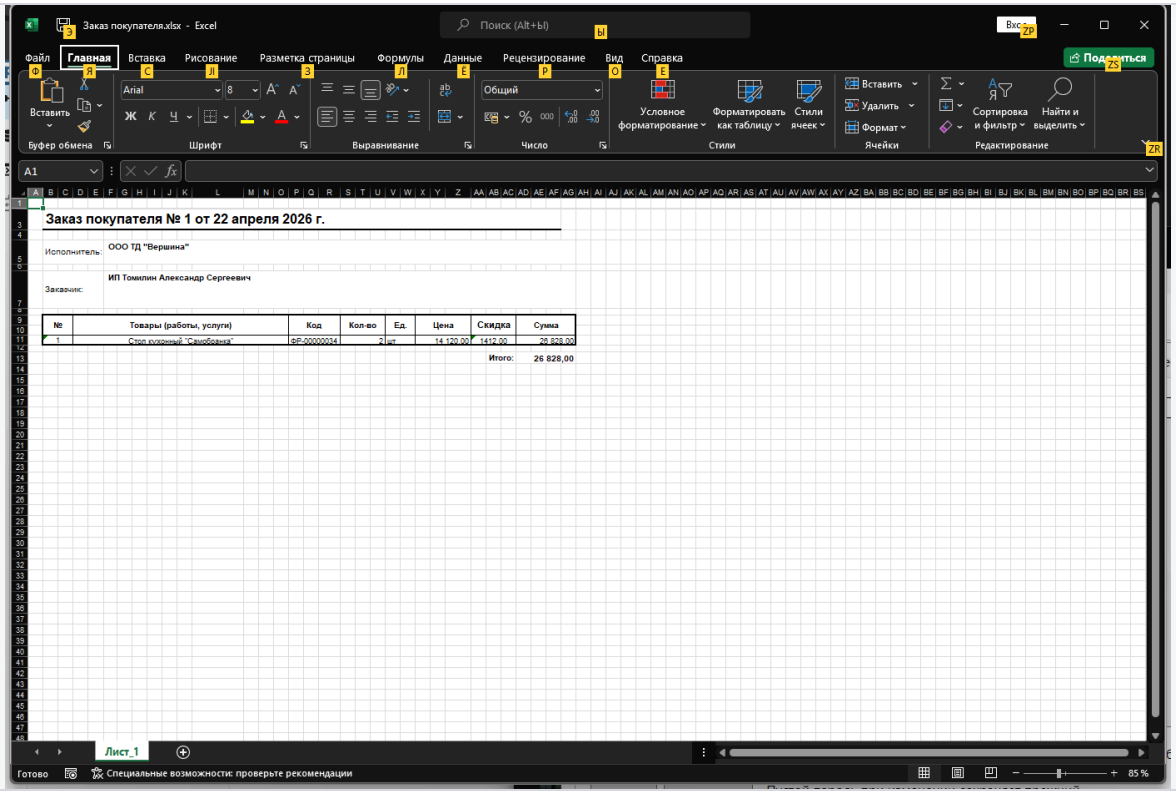
Заказ покупателя.xlsx, B3:F7, D11:AD11

Откройте: Заказ покупателя.xlsx

ШАГ 6 ИЗ 9 · 2/2

Прочитайте заказ покупателя

Покажите на экране



Заказ покупателя из приложения открыт в **Excel**. Маршрут: **WPF / PostgreSQL**.

Заказ покупателя.**xlsx**, **B3:F7, D11:AD11**

ШАГ 7 ИЗ 9 · 1/2

Прочитайте состав и цены

Спецификация показывает, сколько ресурсов нужно на один стол. Прайс показывает, сколько стоит единица каждого ресурса.

1. Откройте «Спецификация.xlsx». В разделе «Состав» найдите «Евровинт 6,5x5». Прочитайте его единицу измерения и количество.
2. Откройте «Цены.xlsx». Найдите то же наименование в столбце «Номенклатура» и прочитайте цену в столбце D.
3. Вернитесь к спецификации. В разделе «Технологическая операция» найдите «Сборка модулей» и прочитайте два значения: «Норма времени» и «Количество».

ЧТО ПРОВЕРИТЬ

На один стол требуется **0,012** тыс. шт. евровинтов, цена в прайсе — **595**. У сборки модулей норма времени **0,75** и количество **1**.

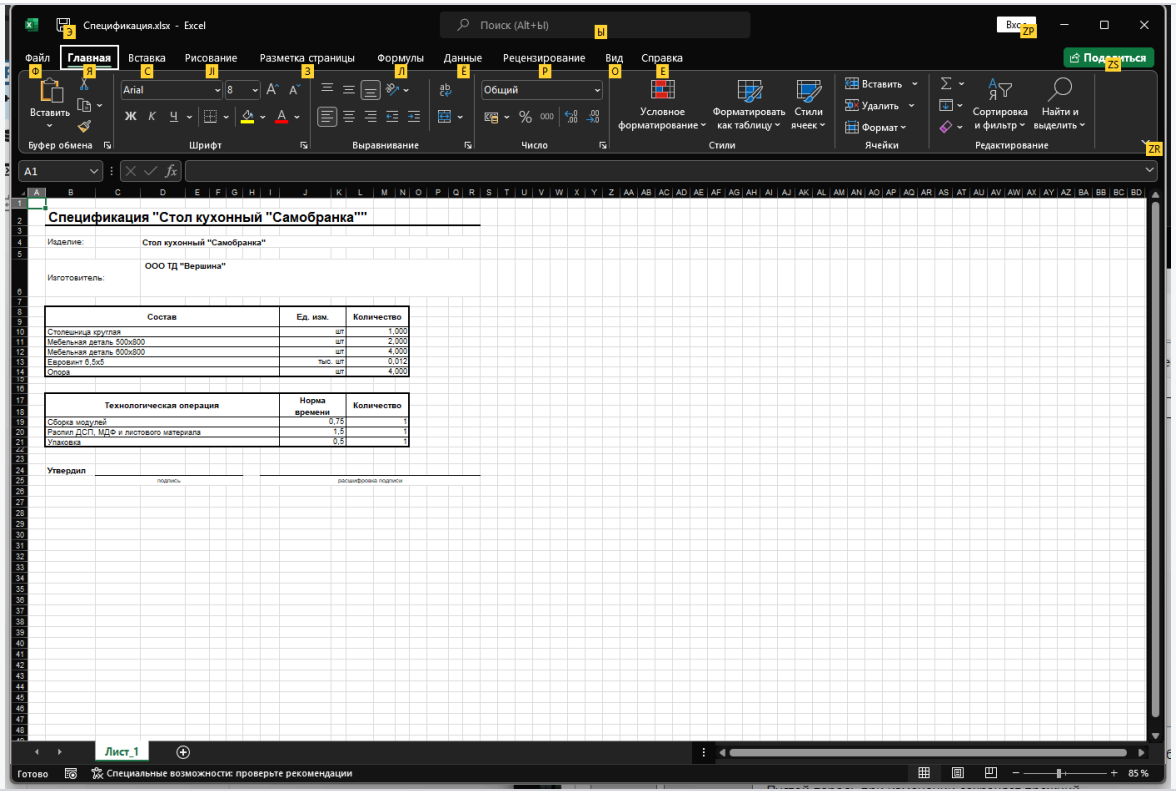
Спецификация.xlsx, B13:L13 и B19:L19 · Цены.xlsx, A2:D2

Откройте: Спецификация.xlsx · Цены.xlsx

ШАГ 7 ИЗ 9 · 2/2

Прочитайте состав и цены

Покажите на экране



Спецификация: материалы, количества и нормы технологических операций в **Excel**. Маршрут: **WPF / PostgreSQL**.

Спецификация.xlsx, B13:L13 и B19:L19 · Цены.xlsx, A2:D2

ШАГ 8 ИЗ 9 · 1/2

Проверьте JSON заказчиков

JSON — текстовый файл со списком записей. Каждая запись станет отдельной строкой справочника контрагентов.

1. Откройте «Заказчики.json». Каждая запись между фигурными скобками описывает одного контрагента.
2. В первой записи найдите **id** (код), **name** (название), **inn** (ИНН), **addres** (адрес), **phone** (телефон) и **type** (тип контрагента).
3. Посмотрите, что записано в **inn** у первого контрагента, и сравните значения **type** у разных записей. При импорте сохраняйте код с ведущими нулями как текст.

ЧТО ПРОВЕРИТЬ

В файле шесть записей. Первый код — **000000001**, первый ИНН пустой. В поле **type** встречаются «Поставщик» и «Покупатель».

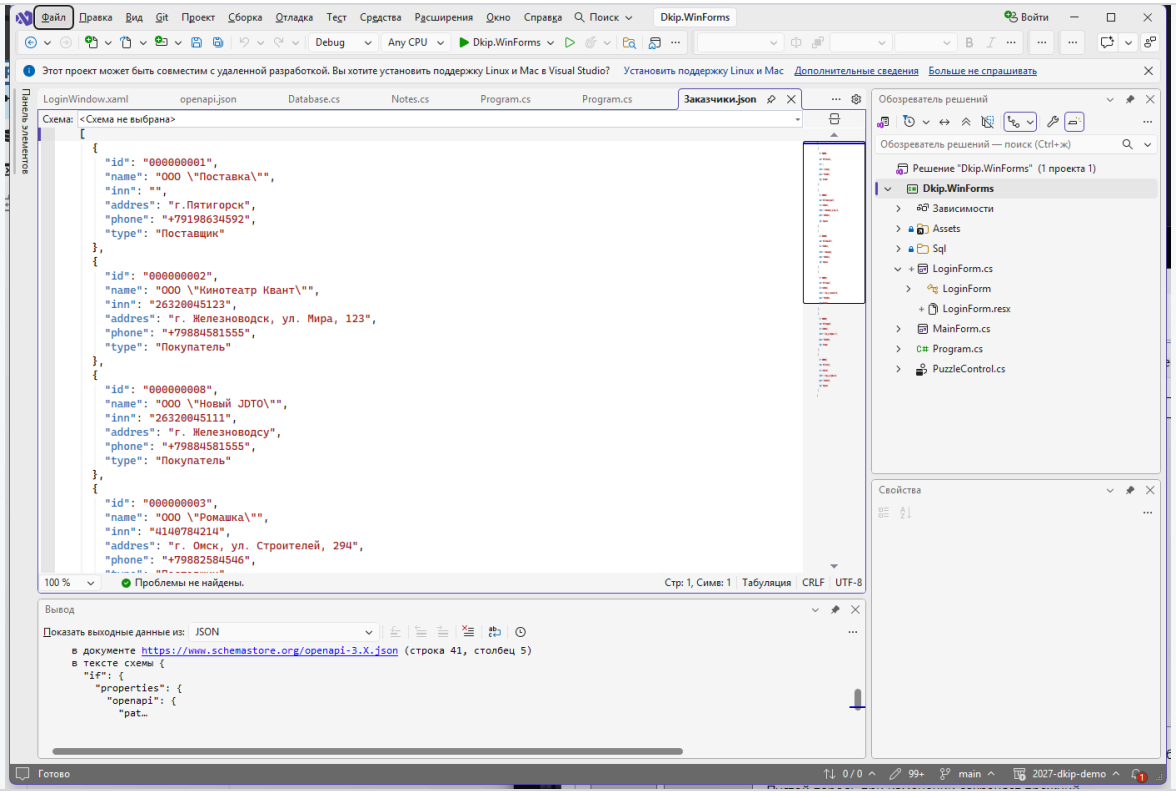
Заказчики.json

Откройте: Заказчики.json

ШАГ 8 ИЗ 9 · 2/2

Проверьте JSON заказчиков

Покажите на экране



Исходный Заказчики.**json** открыт в редакторе без изменения значений. Маршрут: **WPF / PostgreSQL**.

Заказчики.**json**

ШАГ 9 ИЗ 9 · 1/2

Сравните документы

Название, код и единица измерения могут различаться между документами. При проектировании важно сохранить значения из каждого источника и пояснить, как вы их сопоставили.

1. Откройте «Заказ покупателя.xlsx» и «Заказ на производство.xlsx». В каждом найдите строку со столом «Самобранка» и сравните столбцы «Код» и «Количество». В обоих документах указано два стола, но коды разные.
2. Откройте «Спецификация.xlsx». Сравните единицы измерения евровинта и опоры в разделе «Состав» с их единицами в разделе «Потребность в материалах» заказа на производство.

ШАГ 9 ИЗ 9 · 2/2

Сравните документы

1. Откройте «Расчет стоимости.xlsx». Найдите «Итого» и сравните с суммой к оплате из заказа покупателя. В первом файле указан исходный расчёт затрат, во втором — сумма продажи со скидкой. Свой расчёт затрат вы выполните в задании 3.

ЧТО ПРОВЕРИТЬ

В заказе покупателя код ФР-**00000034**, в заказе на производство — НФ-**00000006**. В учебном примере это одно изделие по совпадению названия, оба исходных кода сохраняются. Евровинт и опора в спецификации указаны в тыс. шт. и шт. соответственно, в заказе на производство — наоборот. Исходный расчёт затрат — **11 448,12**, сумма к оплате — **26 828**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx** · Спецификация.**xlsx** · Расчет стоимости.**xlsx**

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

На рабочем столе есть папка **Surname + Subject Area**. Исходные файлы остаются отдельно от результатов.

ЧТО ПРОВЕРИТЬ

Рабочая папка с исходными документами и отдельным местом для результатов.

ЗАДАНИЕ 1 · 20 МИН

Проектирование ER

Из документов выделим сущности — то, о чём надо хранить отдельные записи. Затем свяжем их ключами.

ЧТО ПРОВЕРИТЬ

ER-диаграмма.**pdf** открывается и читается. ER-диаграмма.**drawio** остаётся для правок.

УСЛОВИЕ ИЗ КИМ · 1/2

Проектирование ER-диаграммы

На основании документов, представленных заказчиком (Заказ на производство.xlsx, Заказ покупателя.xlsx, Заказчики.json, Расчет стоимости.xlsx, Спецификация.xlsx, Цены.xlsx), необходимо спроектировать ER-диаграмму для информационной системы.

Предприятие производит продукцию согласно установленным спецификациям и реализует готовую продукцию заказчикам. Каждая продукция имеет свою цену, зависящую от стоимости материалов и технологических операций.

УСЛОВИЕ ИЗ КИМ · 2/2

Проектирование ER-диаграммы

Обязательна 3 нормальная форма с обеспечением ссылочной целостности. При разработке диаграммы обратите внимание на согласованную осмысленную схему именования, создайте необходимые первичные и внешние ключи.

ER-диаграмма должна быть представлена в формате .pdf и содержать таблицы, связи между ними, атрибуты и ключи (типами данных на данном этапе можно пренебречь).

Приложение: Прил_1_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 30 Исходные файлы · ZIP

ШАГ 1 ИЗ 12

Откройте draw.io

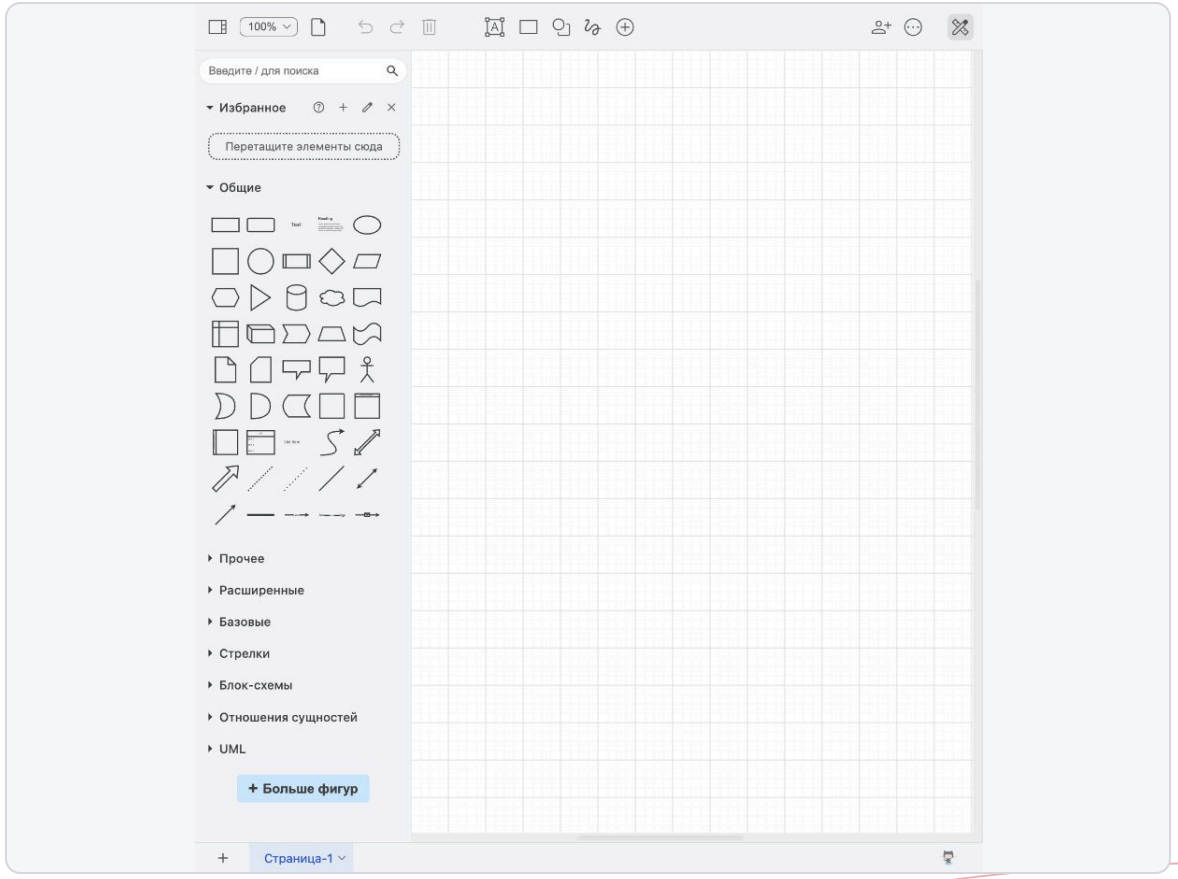
Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в draw.io и сверьте ключи и связи с учебным примером.

1. Для тренировки откройте draw.io по ссылке выше. На экзамене будут установлены настольный draw.io и Microsoft Visio.
2. Если редактор предлагает выбрать место хранения, нажмите «Устройство» (Device). Создайте пустую диаграмму.

ЧТО ПРОВЕРИТЬ

Открыт редактор **draw.io**. На холсте пока нет фигур, слева видна панель фигур.

Откройте: drawio



Браузерный **draw.io** с пустой диаграммой. Слева видна панель фигур, внизу кнопка «Больше фигур».

ШАГ 2 ИЗ 12 · 1/2

Подключите библиотеку ER

Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в **draw.io** и сверьте ключи и связи с учебным примером.

1. Внизу левой панели нажмите «Больше фигур» (**More Shapes**).
2. В разделе «Программное обеспечение» (**Software**) найдите и отметьте «Отношения сущностей» (**Entity Relation**). Если галочка уже стоит, оставьте её.
3. Нажмите «Применить» (Apply). В левой панели раскройте «Отношения сущностей», чтобы увидеть фигуры таблиц и связей.

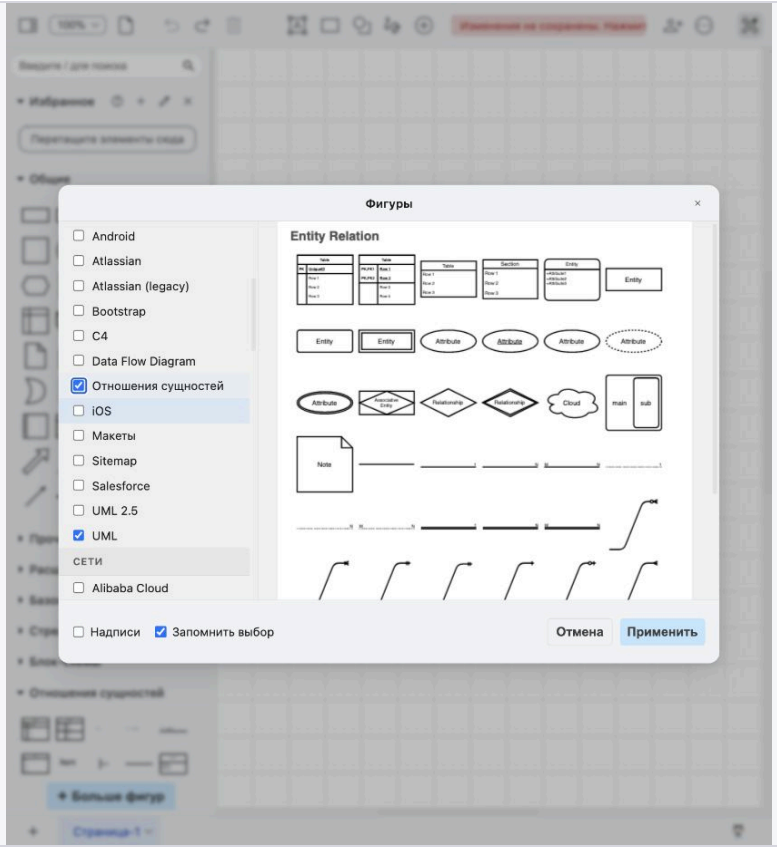
ЧТО ПРОВЕРИТЬ

В левой панели раскрыт раздел «Отношения сущностей». В нём видны фигуры таблиц и линий связи.

ШАГ 2 ИЗ 12 · 2/2

Подключите библиотеку ER

Покажите на экране



В окне «Фигуры» выбрана библиотека «Отношения сущностей». Внизу справа кнопка «Применить».

ШАГ 3 ИЗ 12

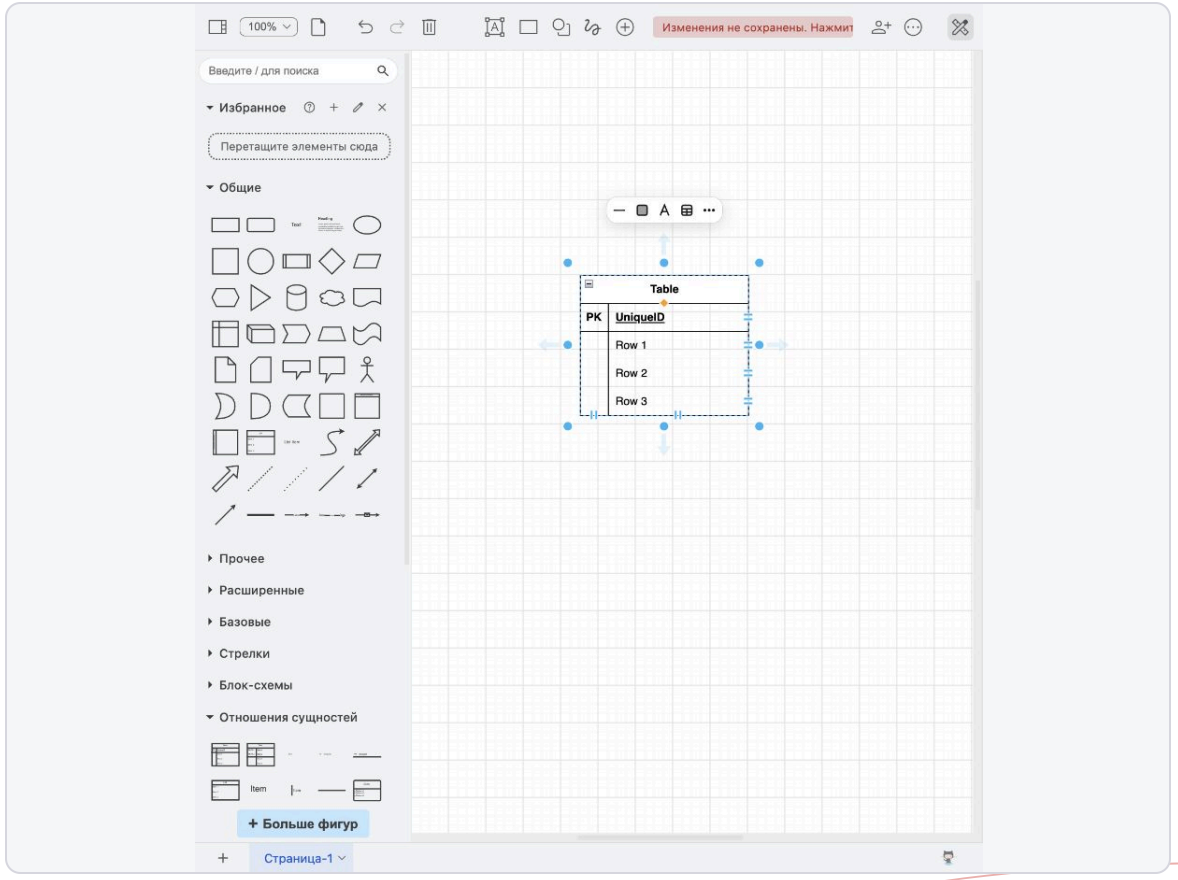
Добавьте фигуру таблицы

Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в draw.io и сверьте ключи и связи с учебным примером.

1. В библиотеке «Отношения сущностей» найдите первую фигуру **Table**. На значке видны заголовок таблицы и строки.
2. Перетащите фигуру на пустой холст. На следующем шаге замените заголовок **Table** и названия строк своими.

ЧТО ПРОВЕРИТЬ

На холсте появилась фигура **Table** с заголовком и строками, включая строку **PK**.



Фигура **Table** из библиотеки «Отношения сущностей» на холсте. Под заголовком видна строка **PK**.

ШАГ 4 ИЗ 12 · 1/3

Отделите сущности от документов

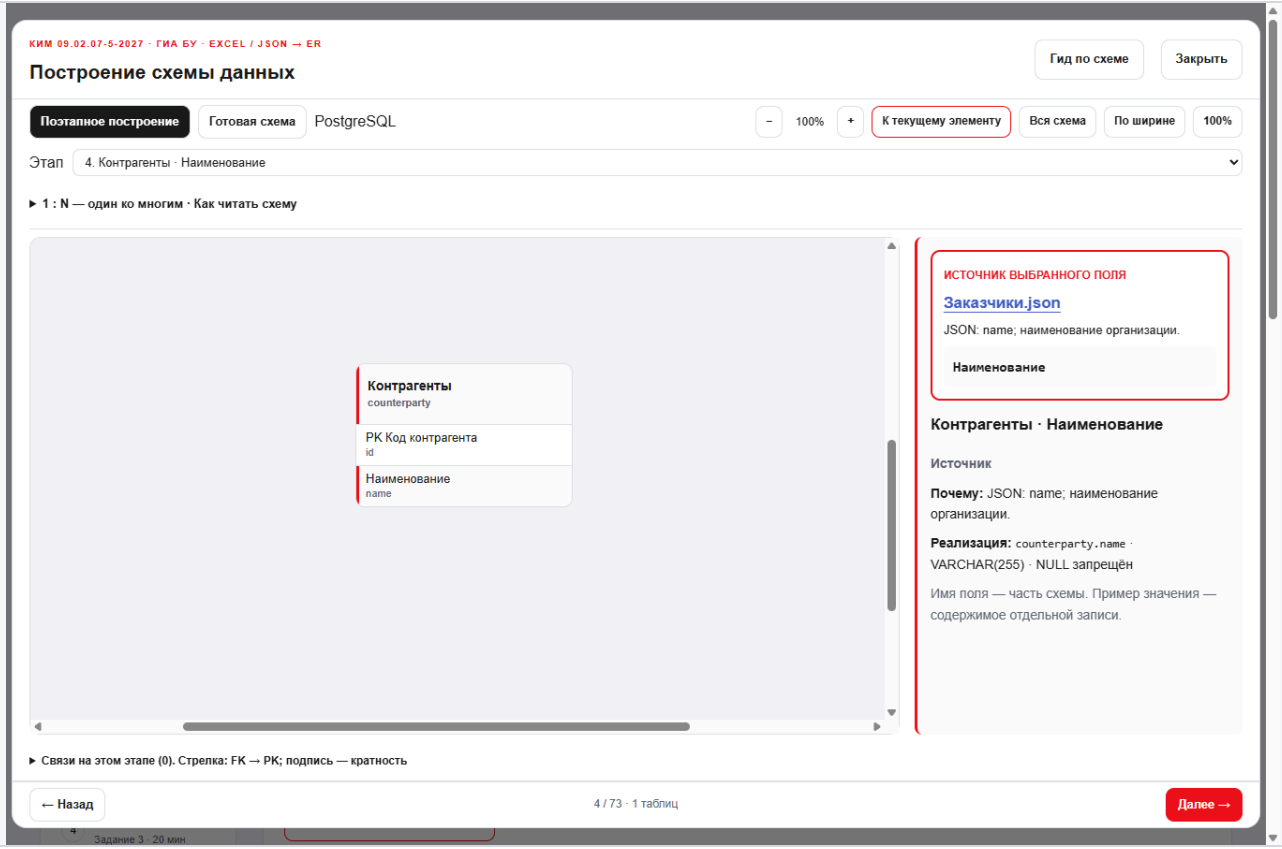
Из одного документа могут получиться несколько таблиц. Сначала выясните, к чему относится каждое поле.

1. Откройте «Заказ покупателя.xlsx». Номер и дата в заголовке и имя покупателя относятся ко всему заказу. Название стола, количество и цена относятся к одной строке товара.
2. Откройте «Заказчики.json». Дважды нажмите заголовок фигуры таблицы в **draw.io** и введите **counterparty**. По первой записи заполните строки. Отметьте **id** как **PK**, ниже добавьте **name**, **inn**, **address**, **phone** и **party_type**.

ШАГ 4 ИЗ 12 · 2/3

Отделите сущности от документов

Покажите на экране



Поэтапный разбор ER: отдельное поле, исходный документ и объяснение справа. Маршрут: WPF / PostgreSQL.

ШАГ 4 ИЗ 12 · 3/3

Отделите сущности от документов

1. Если строк не хватает, нажмите правой кнопкой по существующей строке и выберите «Дублировать» (**Duplicate**). Затем измените название в новой строке.
2. Сверьте имена с **JSON**. Поле **addres** соответствует **address**, а **type** — **party_type**. Значения, в том числе код с ведущими нулями и пустой ИНН, остаются в исходном файле.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть полей. Ключ **id** обозначает одного контрагента. Вы можете показать, где в заказе шапка, а где строка товара.

Откройте: Заказ покупателя.**xlsx** · Заказчики.**json**

ШАГ 5 ИЗ 12 · 1/3

Добавьте номенклатуру и цены

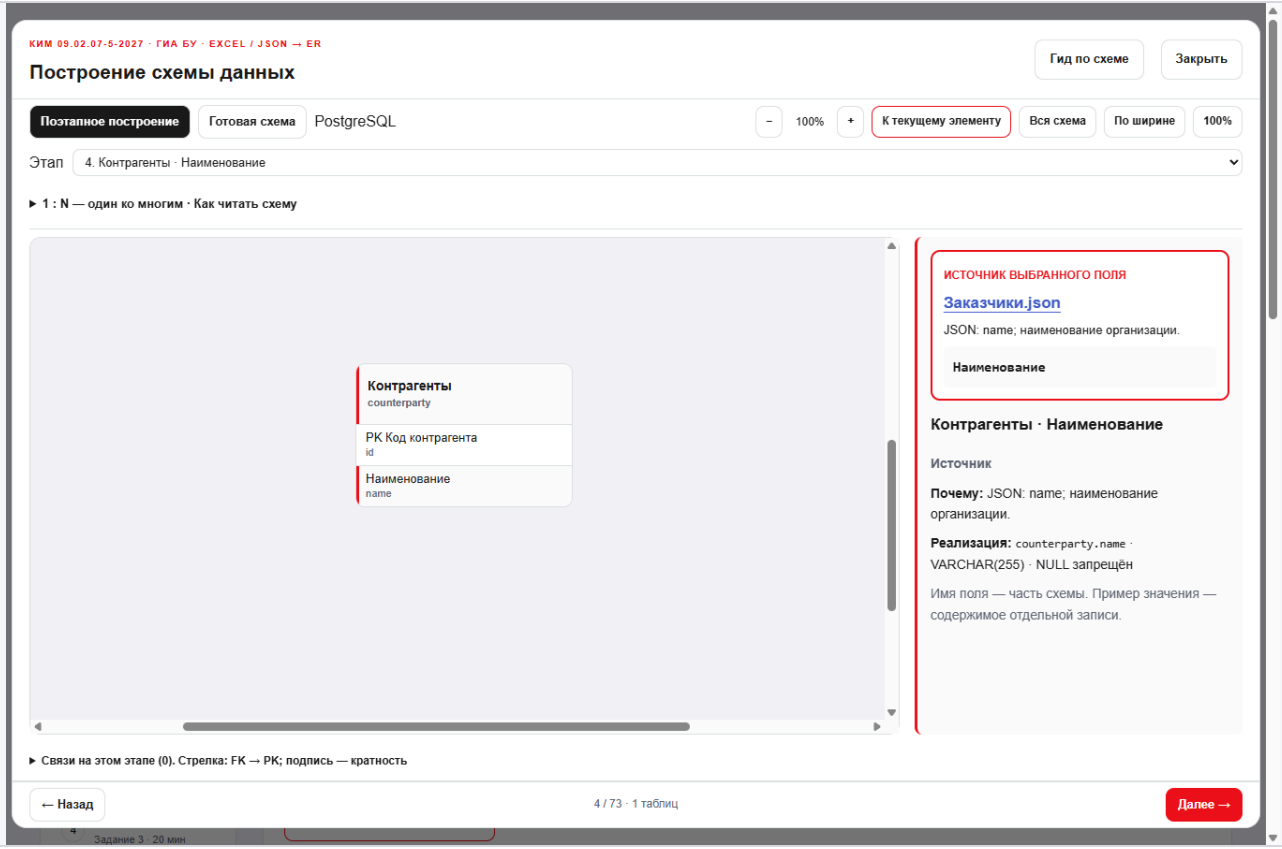
Один ресурс может встречаться в разных спецификациях. Его название хранится в номенклатуре, а цена может меняться со временем.

1. Откройте «Заказ на производство.xlsx». Найдите евровинт в разделе «Потребность в материалах» и его код ФР-00000027. Добавьте таблицу **item** с PK **id** и полями **code**, **name**, **kind**, **unit**.
2. Поле **kind** различает изделие (**product**), материал (**material**) и операцию (**operation**). У стола вид **product**, у евровинта **material**, у сборки **operation**.
3. В «Цены.xlsx» найдите «Евровинт 6,5x5» и цену 595. Добавьте таблицу **price** с полями **item_id**, **valid_from** и **amount**. Проведите связь **price.item_id** → **item.id**.

ШАГ 5 ИЗ 12 · 2/3

Добавьте номенклатуру и цены

Покажите на экране



Поэтапный разбор ER: отдельное поле, исходный документ и объяснение справа. Маршрут: WPF / PostgreSQL.

ШАГ 5 ИЗ 12 · 3/3

Добавьте номенклатуру и цены

1. Пометьте **item_id** и **valid_from** как части одного PK. Даты в прайсе нет. В учебном расчёте используем 01.04.2026 как дату начала действия цен.

ЧТО ПРОВЕРИТЬ

У каждой цены есть ссылка на ресурс. Для одного ресурса можно записать цены с разными датами, и пара «ресурс + дата» уникальна.

Откройте: Заказ на производство.**xlsx** · Цены.**xlsx**

ШАГ 6 ИЗ 12 · 1/3

Добавьте спецификацию и состав

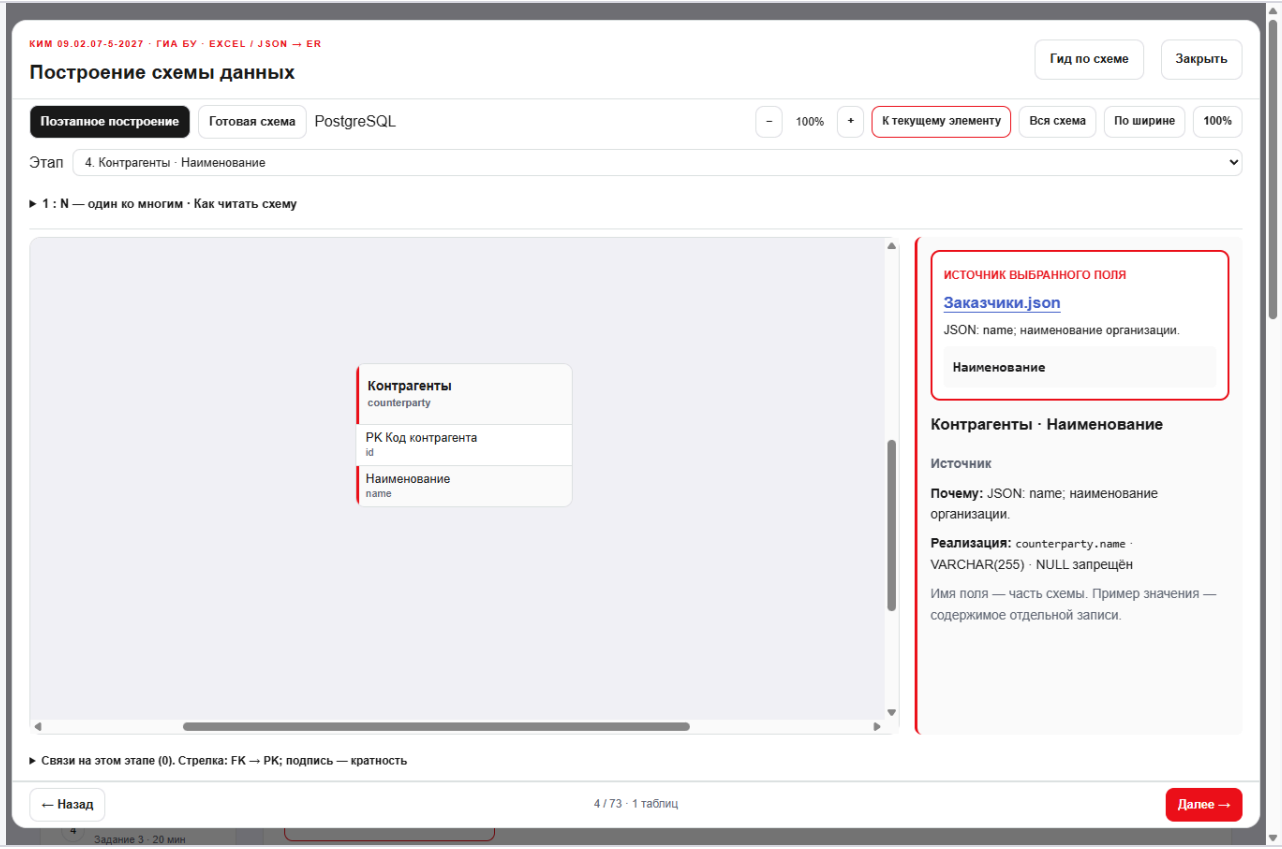
Шапка описывает спецификацию целиком. В её составе каждая строка связывает спецификацию с одним материалом или операцией.

1. Откройте «Спецификация.xlsx». В шапке найдите изделие, изготовителя и название. Добавьте **specification** с PK **id**, полями **name** и **output_qty**. В примере нормы заданы на один стол.
2. Свяжите **product_id** → **item.id** и **manufacturer_id** → **counterparty.id**. Пометьте **product_id** как **UNIQUE**, чтобы у изделия могла быть максимум одна спецификация.
3. В разделе «Состав» найдите евровинт. Добавьте **specification_component**. Проведите связи **specification_id** → **specification.id** и **item_id** → **item.id**. Эти два поля вместе образуют PK.

ШАГ 6 ИЗ 12 · 2/3

Добавьте спецификацию и состав

Покажите на экране



Поэтапный разбор **ER**: отдельное поле, исходный документ и объяснение справа. Маршрут: **WPF / PostgreSQL**.

ШАГ 6 ИЗ 12 · 3/3

Добавьте спецификацию и состав

1. Добавьте **qty** для количества по норме и **time_norm** для нормы времени. Для «Сборки модулей» количество равно 1, норма времени — 0,75. Для материалов в примере используем множитель времени 1.

ЧТО ПРОВЕРИТЬ

Строка с евровинтом относится к спецификации стола и ссылается на евровинт в **item**. В другой спецификации у того же евровинта может быть своё количество.

Откройте: Спецификация.**xlsx**

ШАГ 7 ИЗ 12 · 1/4

Добавьте заказ и план производства

Заказ покупателя

Шапка хранит сведения о документе. Отдельные строки хранят изделия и ресурсы, которые можно связать с номенклатурой.

1. По заголовку заказа добавьте **customer_order** с PK **id**. В шапке разместите номер и дату заказа. Поля **customer_id** и **executor_id** свяжите с **counterparty.id**.
2. По строке с двумя столами добавьте **customer_order_line** с PK **id**. Свяжите **order_id** → **customer_order.id** и **product_id** → **item.id**. Здесь же разместите количество, продажную цену, скидку и исходный код ФР-00000034 в **source_code**.

ШАГ 7 ИЗ 12 · 2/4

Добавьте заказ и план производства

Заказ на производство

1. По заголовку и шапке добавьте **production_order** с PK **id**, номером документа, датой, датой запуска и подразделением.
2. Раздел «Продукция, работа» перенесите в модель как **production_product**, разделы материалов и операций — как **production_resource**. В обеих таблицах свяжите **production_id** с **production_order.id**, а изделие или ресурс — с **item.id**. Каждая пара внешних ключей образует PK своей таблицы.

ШАГ 7 ИЗ 12 · 3/4

Добавьте заказ и план производства

Заказ на производство

1. Покажите количество в строках выпуска и плана. В **production_resource.source_unit** сохраните единицу из плана. Например, для упаковки в плане указаны штуки, а в учебном расчёте тариф принят за час.

ШАГ 7 ИЗ 12 · 4/4

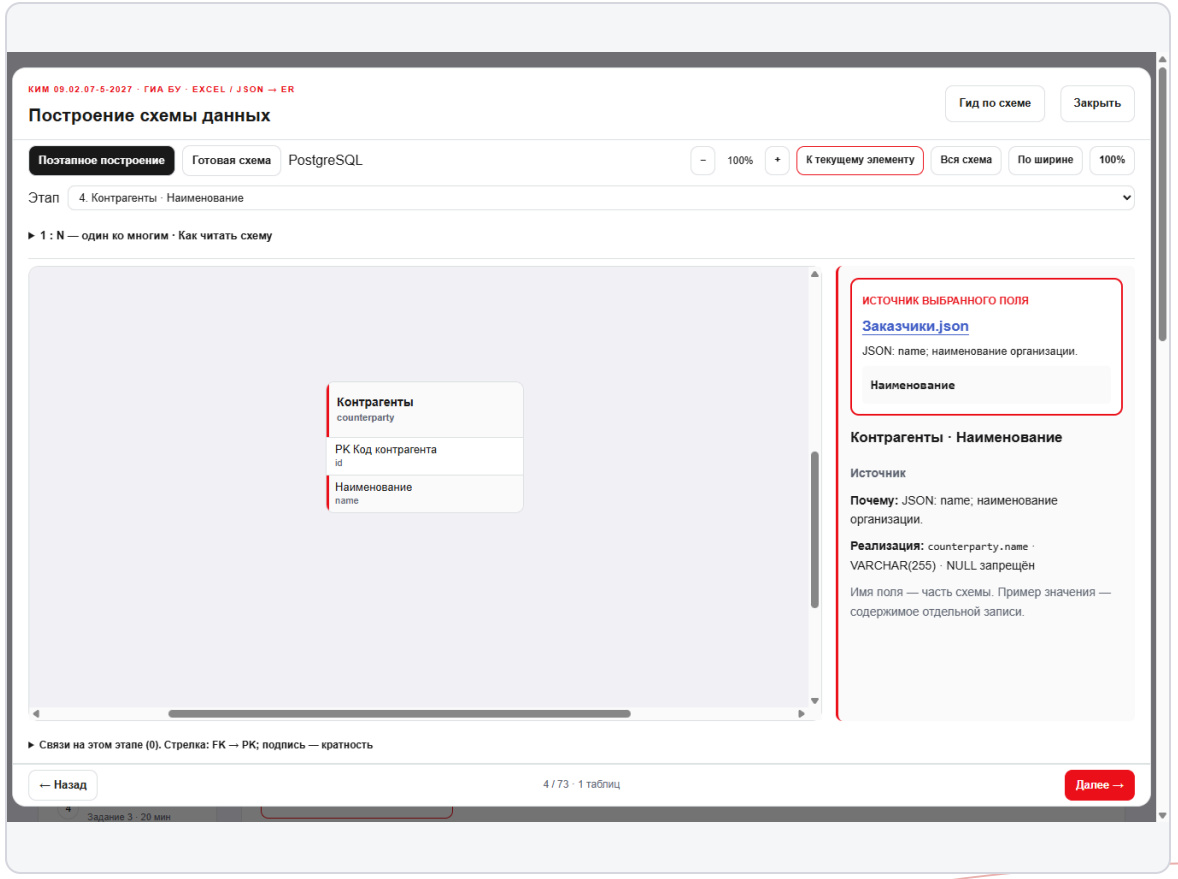
Добавьте заказ и план производства

Покажите результат на экране

ЧТО ПРОВЕРИТЬ

Строка продажи ссылается на заказ покупателя. Строка выпуска и строки ресурсов ссылаются на свой производственный заказ. Оба документа о столе связаны с одной записью в **item**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx**



Поэтапный разбор **ER**: отдельное поле, исходный документ и объяснение справа. Маршрут: **WPF / PostgreSQL**.

ШАГ 8 ИЗ 12 · 1/2

Добавьте пользователей

Таблица **users** дополняет учебную схему для входа и ролей в задании 4. У автора заметки будет ссылка на эту учётную запись.

1. Откройте Приложение 1 к заданию 4. Найдите требования к входу, ролям и блокировке.
2. Добавьте **users** с PK **id**. Для входа отметьте уникальный **login** и поле **password_hash** для хеша пароля.
3. Добавьте **role** для роли, **failed_attempts** для числа неудачных попыток и **is_locked** для состояния блокировки.

ЧТО ПРОВЕРИТЬ

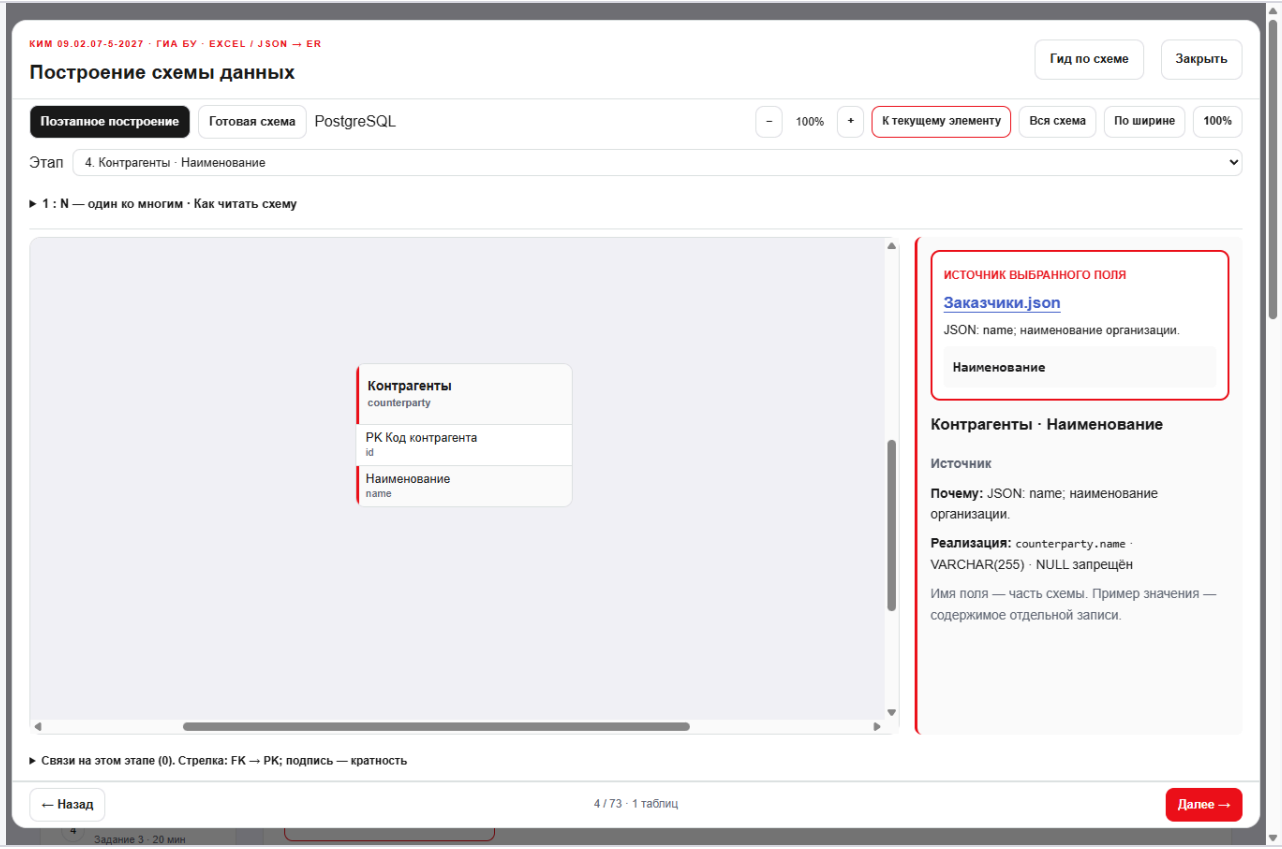
На схеме видны ключ **id**, уникальный логин, хеш пароля, роль и два поля, по которым приложение ведёт блокировку.

Откройте: Приложение **1.pdf**

ШАГ 8 ИЗ 12 · 2/2

Добавьте пользователей

Покажите на экране



Поэтапный разбор **ER**: отдельное поле, исходный документ и объяснение справа. Маршрут: **WPF / PostgreSQL**.

ШАГ 9 ИЗ 12 · 1/2

Добавьте заметки

Таблица **notes** дополняет учебную схему для задания 5. Через автора заметки API получает его логин.

1. В приложении к заданию 5 найдите раздел «Структура таблицы базы данных». В **notes** добавьте поля **id**, **title**, **content**, **id_user** и **created_at**.
2. Отметьте **id** как **PK**, **id_user** как **FK**. Проведите связь **notes.id_user** → **users.id**. Каждая заметка имеет одного автора. У пользователя может быть несколько заметок.
3. В ответе API **title_user** составляется из заголовка и логина, а **formatted_date** получается из даты создания. На схеме **notes** остаются исходные поля.

ЧТО ПРОВЕРИТЬ

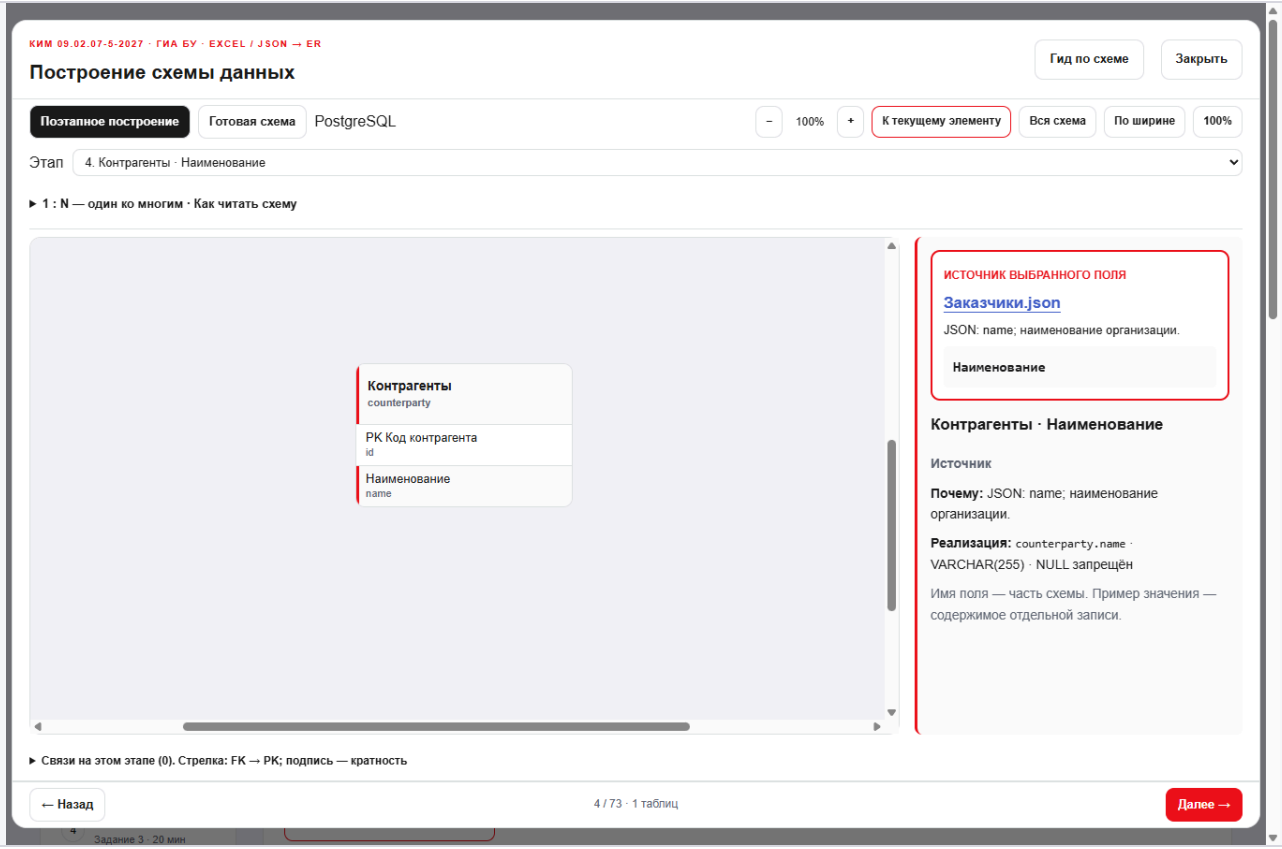
В **notes** пять полей, а связь автора заканчивается на **users.id**. Поля ответа API можно получить из **notes** и **users**.

Откройте: Прил_5_ОЗ_КИМ_09.02.07-5-2027.pdf

ШАГ 9 ИЗ 12 · 2/2

Добавьте заметки

Покажите на экране



Поэтапный разбор **ER**: отдельное поле, исходный документ и объяснение справа. Маршрут: **WPF / PostgreSQL**.

ШАГ 10 ИЗ 12

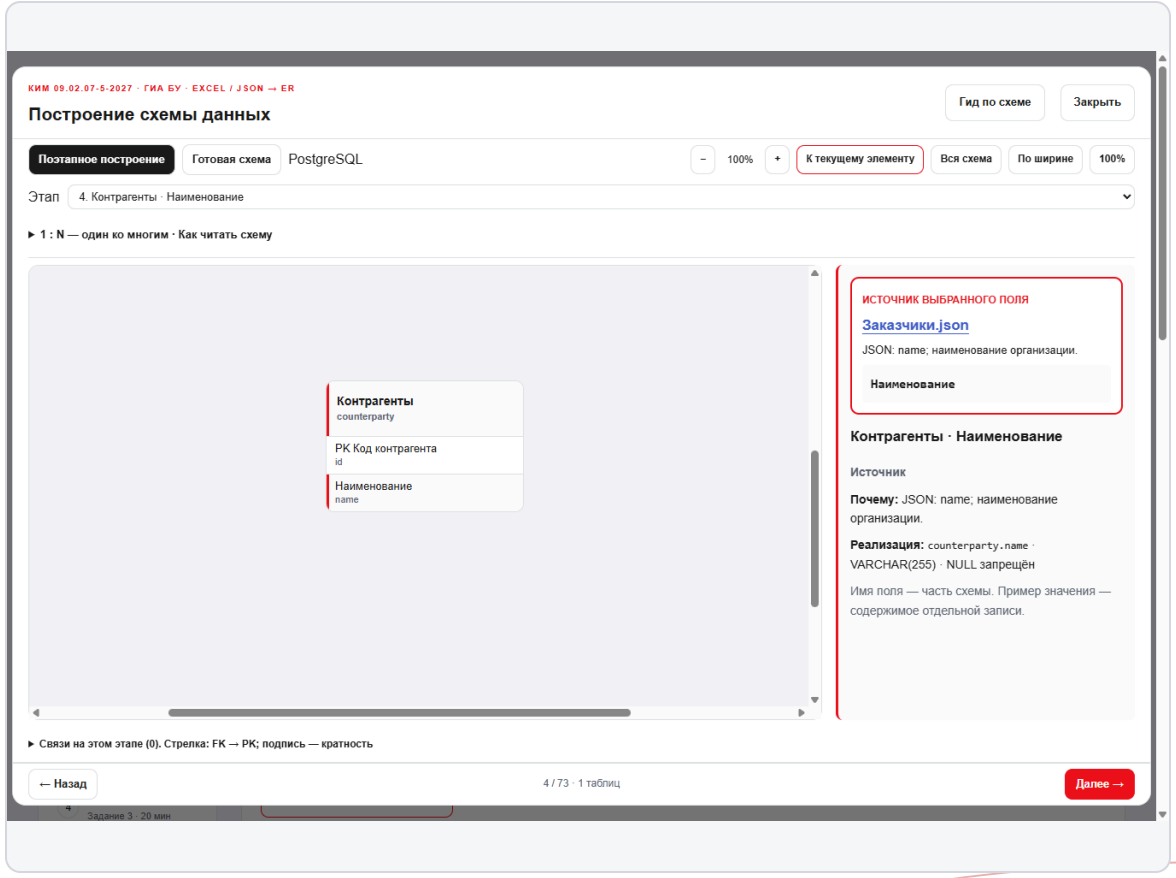
Сверьте свою ER-диаграмму

В разборе ниже выберите нужное поле в режиме «Поэтапное построение». Чтобы увидеть все таблицы, переключитесь на «Готовая схема».

1. Строки заказа → Заказ. Проверьте, что позиция с товаром ведёт к шапке заказа.
2. Состав спецификации → Материал / операция. Проверьте, что строка состава ведёт к одному ресурсу из номенклатуры.
3. Заметки → Автор. Проверьте связь id_user с ключом пользователя.

ЧТО ПРОВЕРИТЬ

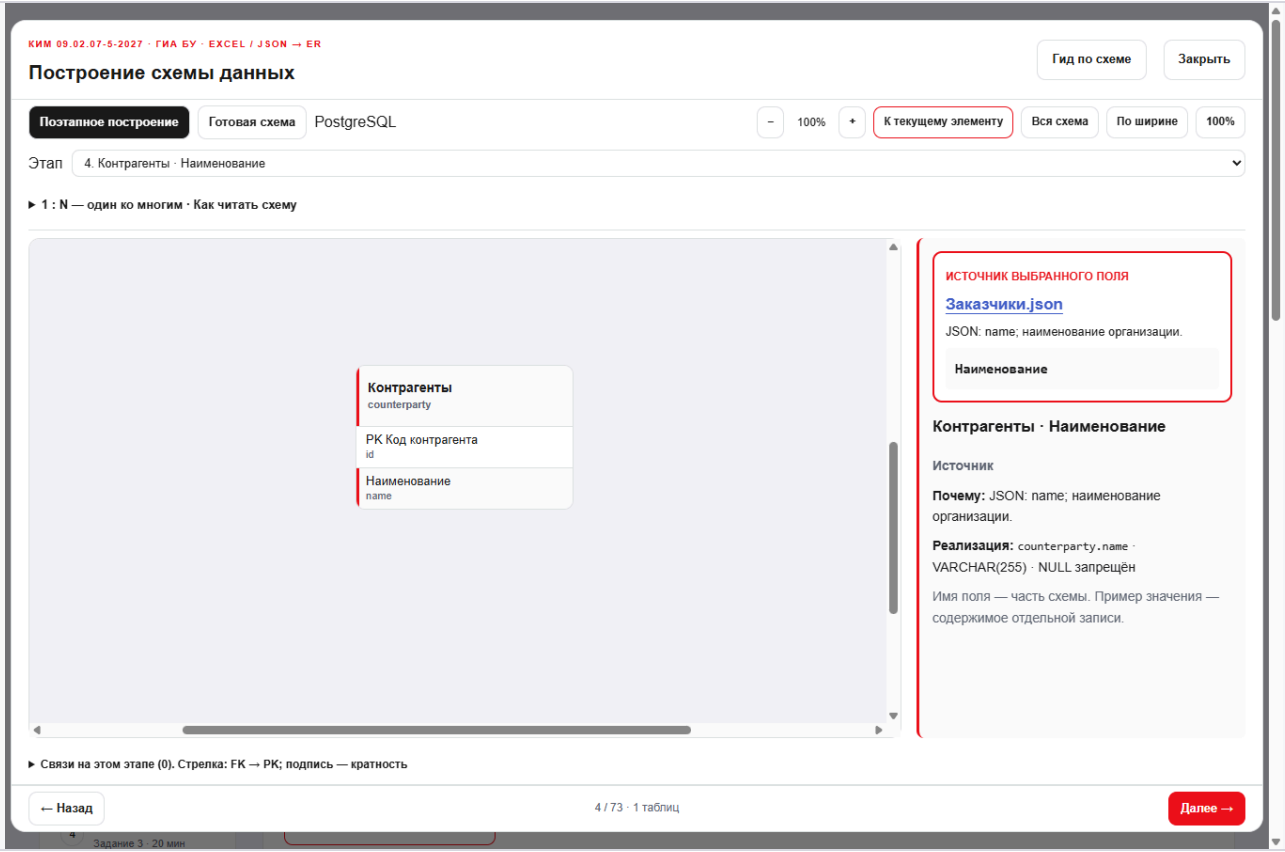
На своей схеме вы можете проследить эти три связи от внешнего ключа до первичного.



Поэтапный разбор ER: отдельное поле, исходный документ и объяснение справа. Маршрут: WPF / PostgreSQL.

ШАГ 10 · СХЕМА

Общая ER-диаграмма



Поэтапный разбор **ER**: отдельное поле, исходный документ и объяснение справа. Маршрут: **WPF / PostgreSQL**.

ER-диаграмма · PDF

ШАГ 11 ИЗ 12

Проверьте нормализацию и связи

Проверьте таблицы и ключи на редактируемой диаграмме перед экспортом.

1. Проверьте, что в одном поле хранится одно значение, а позиции заказов и строки состава вынесены отдельно.
2. Проследите зависимости. Телефон относится к контрагенту и хранится в **counterparty**. Количество евровинтов зависит от пары «спецификация + ресурс» и хранится в строке состава.
3. Для каждой связи найдите **FK** и соответствующий **PK**. Обозначьте кратность, в том числе «один к нулю или одному» между изделием и спецификацией в учебной схеме.

ЧТО ПРОВЕРИТЬ

В схеме обозначены ключи и кратности. Контакты хранятся в справочнике, а количество материала — в строке состава.

КИМ, стр. 30

ШАГ 12 ИЗ 12

Экспорт в draw.io

По итоговому PDF преподаватель проверяет таблицы, поля, ключи и связи. Типы данных на этом этапе необязательны.

1. Сохраните редактируемую схему в папку «Результаты» под именем ER-диаграмма.drawio. Выберите «Файл» → «Экспортировать как» → PDF.
2. Если схема занимает несколько страниц, выберите экспорт всех страниц. Сохраните ER-диаграмма.pdf в папку «Результаты».
3. Откройте PDF. Увеличьте масштаб и проверьте, что все таблицы на месте, названия полей и ключей читаются, а обозначения связей видны полностью.

ЧТО ПРОВЕРИТЬ

В папке «Результаты» открывается ER-диаграмма.pdf. В нём видны таблицы, поля, PK, FK и кратности.

КИМ, стр. 30

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

ER-диаграмма.pdf открывается и читается. ER-диаграмма.drawio остаётся для правок.

ЧТО ПРОВЕРИТЬ

ER-диаграмма.pdf с таблицами, полями, ключами и связями.

ЗАДАНИЕ 2 · 30 МИН

Создание БД и импорт

Схема из задания 1 становится таблицами в PostgreSQL. Импортируем клиентов после создания таблицы.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть строк. Первый **id** — **000000001**, ведущие нули сохранены. Есть файл выгрузки **dkip2027-postgresql.sql**.

УСЛОВИЕ ИЗ КИМ

Разработка базы данных на основании ER-диаграммы

Создайте базу данных на основании разработанной ER-диаграммы, используя предпочтительную платформу, на сервере баз данных, который вам предоставлен. Создайте таблицы основных сущностей, атрибуты, отношения и необходимые ограничения.

После создания базы данных требуется импортировать данные из файла "Заказчики.json".

Приложение: Прил_2_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 30–31 Исходные файлы · ZIP

ШАГ 1 ИЗ 10 · 1/2

Откройте редактор базы данных

pgAdmin подключается к запущенному серверу PostgreSQL.

1. Откройте pgAdmin и раскройте Servers. Дважды нажмите настроенный учебный сервер. Введите пароль, если pgAdmin попросит.
2. Если подключения нет, нажмите правой кнопкой на Servers → Register → Server. На вкладке Connection укажите адрес, порт и имя пользователя, выданные для вашего сервера. Нажмите Save.
3. Раскройте Databases. Убедитесь, что соединение открыто.

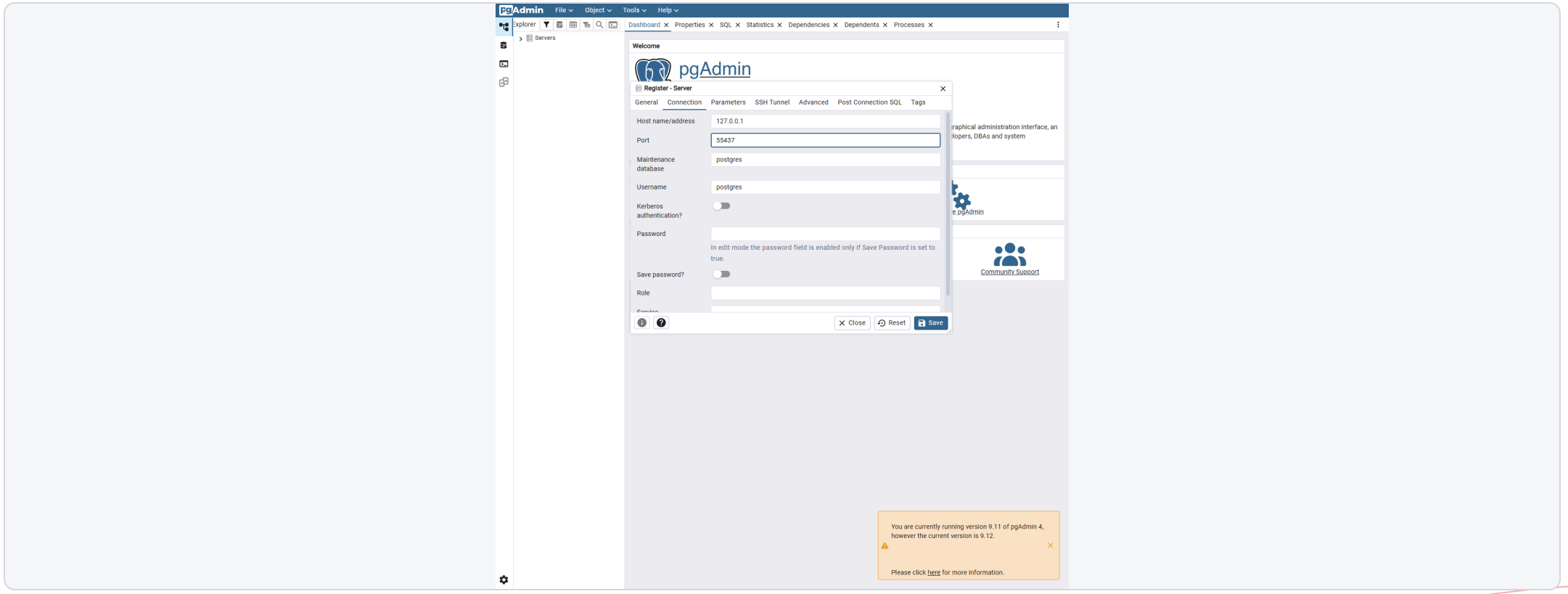
ЧТО ПРОВЕРИТЬ

Редактор показывает список баз данных. Теперь можно создать свою учебную базу.

ШАГ 1 ИЗ 10 · 2/2

Откройте редактор базы данных

Покажите на экране



pgAdmin: регистрация подключения к локальному **PostgreSQL**. На снимке отдельный учебный сервер с портом **55437**; укажите порт своего сервера, обычно **5432**.

ШАГ 2 ИЗ 10

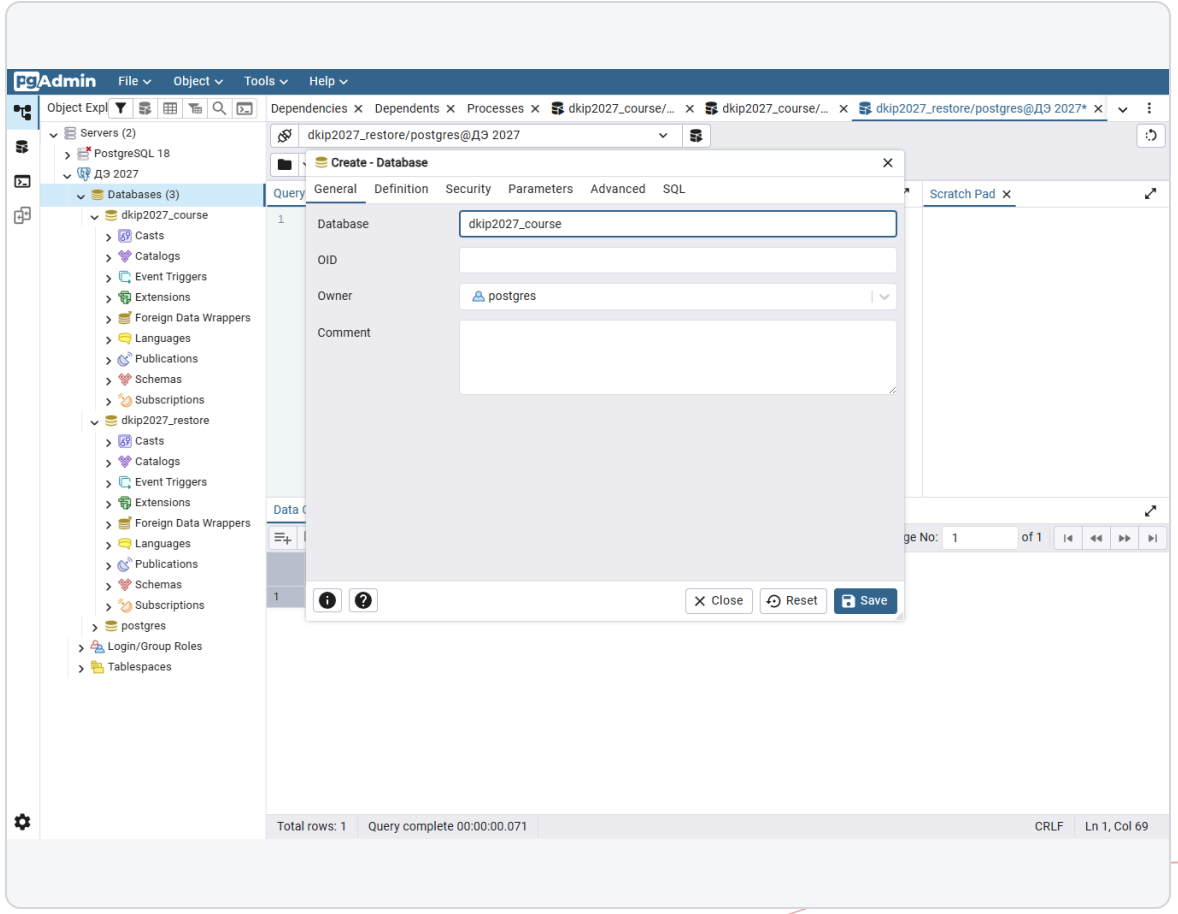
Создайте пустую учебную базу

Имя выбранной базы помогает проверить, куда попадут таблицы и данные.

1. В pgAdmin нажмите правой кнопкой по **Databases** → **Create** → **Database**. В поле **Database** введите **dkip2027_course**. Нажмите **Save**.
2. Если ваша роль не может создавать базы, откройте пустую учебную базу, которую выдал преподаватель. Раскройте её → **Schemas** → **public** → **Tables** и проверьте список.

ЧТО ПРОВЕРИТЬ

Выбрана **dkip2027_course**. Список таблиц в новой базе пуст.



Поля создания отдельной учебной базы данных. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 · НЕОБЯЗАТЕЛЬНАЯ ПРОВЕРКА SQL

Создайте пустую учебную базу

```
SELECT current_database();
```

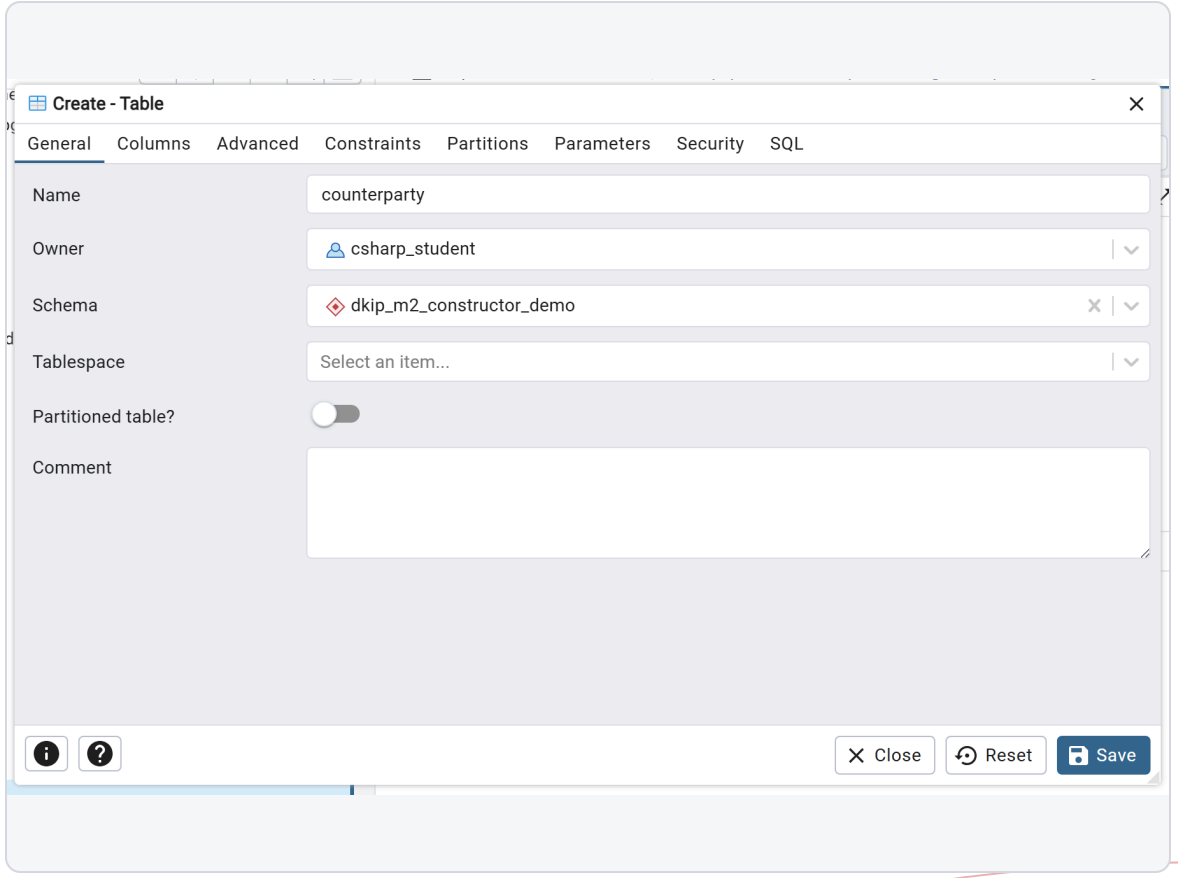
ШАГ 3 ИЗ 10 · 1/3

Создайте counterparty в конструкторе

Откройте форму

Первичный ключ отличает записи. Текстовый тип сохраняет ведущие нули в кодах.

1. В своей базе раскройте **Schemas** → нужная схема (обычно **public**). Нажмите правой кнопкой по **Tables** → **Create** → **Table**. На вкладке **General** введите **counterparty** и перейдите в **Columns**.



pgAdmin, форма **Create Table**. На вкладке **General** введено имя **counterparty** и выбрана учебная схема.

ШАГ 3 ИЗ 10 · 2/3

Создайте counterparty в конструкторе

Добавьте поля

1. Создайте шесть полей в таком порядке: **id** (32), **name** (255), **inn** (20), **address** (500), **phone** (64), **party_type** (24). Число в скобках — длина текстового поля.
2. Нажимайте «+» в **Columns** для добавления строки. Для каждого поля выберите **character varying** и укажите **Length**. Включите **Not NULL** для всех полей и **Primary key** для **id**.
3. Нажмите **Save**. При необходимости обновите **Tables** и раскройте **counterparty** → **Columns**.

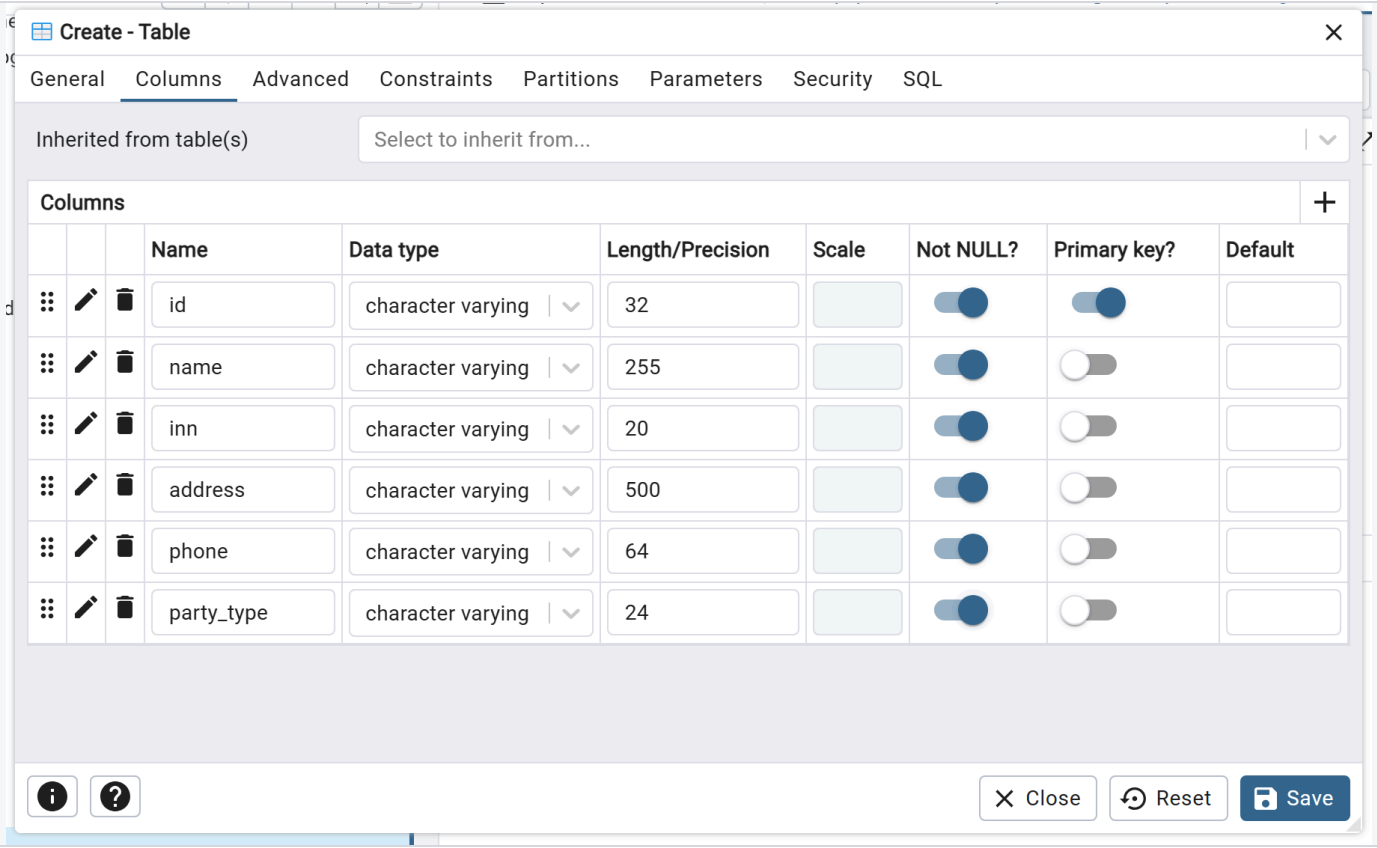
ЧТО ПРОВЕРИТЬ

В базе появилась пустая **counterparty**. Поле **id** имеет текстовый тип и отмечено как первичный ключ.

ШАГ 3 ИЗ 10 · 3/3

Создайте counterparty в конструкторе

Добавьте поля



Вкладка **Columns** в **pgAdmin**. Для шести полей **counterparty** заданы текстовые типы и длины, **id** отмечен как первичный ключ.

ШАГ 4 ИЗ 10 · 1/3

Создайте остальные таблицы и связи

Перенесите таблицы из ER

Таблицы для ссылок создают раньше таблиц с внешними ключами.

1. Выбирайте таблицы в разборе ниже по порядку. Для каждой откройте форму создания, перенесите поля из карточки и сохраните таблицу. Начните с **item**, если **counterparty** уже создана на предыдущем шаге.
2. Для составного ключа откройте **Constraints** → **Primary Key** в форме таблицы. Нажмите «+», откройте созданную строку карандашом и в **Definition** → **Columns** добавьте оба поля в один ключ. Для **specification_component** это **specification_id** и **item_id**.

ШАГ 4 ИЗ 10 · 2/3

Создайте остальные таблицы и связи

Проведите связи

1. В дереве зависимой таблицы нажмите правой кнопкой по **Constraints** → **Create** → **Foreign Key**. В **General** задайте имя ключа. В **Columns** для примера **notes** укажите **Local column** — **id_user**, **References** — **users** в своей схеме, **Referencing** — **id**. Нажмите «+» и **Save**. Повторите для каждой связи из карточки.
2. Выберите свою базу и откройте **Tools** → **ERD For Database**, если этот пункт доступен. Сверьте связи на схеме.

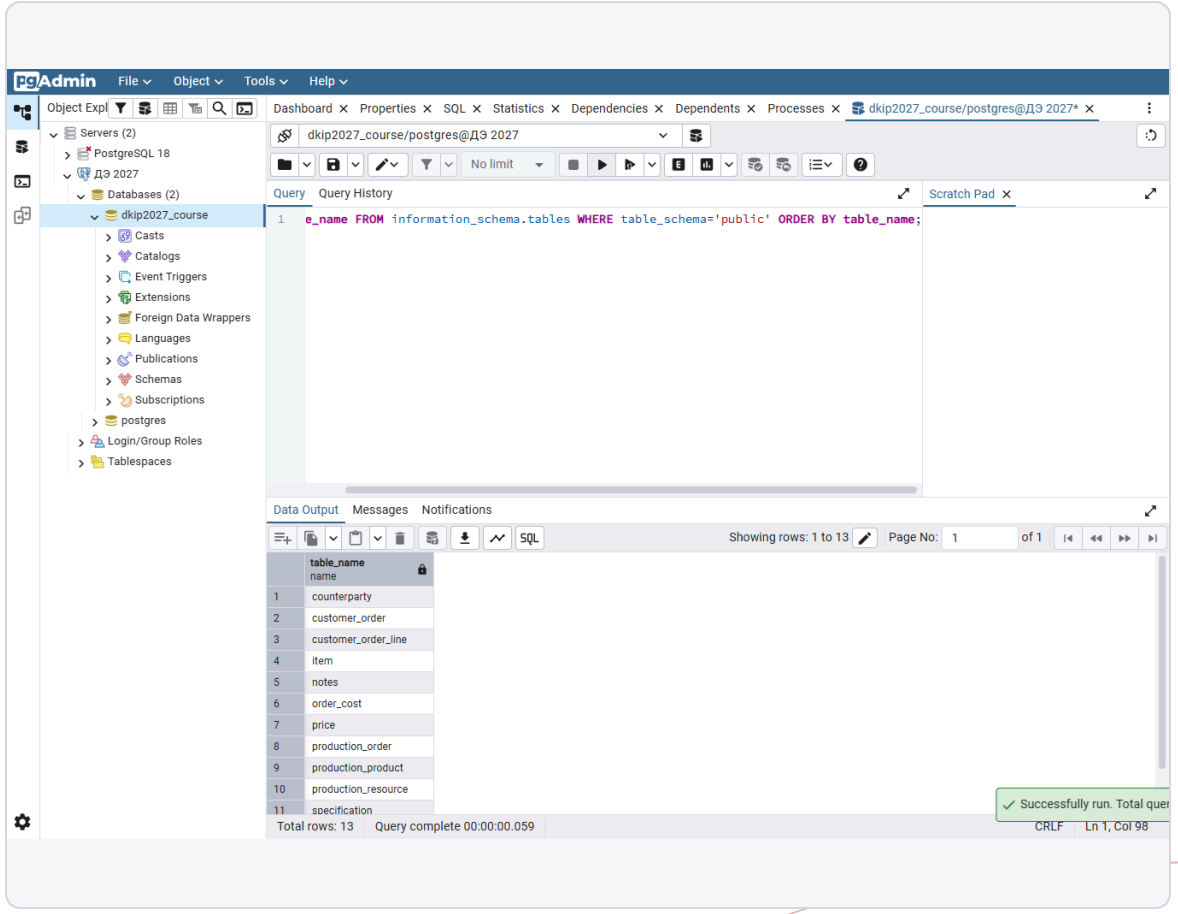
ШАГ 4 ИЗ 10 · 3/3

Создайте остальные таблицы и связи

Покажите результат на экране

ЧТО ПРОВЕРИТЬ

В редакторе видны таблицы с вашей ER-диаграммы и заданные связи. Учебный пример содержит 12 таблиц.



Созданные таблицы учебной базы в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 5 ИЗ 10 · 1/2

Проверьте первичные и внешние ключи

Первичный ключ отличает строки, а внешний показывает, на какую таблицу и поле ссылается запись.

1. Раскройте **specification_component** → **Constraints**. Откройте свойства **Primary Key** и проверьте столбцы **specification_id** и **item_id**.
2. Раскройте **Constraints** в каждой зависимой таблице и откройте внешние ключи. Начните с **notes.id_user** → **users.id**. Сверьте поля каждой связи по карточкам таблиц.

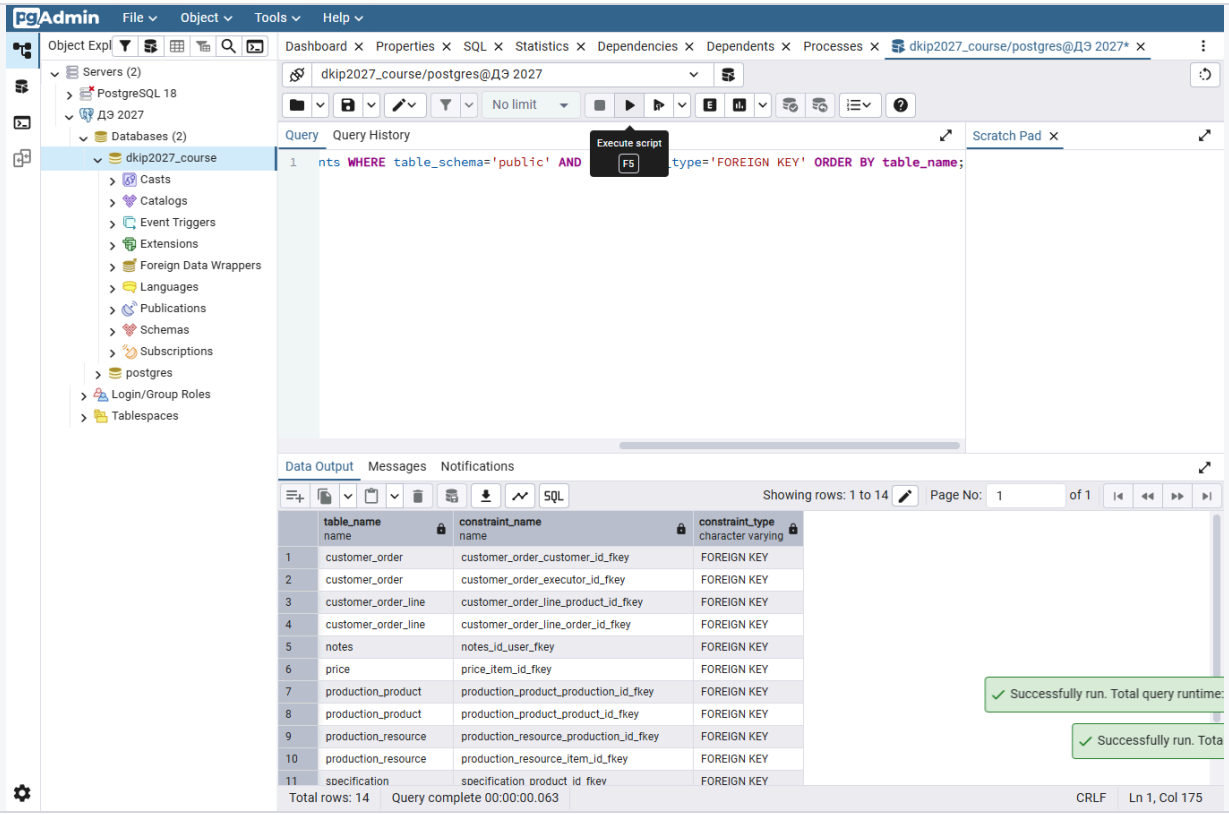
ЧТО ПРОВЕРИТЬ

В примере **12** таблиц, у каждой есть первичный ключ. В каталоге внешних ключей должно быть **14** пар полей.

ШАГ 5 ИЗ 10 · 2/2

Проверьте первичные и внешние ключи

Покажите на экране



Проверка по каталогу **PostgreSQL** показывает **14** внешних ключей учебной схемы. Поля отдельной связи открываются через **Constraints** соответствующей таблицы.

ШАГ 6 ИЗ 10

Подготовьте CSV с заказчиками

Каждый объект исходного JSON станет одной строкой CSV и затем одной строкой в **counterparty**.

1. Откройте «Заказчики.json» из папки «Исходные данные → Задание 2». В первой записи найдите код 000000001 и пустой ИНН.
2. Откройте «Справка» → «CSV через Excel». Загрузите JSON в Power Query, задайте полям тип «Текст» и сохраните «Заказчики.csv» в формате CSV UTF-8.
3. Откройте CSV в Блокноте. Проверьте порядок заголовков id, name, inn, address, phone, party_type. Запомните разделитель полей.

ЧТО ПРОВЕРИТЬ

В «Заказчики.csv» есть заголовок и шесть записей. Первый код сохранился как 000000001.

Откройте: Заказчики.json

ШАГ 7 ИЗ 10 · 1/2

Импортируйте CSV в counterparty

Редактор базы переносит строки из подготовленного CSV в уже созданную таблицу.

1. Нажмите правой кнопкой по пустой **counterparty** → **Import/Export Data**. В **General** выберите **Import**, «Заказчики.csv», **csv** и **UTF8**.
2. В **Options** включите **Header**, выберите разделитель из файла и двойную кавычку (") в **Quote** и **Escape**. В **Columns** добавьте **inn** в **NOT NULL columns**.
3. Нажмите **OK** и дождитесь **Process completed**. Подробные настройки по шагам показаны в «Справке» → «**CSV через Excel**».

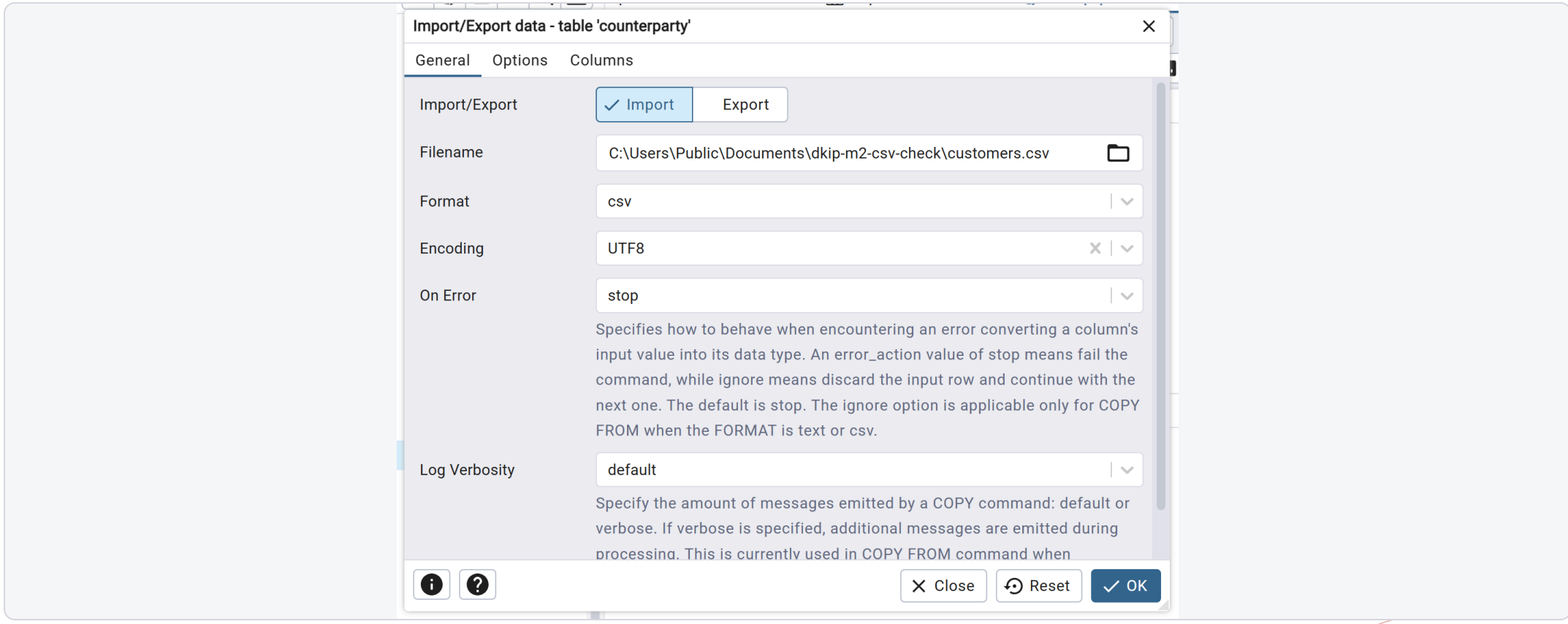
ЧТО ПРОВЕРИТЬ

Редактор завершил импорт. В **counterparty** должно быть шесть строк.

ШАГ 7 ИЗ 10 · 2/2

Импортируйте CSV в counterparty

Покажите на экране



pgAdmin, вкладка **General**. Для импорта в **counterparty** выбран учебный файл **CSV** в кодировке **UTF8**.

ШАГ 8 ИЗ 10

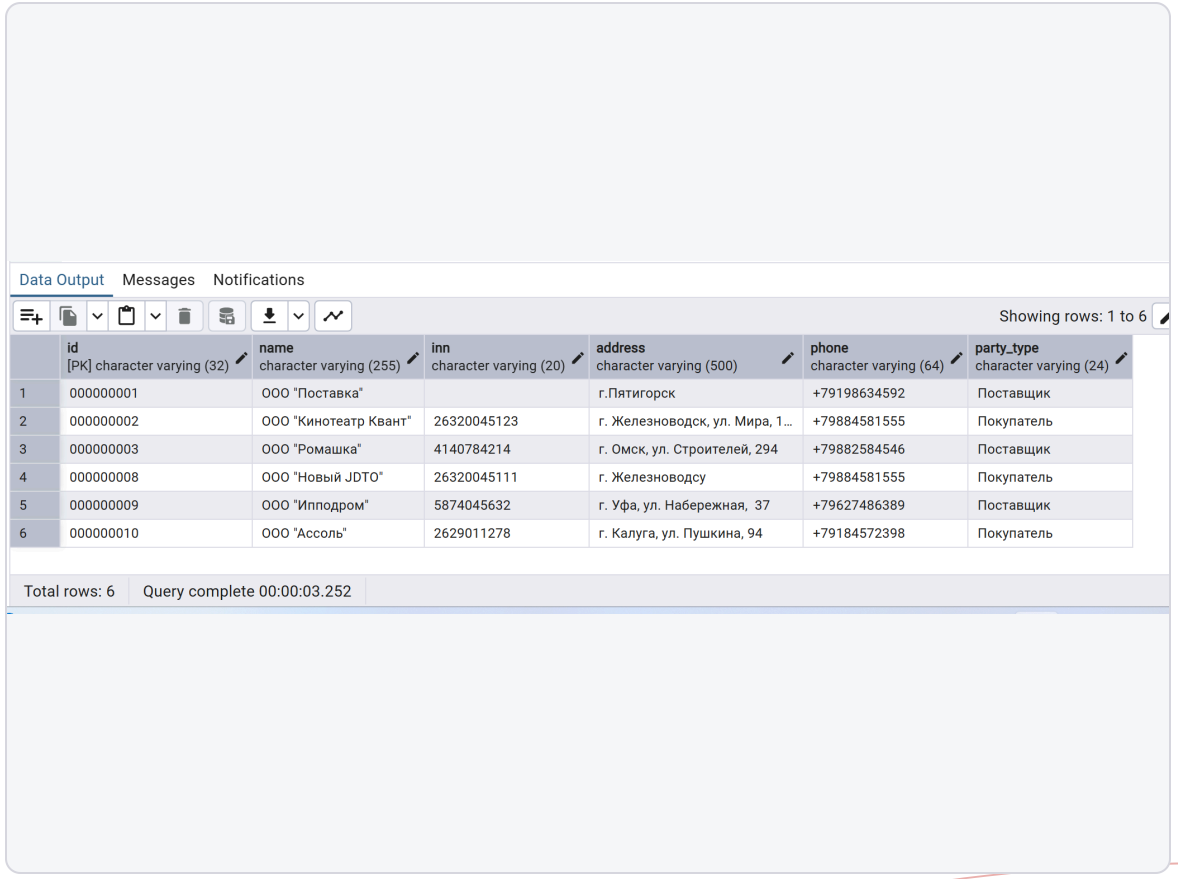
Сверьте импортированные значения

Проверьте, что в таблицу попали все записи и сохранились исходные значения.

1. Нажмите правой кнопкой на **counterparty** → **View/Edit Data** → **All Rows**. Сравните строки с «Заказчики.csv».
2. Проверьте первый **id** 000000001, пустой ИНН, кавычки в названиях, знак + в телефонах и значения **address** и **party_type**.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть строк. Значения совпадают с подготовленным **CSV**.



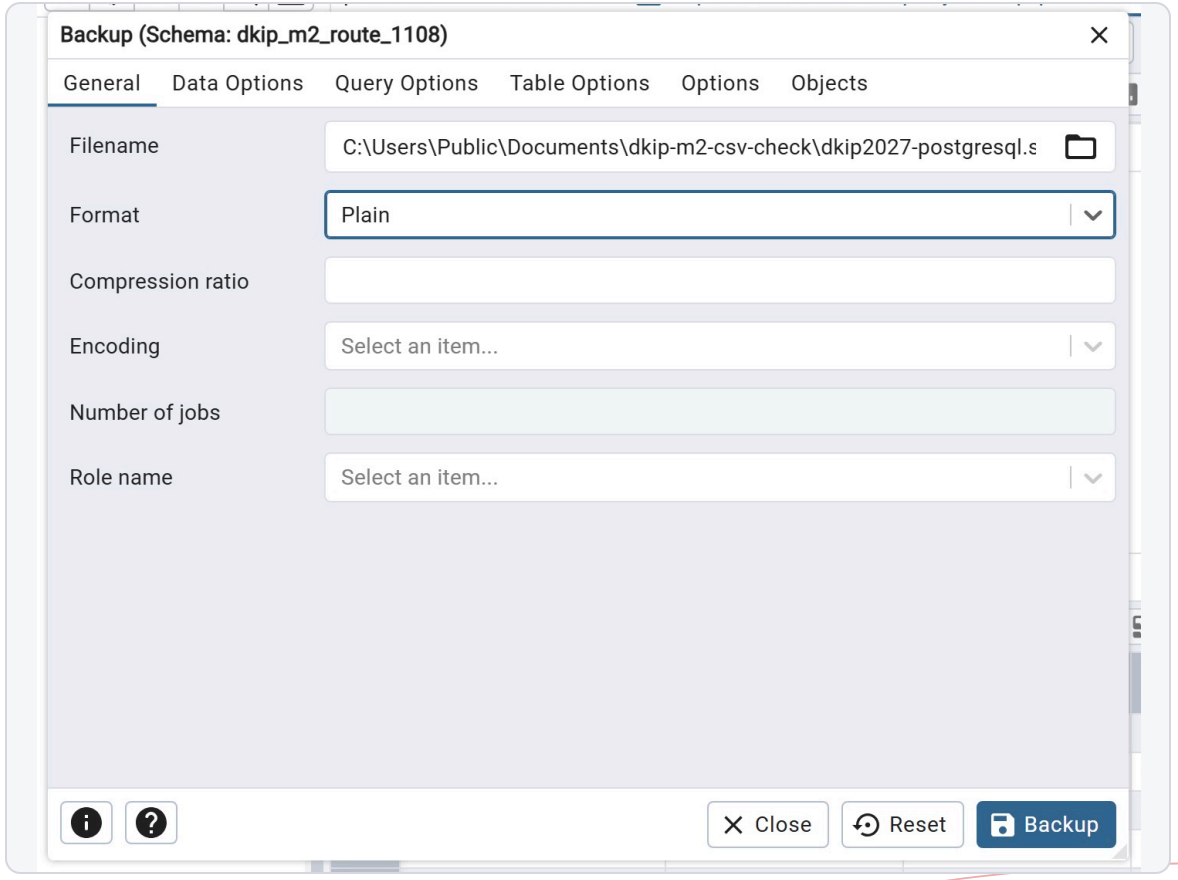
В **counterparty** загружены шесть строк. Видны ведущие нули, кавычки в названиях, пустой ИНН и знак + в телефонах.

ШАГ 9 ИЗ 10 · 1/2

Сохраните выгрузку

Выгрузка переносит структуру и загруженные данные в другую базу.

1. Нажмите правой кнопкой по своей базе → **Backup**. Если на учебном сервере вам выделили отдельную схему в общей базе, выберите **Backup** у своей схемы. В **Filename** укажите путь к **dkip2027-postgresql.sql** в папке «Результаты».
2. Выберите **Format** → **Plain**. В **Data Options** оставьте выключенными **Only data** и **Only schemas**, чтобы в файл вошли структура и данные. Нажмите **Backup** и дождитесь статуса **Finished** в **Processes**.



pgAdmin, окно **Backup** учебной схемы. Формат **Plain** создаёт SQL-файл со структурой и данными.

ШАГ 9 ИЗ 10 · 2/2

Сохраните выгрузку

1. Откройте сохранённый .sql в Блокноте. Найдите **CREATE TABLE** и блок данных **counterparty**. Расширение файла само по себе не определяет его формат.

ЧТО ПРОВЕРИТЬ

В папке «Результаты» есть **SQL**-файл со структурой и данными таблиц.

ШАГ 10 ИЗ 10 · 1/2

Проверьте восстановление

Отдельная пустая база позволяет проверить, что выгрузка содержит всё нужное.

1. Создайте пустую базу **dkip2027_restore**. Нажмите правой кнопкой по ней → **Restore**.
2. Выберите формат **Plain**, укажите свой **SQL**-файл и запустите **Restore**. В **Processes** дождитесь статуса **Finished**. Если **Restore** не предлагает **Plain**, для проверки сделайте отдельный архив **Backup** в формате **Custom** с расширением **.backup** и выберите **Custom** в **Restore**.
3. Обновите дерево восстановленной базы. Сверьте 12 таблиц и 14 внешних ключей в **Constraints**. Откройте **counterparty** → **View/Edit Data** → **All Rows** и сравните шесть строк с исходным файлом.

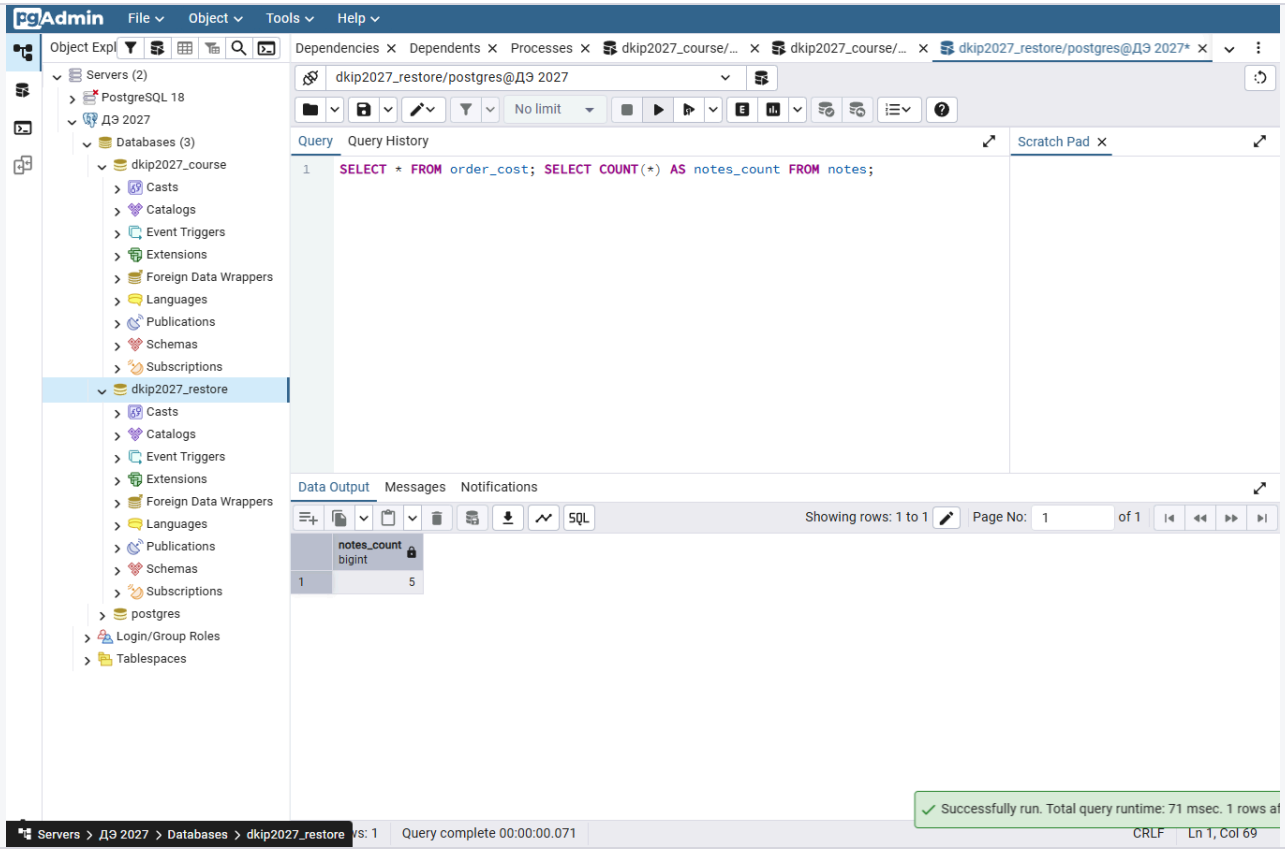
ЧТО ПРОВЕРИТЬ

В восстановленной базе есть таблицы, ключи и те же шесть исходных записей.

ШАГ 10 ИЗ 10 · 2/2

Проверьте восстановление

Покажите на экране



Запрос к отдельной восстановленной базе **dkip2027_restore**. Маршрут: **WPF / PostgreSQL**.

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

В counterparty шесть строк. Первый id — 000000001, ведущие нули сохранены. Есть файл выгрузки dkip2027-postgresql.sql.

ЧТО ПРОВЕРИТЬ

Рабочая база с импортированными заказчиками и выгрузка.

ЗАДАНИЕ 3 · 20 МИН

Расчёт стоимости

Рассчитаем затраты на изготовление двух столов. Скидка и итог к оплате берутся из другого расчёта.

ЧТО ПРОВЕРИТЬ

03-cost.sql даёт **14 374,28**: материалы **9 974,28** и операции **4 400**.

УСЛОВИЕ ИЗ КИМ

Создание запроса

Создайте запрос, позволяющий вычислить полную стоимость заказа покупателя с учетом следующей информации:

- количество продукции в заказе;

- стоимость всех материалов и технологических операций, использованных для производства данной продукции (учитывая норму расхода).

Приложение: Прил_3_ОЗ_КИМ_09.02.07-5-2027.rar

КИМ · стр. 31 Исходные файлы · ZIP

ШАГ 1 ИЗ 7 · 1/8

Подготовьте строки для расчёта

Откройте свою базу из задания 2

Расчёт соединяет количество заказа, норму спецификации и цену ресурса. Сначала проверяйте одну строку вручную; затем стройте общий SQL.

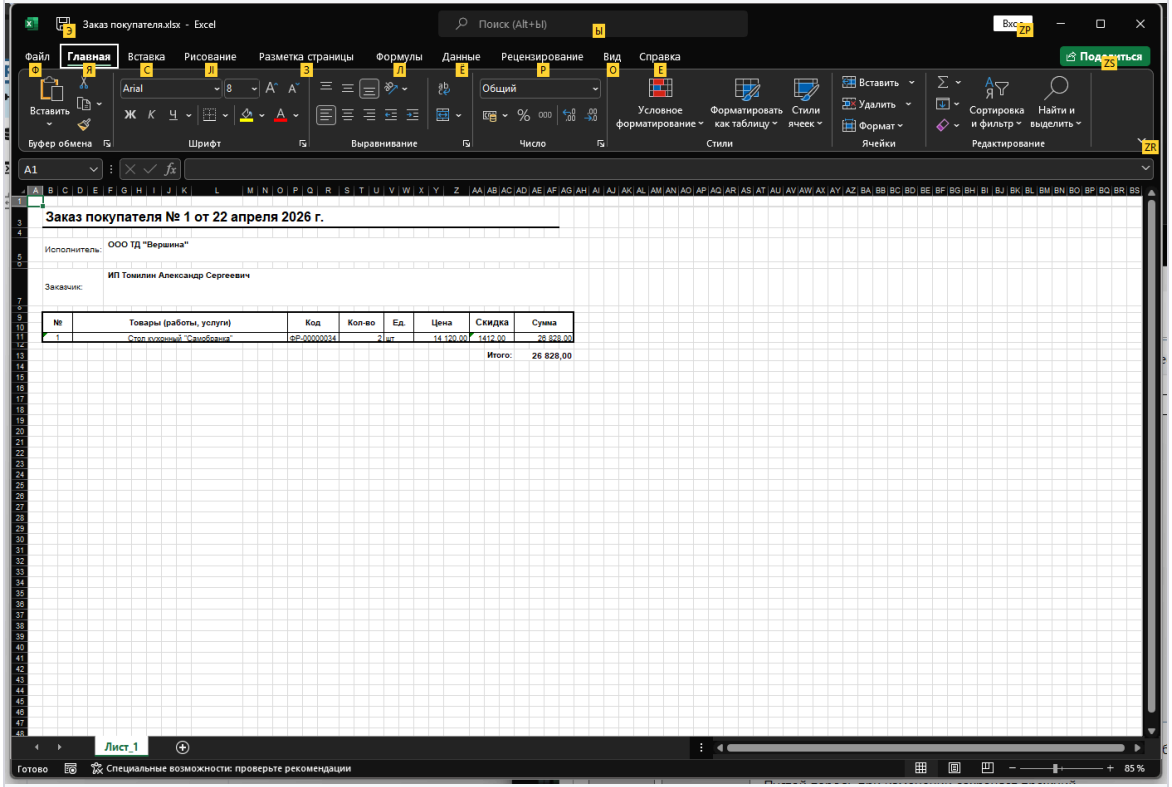
1. Откройте свою базу из задания 2. В **counterparty** уже есть шесть записей из **JSON**. Откройте четыре таблицы **Excel** по ссылкам шага.
2. В **pgAdmin** откройте **Schemas** → **public** → **Tables**. Правой кнопкой по таблице → **View/Edit Data** → **All Rows**. Нажмите **Add row**, заполните строку и нажмите **Save Data Changes**. Поля **Identity** оставляйте пустыми. После сохранения посмотрите присвоенный **id**.

ШАГ 1 ИЗ 7 · 2/8

Подготовьте строки для расчёта

Добавьте участников заказа

1. В «Заказ покупателя.xlsx» найдите исполнителя в F5 и покупателя в F7. Их нет среди шести записей из JSON. В counterparty добавьте покупателя ИП Томилин Александр Сергеевич и исполнителя ООО ТД «Вершина».
2. Откройте Query Tool и выполните блок «Контрагенты из заказа». Он добавит две строки с учебными кодами DOC-CUSTOMER и DOC-MAKER. ИНН, адрес и телефон в этих строках пустые, поскольку таких сведений в заказе нет.



Заказ покупателя из приложения открыт в Excel. Маршрут: WPF / PostgreSQL.

ШАГ 1 ИЗ 7 · 3/8

Подготовьте строки для расчёта

Заполните номенклатуру

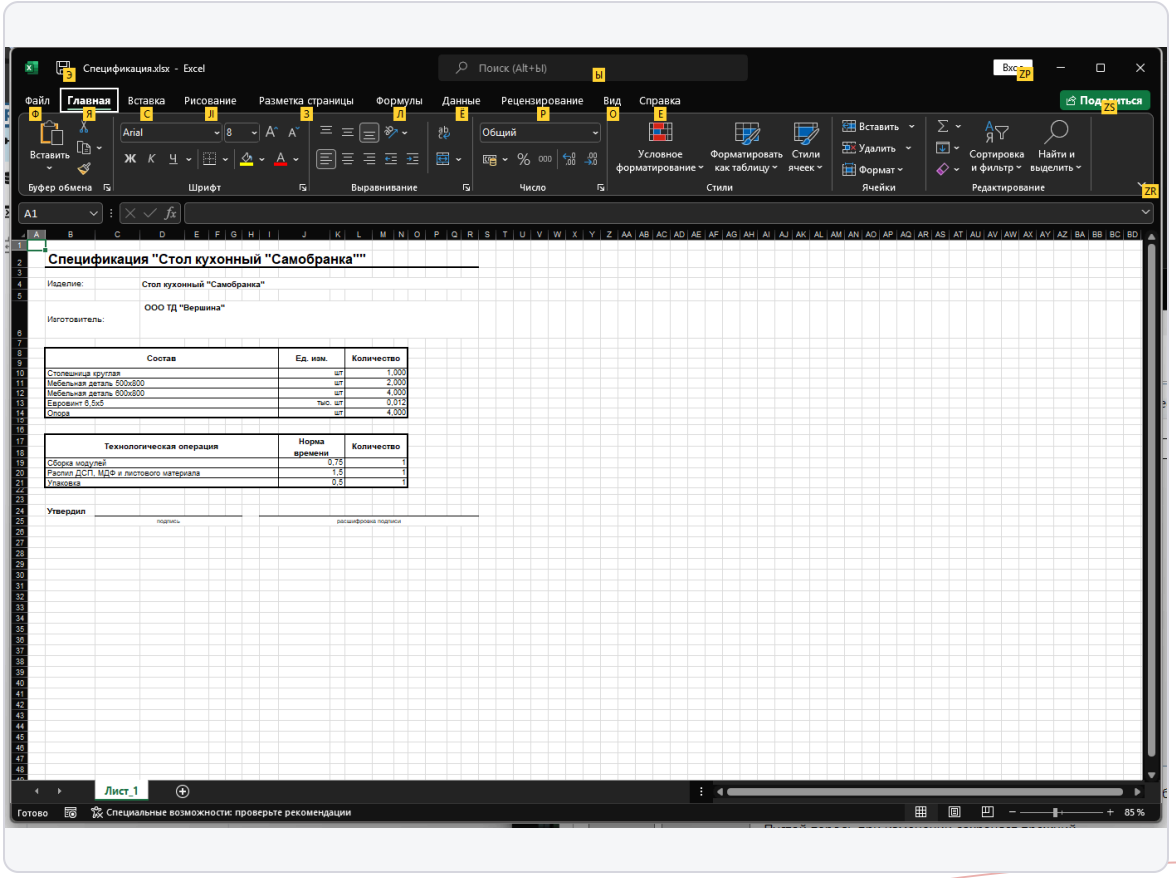
1. В item добавьте стол с кодом НФ-00000006 из S11 заказа на производство, названием из D11, kind = product и unit = шт. Запишите присвоенный id стола.
2. Добавьте пять материалов и три операции. Названия и коды возьмите из столбцов D и S заказа на производство, строки 17–21 и 27–29. Для материалов выберите kind = material, единицы возьмите из J10:J14 спецификации. Для операций укажите kind = operation и unit = ч.

ШАГ 1 ИЗ 7 · 4/8

Подготовьте строки для расчёта

Соберите спецификацию на один стол

1. В specification укажите product_id стола, название из B2 спецификации, output_qty = 1 и manufacturer_id = DOC-MAKER. Запишите присвоенный id спецификации.
2. В specification_component добавьте восемь строк. В каждой укажите specification_id и item_id соответствующего ресурса. Для qty возьмите L10:L14 и L19:L21. Для time_norm поставьте 1 у материалов и возьмите J19:J21 для операций.



Спецификация: материалы, количества и нормы технологических операций в Excel. Маршрут: WPF / PostgreSQL.

ШАГ 1 ИЗ 7 · 5/8

Подготовьте строки для расчёта

Соберите спецификацию на один стол

1. Например, у евровинта qty = 0.012 тыс. шт. и time_norm = 1. У сборки модулей qty = 1 и time_norm = 0.75 ч. Эти нормы относятся к одному столу.

ШАГ 1 ИЗ 7 · 6/8

Подготовьте строки для расчёта

Добавьте цены и заказ

1. В price добавьте по строке для каждого из восьми ресурсов. По названию в A2:A9 файла «Цены.xlsx» найдите item_id, цену возьмите из столбца D. Прайс без даты и единиц тарифа. Для всех строк примите valid_from = 2026-04-01, для операций — тариф за час.
2. В customer_order внесите номер 1 и дату 2026-04-22 из B3 заказа покупателя. Укажите customer_id = DOC-CUSTOMER и executor_id = DOC-MAKER. Запишите присвоенный id заказа.

ШАГ 1 ИЗ 7 · 7/8

Подготовьте строки для расчёта

Добавьте цены и заказ

1. В customer_order_line укажите order_id заказа и product_id стола. Из строки 11 возьмите source_code = ФР-00000034 (P11), qty = 2 (S11), sale_price = 14120 (X11) и discount = 1412 (AA11). По совпадению названия оба кода относятся к одному столу. В item достаточно одной записи о нём.

ШАГ 1 ИЗ 7 · 8/8

Подготовьте строки для расчёта

Сверьте восемь ресурсов перед расчётом

1. Откройте Query Tool выбранной базы. Выполните блок «Проверка данных заказа». В нём видны количество столов, нормы на один стол и цены из прайса, принятые на 1 апреля 2026 года.
2. Если строк меньше восьми, проверьте связи заказа, стола и состава спецификации. Если **amount** пустое (NULL), проверьте **item_id** ресурса и дату его цены.

ЧТО ПРОВЕРИТЬ

Запрос показывает пять материалов и три операции. Во всех строках **tables_ordered = 2**. Для евровинта **qty_per_table = 0,012** и **amount = 595**, для сборки **time_norm = 0,75** и **amount = 1400**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx** · Спецификация.**xlsx** · Цены.**xlsx**

ШАГ 1 · КОНТРАГЕНТЫ ИЗ ЗАКАЗА

Подготовьте строки для расчёта

```
INSERT INTO counterparty (id, name, inn, address, phone, party_type)
VALUES
  ('DOC-CUSTOMER', 'ИП Томилин Александр Сергеевич', '', '', '', 'Покупатель'),
  ('DOC-MAKER', 'ООО ТД «Вершина»', '', '', '', 'Производитель');
```

ШАГ 1 · ПРОВЕРКА ДАННЫХ ЗАКАЗА

Подготовьте строки для расчёта

```
SELECT o.doc_no, l.qty AS tables_ordered, r.name AS resource,
       c.qty AS qty_per_table, c.time_norm, p.amount
FROM customer_order o
JOIN customer_order_line l ON l.order_id = o.id
JOIN specification s ON s.product_id = l.product_id
JOIN specification_component c ON c.specification_id = s.id
JOIN item r ON r.id = c.item_id
LEFT JOIN price p ON p.item_id = r.id
                 AND p.valid_from = '2026-04-01'
WHERE o.doc_no = '1'
ORDER BY r.kind, r.code;
```

ШАГ 2 ИЗ 7

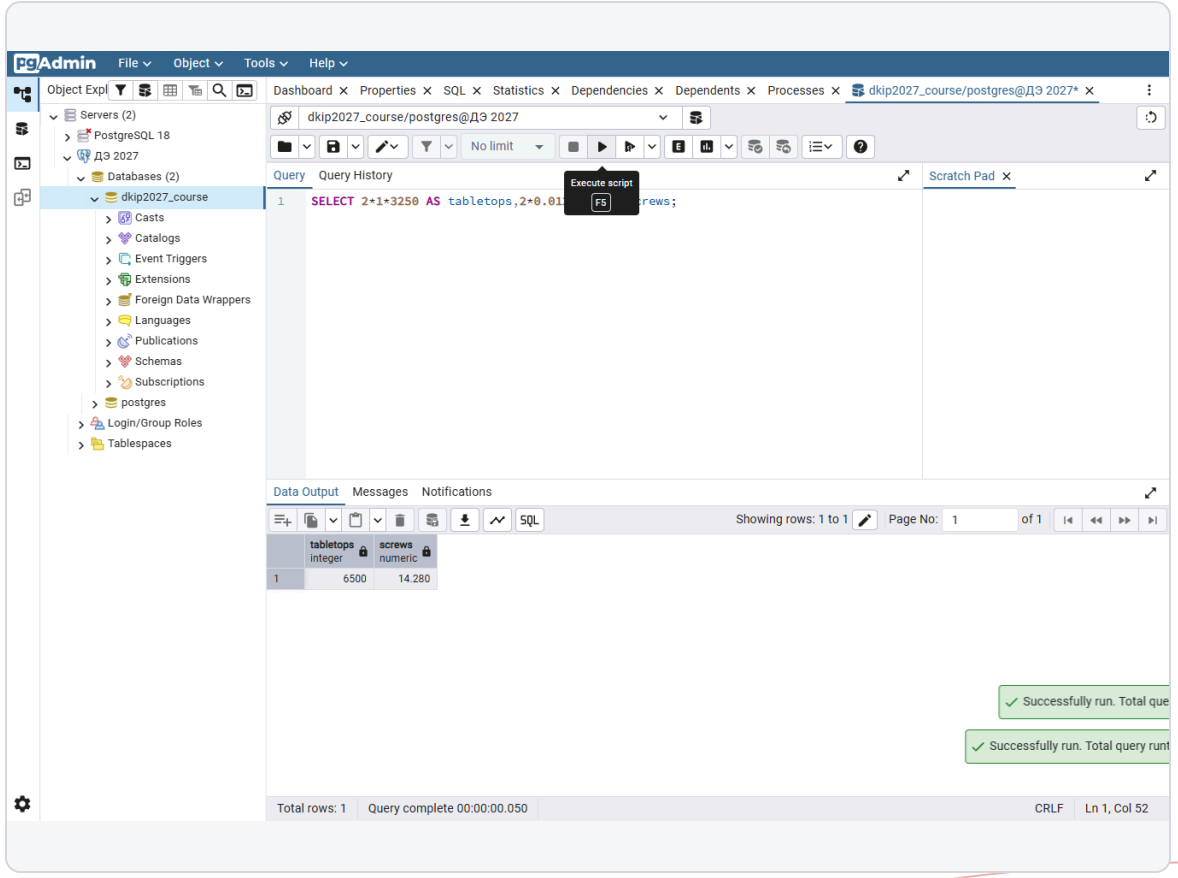
Посчитайте одну строку материала

Стоимость материала = количество изделий × норма × цена.

1. Найдите столешницу: 1 на стол, цена 3250.
2. Для 2 столов вычислите 2 × 1 × 3250.
3. Проверьте евровинт: 2 × 0,012 × 595.

ЧТО ПРОВЕРИТЬ

Столешницы: **6500**; евровинты: **14,28**.



Проверка двух строк материалов через SQL: 6500 и 14,28. Маршрут: WPF / PostgreSQL.

ШАГ 2 · КОД ДЛЯ ЭТОГО ШАГА

Посчитайте одну строку материала

```
SELECT 2*1*3250 AS tabletops, 2*0.012*595 AS screws;
```

ШАГ 3 ИЗ 7

Посчитайте технологические операции

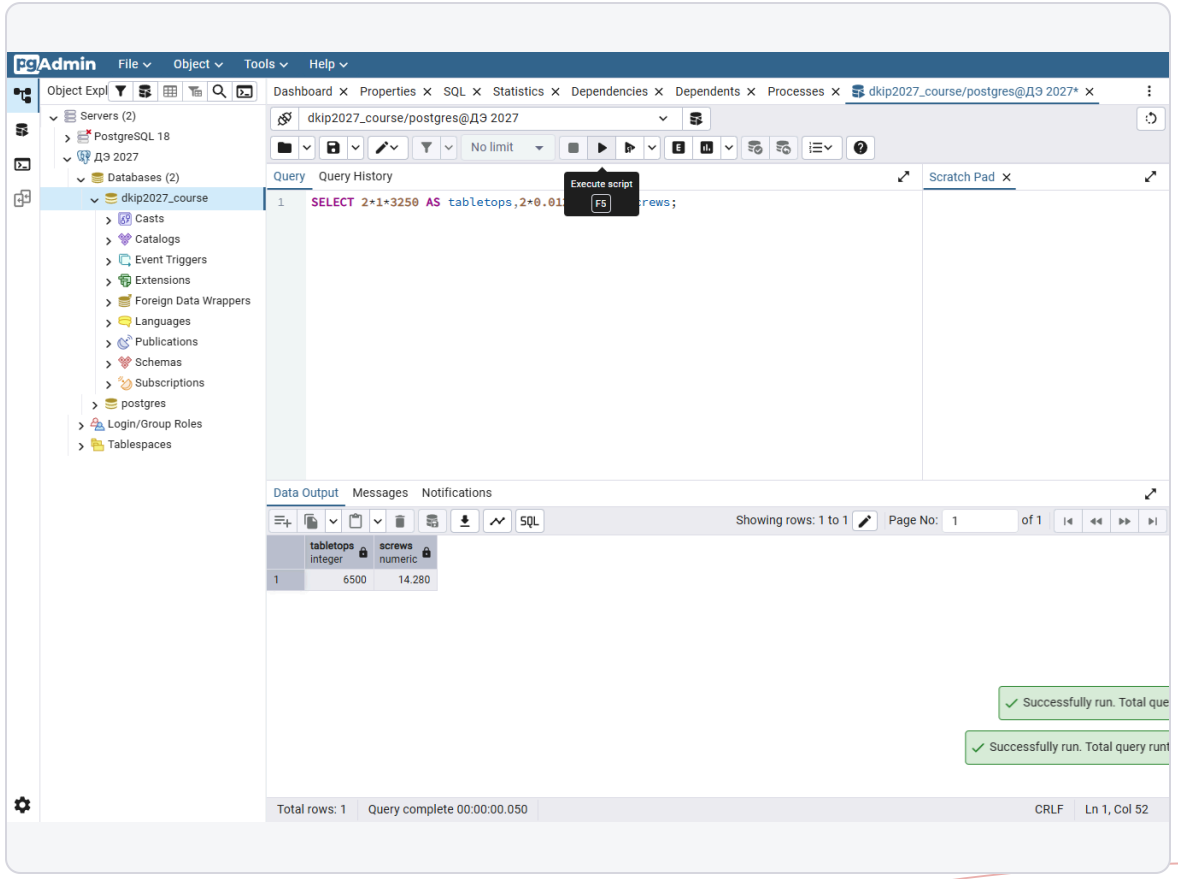
Для операции учитываются количество и норма времени. Тариф за час — принятое допущение.

1. Сборка: 2 × 1 × 0,75 × 1400.
2. Распил: 2 × 1 × 1,5 × 450.
3. Упаковка: 2 × 1 × 0,5 × 950.

ЧТО ПРОВЕРИТЬ

Операции стоят 2100 + 1350 + 950 = 4400.

Спецификация.xlsx, J19:L21; Цены.xlsx, D6:D9



Проверка двух строк материалов через SQL: 6500 и 14,28. Маршрут: WPF / PostgreSQL.

ШАГ 4 ИЗ 7

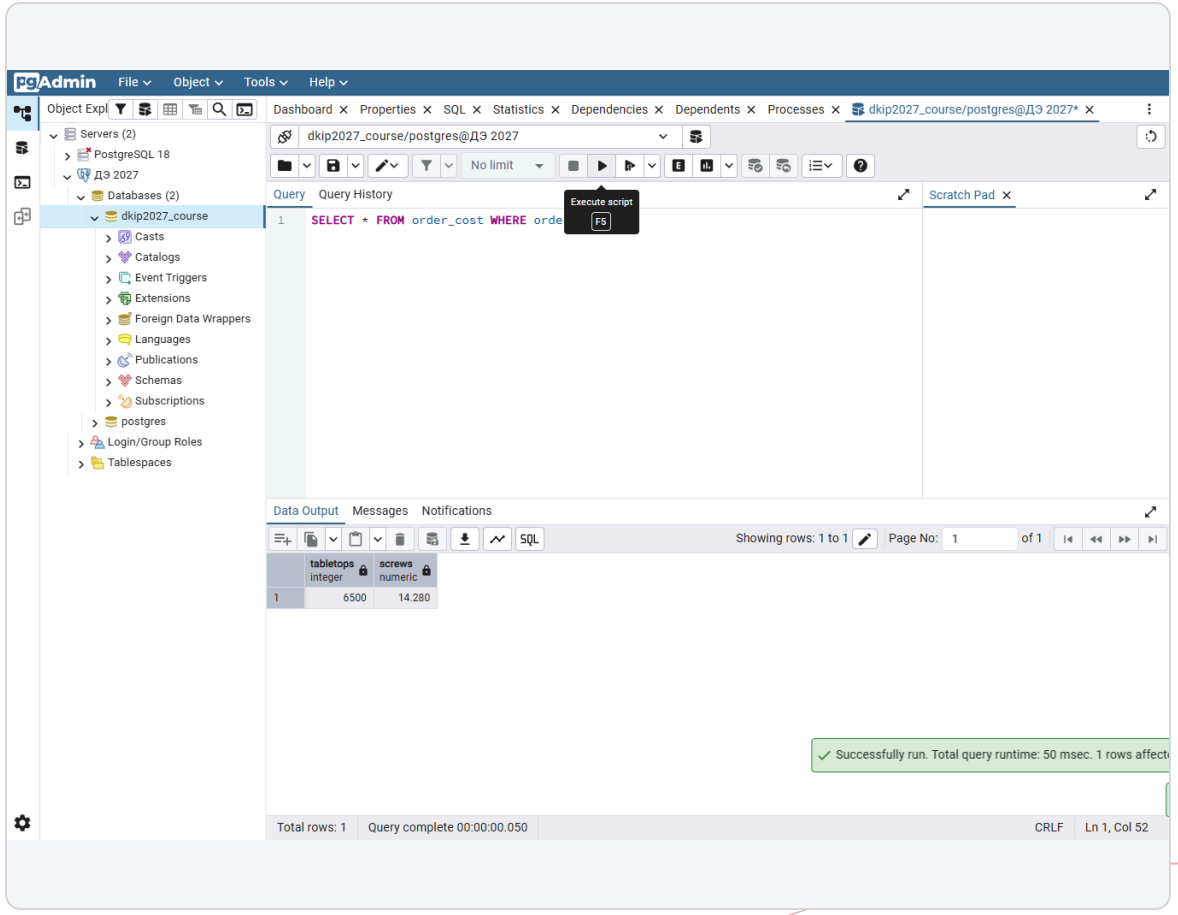
Соедините количество и нормы

JOIN связывает строки по ключам. Один составной список ресурсов предотвращает перемножение материалов и операций.

1. От customer_order перейдите к customer_order_line.
2. По product_id найдите specification.
3. По specification_id получите specification_component и price.

ЧТО ПРОВЕРИТЬ

Каждый ресурс изделия участвует в сумме ровно один раз.



Выполнение запроса к представлению **order_cost**. Маршрут: **WPF / PostgreSQL**.

ШАГ 5 ИЗ 7

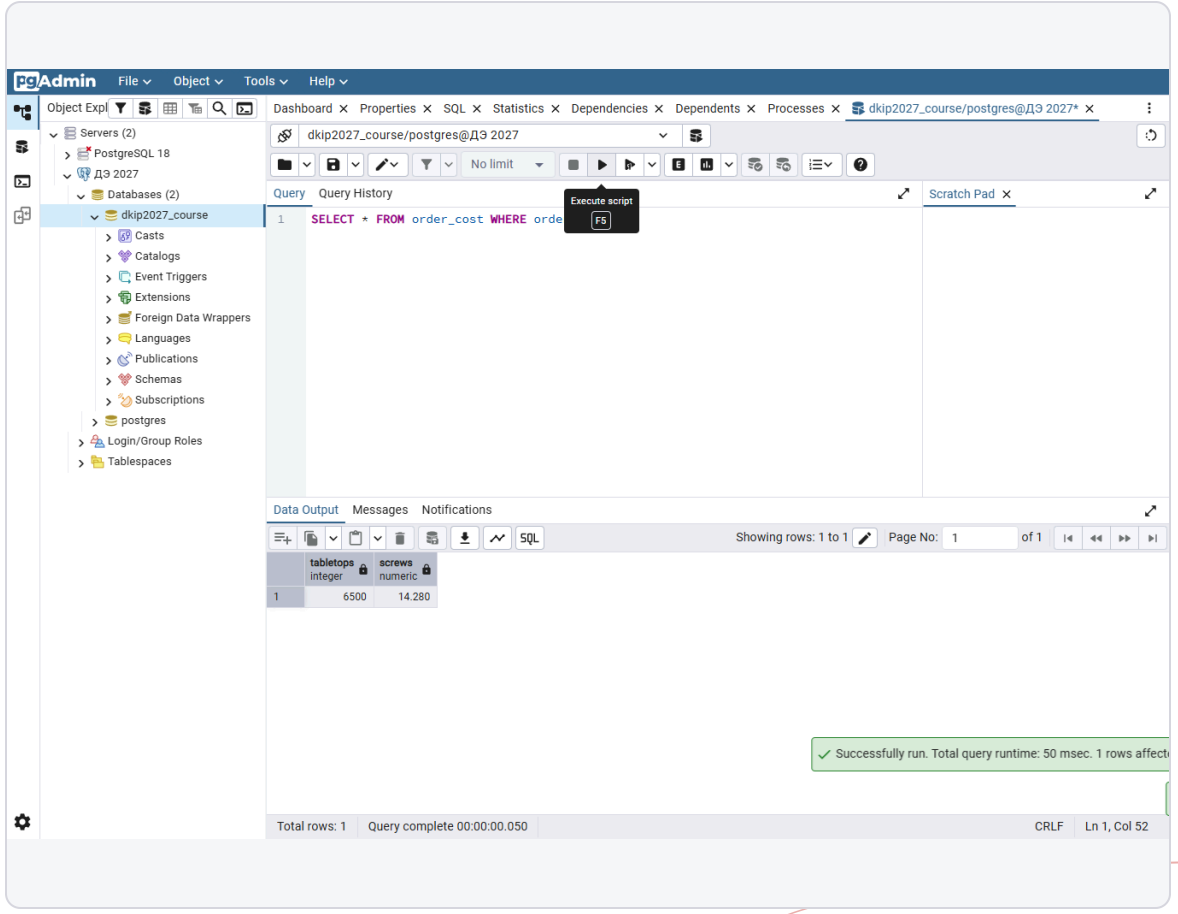
Выберите цену на дату заказа

Поздняя цена не должна менять расчёт старого заказа.

1. Напишите условие выбора последней цены ресурса, действовавшей на дату заказа.
2. Используйте MAX(valid_from) с условием valid_from <= дата заказа.
3. Проверьте, что для каждого ресурса есть допустимая цена.

ЧТО ПРОВЕРИТЬ

На 22.04.2026 используются цены с 01.04.2026.



Выполнение запроса к представлению **order_cost**. Маршрут: **WPF / PostgreSQL**.

ШАГ 6 ИЗ 7

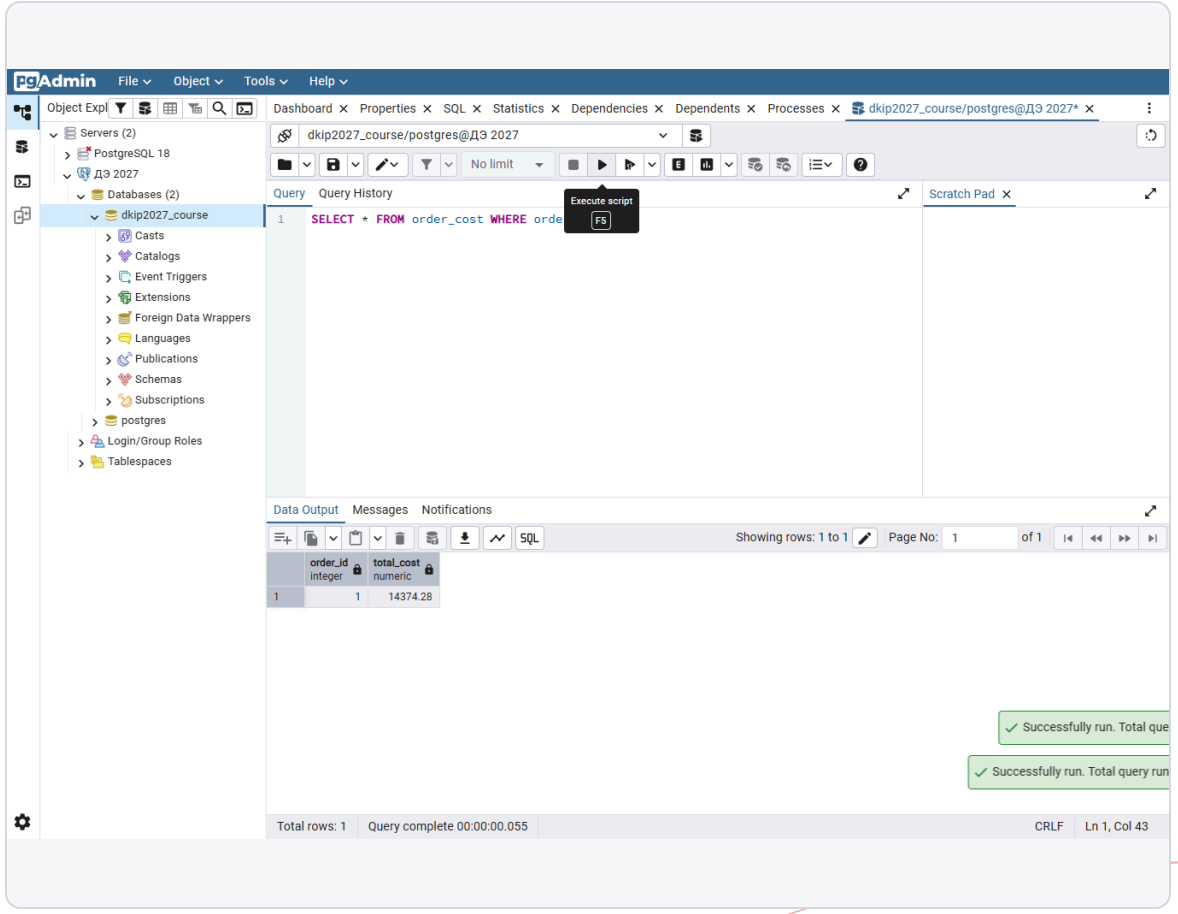
Выполните итоговый запрос

SUM суммирует ресурсы, ROUND округляет общий итог до двух знаков.

1. В pgAdmin запишите собственный запрос: соедините заказ, строки состава и цены, затем вычислите SUM.
2. Оформите запрос как представление order_cost и выполните выборку для своего заказа; в примере его order_id=1.
3. Сверьте с ручной суммой: материалы 9974,28 плюс операции 4400.

ЧТО ПРОВЕРИТЬ

Нормативная стоимость двух столов: 14374,28.



Результат расчёта в СУБД: 14 374,28. Маршрут: WPF / PostgreSQL.

ШАГ 6 · КОД ДЛЯ ЭТОГО ШАГА

Выполните итоговый запрос

```
SELECT * FROM order_cost WHERE order_id=1;
```

ШАГ 7 ИЗ 7

Обработайте неполные данные

Если отсутствует спецификация или цена, частичная сумма неверна.

1. Используйте **LEFT JOIN** для спецификации, её состава и действующих цен. Так строка с недостающими данными останется в расчёте.
2. Сравните **COUNT(price.item_id)** и **COUNT(*)** в каждой группе. Если хотя бы у одной строки нет цены, верните **NULL** вместо суммы.
3. В копии базы оставьте один ресурс без цены на дату заказа и выполните **03-cost.sql**. Затем верните цену и повторите запрос.

ЧТО ПРОВЕРИТЬ

Без цены итог равен **NULL** и видна причина. После восстановления цены итог снова **14 374,28**.



ШАГ 7 · 03-COST.SQL

Обработайте неполные данные

```
SELECT order_id, total_cost,
       CASE WHEN total_cost IS NULL THEN 'Недостаточно норм или цен на дату заказа'
            ELSE 'Расчёт выполнен'
       END AS cost_status
FROM order_cost WHERE order_id=1;
-- Ожидается 14374.28 при допущениях docs/DECISIONS.md.
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

03-cost.sql даёт 14 374,28: материалы 9 974,28 и операции 4 400.

ЧТО ПРОВЕРИТЬ

Запрос SQL, результат и записанные допущения.

ЗАДАНИЕ 4 · 60 МИН

Формы и WPF

Сначала собираем форму в конструкторе, затем связываем кнопки с кодом. После входа пользователь и администратор увидят разные возможности.

ЧТО ПРОВЕРИТЬ

Есть проект **MyApp** с исходниками и снимки входа, администрирования и проверки блокировки.

УСЛОВИЕ ИЗ КИМ

Разработка информационной системы

Разработайте модули информационной системы на основании технического задания (Приложение 1)

Приложение: Прил_4_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 31; Приложение 1 Исходные файлы · ZIP

ШАГ 1 ИЗ 24

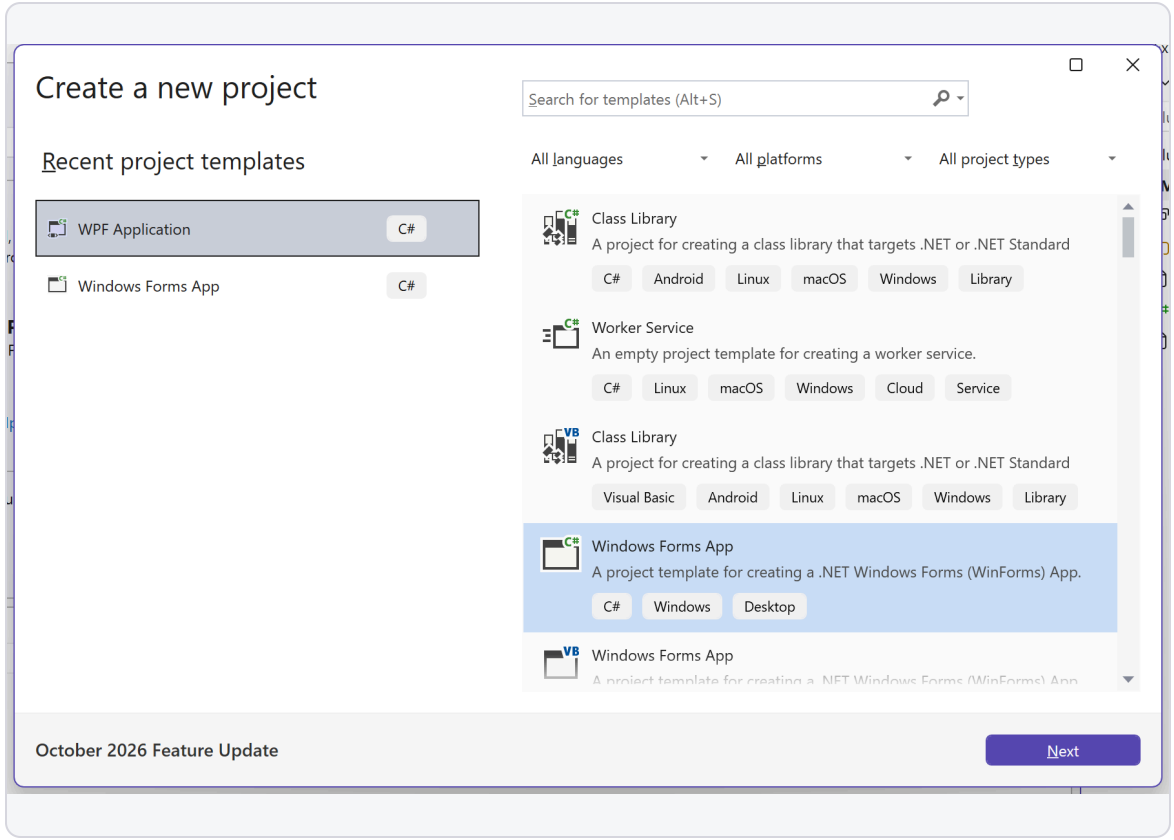
Откройте создание проекта MyApp

Проект объединяет исходные файлы и настройки сборки. Сначала откройте мастер создания нового проекта в Visual Studio.

1. Если видно стартовое окно Visual Studio, нажмите **Create a new project**. Если проект уже открыт, выберите «Файл» → «Создать» → «Проект» (Ctrl+Shift+N). Откроется окно поиска шаблона. Вводите название шаблона в поле поиска, а не в окно кода.

ЧТО ПРОВЕРИТЬ

Открылось окно **Create a new project** с полем поиска шаблонов.



Мастер **Create a new project** в **Visual Studio**: поле **Search for templates** и кнопка **Next**. Введите шаблон из текущего шага; список **Recent** зависит от ранее открытых проектов.

ШАГ 2 ИЗ 24

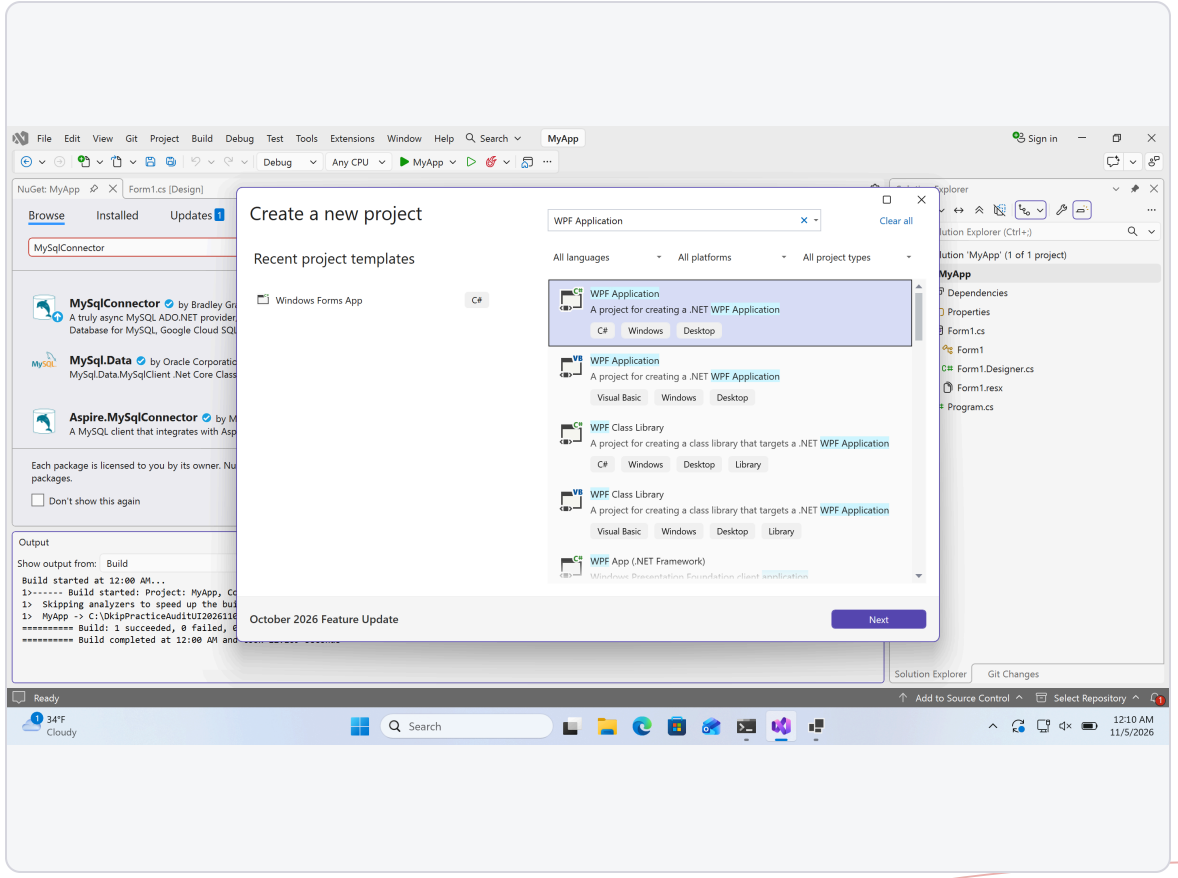
Выберите шаблон MyApp

Шаблон определяет тип приложения и начальные файлы. Названия современного .NET и .NET Framework похожи, но это разные платформы.

1. Найдите «Приложение WPF (WPF Application)». Проверьте язык C#. Для Windows Forms/WPF выбирайте современный .NET, без надписи .NET Framework. Нажмите «Далее». Если шаблона нет, остановитесь и проверьте компоненты Visual Studio с преподавателем.

ЧТО ПРОВЕРИТЬ

Выбран нужный шаблон C# современного .NET; открыта настройка имени проекта.



Выбор современного шаблона WPF Application на C# в Visual Studio. Не выбирайте вариант .NET Framework.

ШАГ 3 ИЗ 24

Задайте имя и папку MyApp

Имя отличает проект в Обозревателе решений, расположение определяет папку исходников на диске, платформа — способ сборки.

1. В поле «Имя проекта» введите **MyApp**. В «Расположение» нажмите кнопку обзора и выберите свою рабочую папку на рабочем столе **Windows**. Нажмите «Далее». Выберите **.NET 10** → «Создать». Дождитесь Обозревателя решений с проектом **MyApp**.

ЧТО ПРОВЕРИТЬ

В Обозревателе решений появился проект с заданным именем и выбранной платформой **.NET 10**.

ШАГ 4 ИЗ 24

Проверьте пустое окно

Сначала сделайте кнопку, которая отвечает на нажатие. Потом проверьте доступ к базе, добавьте вход, пазл и рабочее окно.

1. Нажмите «Отладка» → «Запуск без отладки» (Ctrl+F5). Дождитесь окна и закройте его крестиком.

ЧТО ПРОВЕРИТЬ

Своё пустое окно открылось без ошибок и закрылось крестиком.

ШАГ 5 ИЗ 24 · 1/2

Создайте окно «Вход»

Добавьте окно

Окно с именем LoginForm или LoginWindow легко найти в проекте. Назначьте его первым окном при запуске.

1. В Обозревателе решений нажмите правой кнопкой на MyApp → «Добавить» → «Окно WPF». Введите LoginWindow.xaml и нажмите «Добавить».
2. Дважды нажмите LoginWindow.xaml, выберите «Конструктор» и кликните по краю окна. Нажмите F4 и в Title введите «Вход», в Width — 540, в Height — 550.

ШАГ 5 ИЗ 24 · 2/2

Создайте окно «Вход»

Назначьте первое окно

1. Откройте App.xaml. Найдите StartupUri="MainWindow.xaml" и замените свойство значением из блока ниже.
2. Нажмите Ctrl+S, затем Ctrl+F5. Убедитесь, что открылось окно с заголовком «Вход». Закройте его крестиком.

ЧТО ПРОВЕРИТЬ

При запуске открывается пустое окно «Вход». В проекте появился файл этого окна.

ШАГ 5 · СВОЙСТВО ОКНА В APP.XAML

Создайте окно «Вход»

```
StartupUri="LoginWindow.xaml"
```

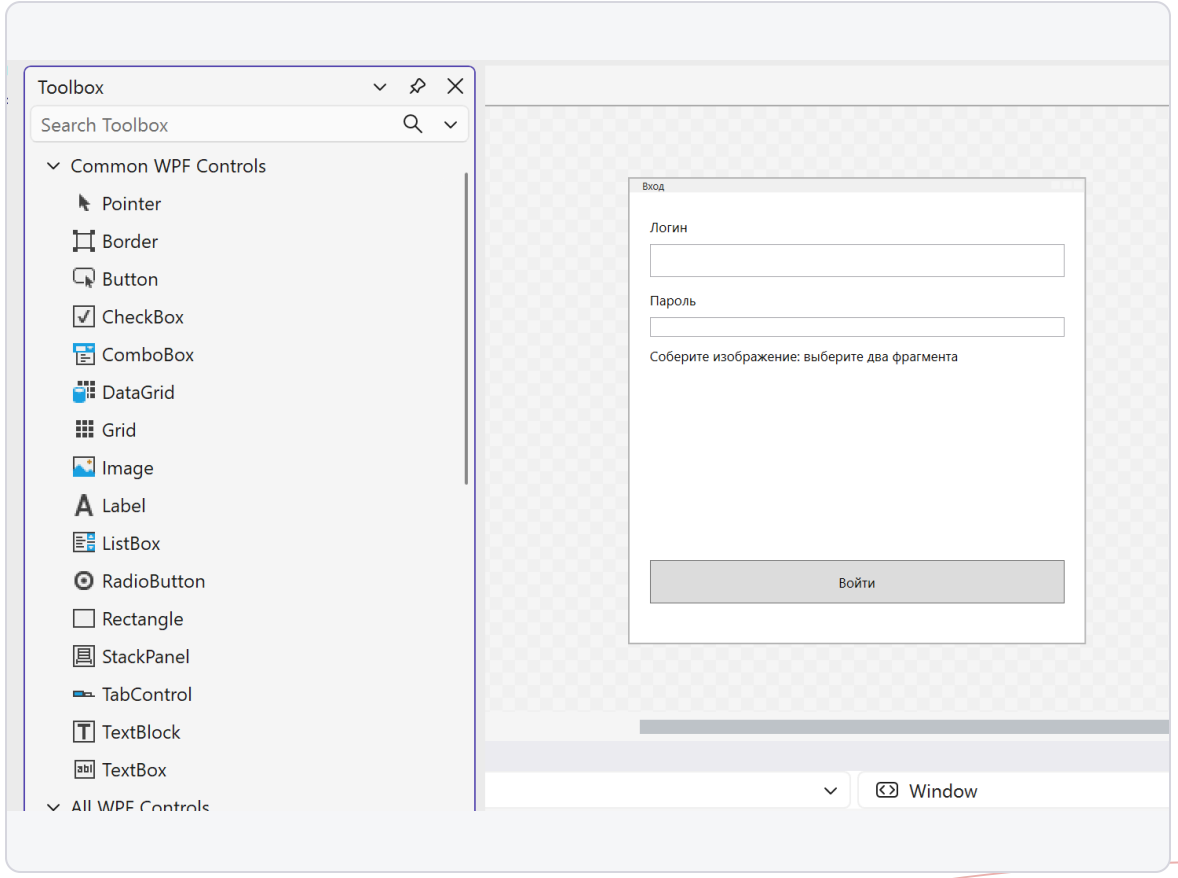
ШАГ 6 ИЗ 24 · 1/3

Добавьте поля логина и пароля

Откройте панель элементов

Элементы получают имена, по которым обработчик позже прочитает логин и пароль.

1. Откройте окно «Вход» в конструкторе. Нажмите Ctrl+Alt+X или выберите «Вид» → «Панель элементов» (Toolbox).
2. В Toolbox откройте Common WPF Controls и найдите StackPanel. Перетащите его на макет. TextBlock и TextBox находятся там же, а PasswordBox можно найти через Search Toolbox. Размещайте элементы внутри StackPanel сверху вниз.



Visual Studio, WPF. В Toolbox слева видны StackPanel, TextBlock, TextBox и Button. Справа — собранный макет LoginWindow.

ШАГ 6 ИЗ 24 · 2/3

Добавьте поля логина и пароля

Добавьте логин

1. Перетащите **TextBlock** в **StackPanel**. В свойствах **Text** задайте «Логин». Ниже добавьте **TextBox**.
2. Выделите **TextBox** → **F4** → в поле **Name** введите **LoginInput** и нажмите **Enter**.

ШАГ 6 ИЗ 24 · 3/3

Добавьте поля логина и пароля

Добавьте пароль

1. Добавьте второй **TextBlock** с текстом «Пароль». Под ним разместите **PasswordBox**. В поле **Name** введите **PasswordInput**.
2. Нажмите **Ctrl+F5**. Впишите несколько символов в оба поля. В логине символы видны, в пароле отображаются точки. Закройте окно.

ЧТО ПРОВЕРИТЬ

В окне «Вход» есть две подписи и два поля. Логин виден, пароль скрывается.

ШАГ 7 ИЗ 24 · 1/3

Добавьте кнопку и свяжите её с кодом

Поставьте кнопку под полями

Нажатие вызывает метод Click. Пока проверяем только этот переход от формы к коду.

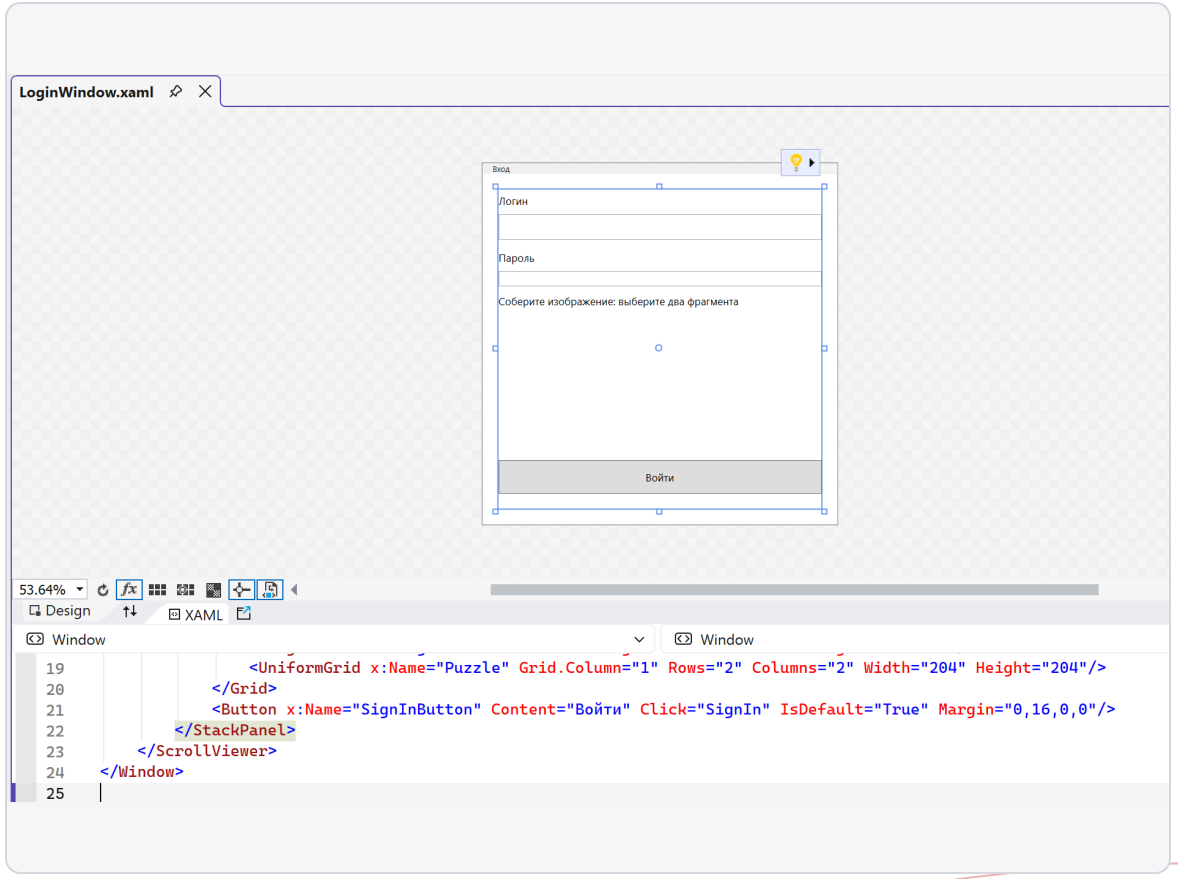
1. В Toolbox найдите Button и перетащите под полем пароля. Выделите кнопку и нажмите F4.
2. В Name введите SignInButton, в Content — «Войти». Нажмите Enter.

ШАГ 7 ИЗ 24 · 2/3

Добавьте кнопку и свяжите её с кодом

Вставьте одну строку в обработчик

1. Дважды щёлкните кнопку на макете. Visual Studio откроет файл C# и создаст метод для события Click. Обычно он называется SignInButton_Click.
2. Поставьте курсор на пустую строку между фигурными скобками метода. Вставьте строку из блока ниже. Скобки и заголовок метода оставьте.
3. Сохраните Ctrl+S, запустите Ctrl+F5 и нажмите «Войти». Если ничего не произошло, вернитесь в конструктор: кнопка → F4 → значок молнии → Click. Там должно стоять имя созданного метода.



Visual Studio, WPF. В нижней панели XAML у кнопки SignInButton видно Click="SignIn". В учебном проекте здесь должно стоять имя вашего обработчика.

ШАГ 7 ИЗ 24 · 3/3

Добавьте кнопку и свяжите её с кодом

Вставьте одну строку в обработчик

1. В WPF та же связь видна в разметке: в LoginWindow.xaml найдите Button с именем SignInButton и сверьте значение Click с именем метода в LoginWindow.xaml.cs.

ЧТО ПРОВЕРИТЬ

Нажатие на «Войти» показывает «Кнопка работает». Дальше код входа добавляется в этот же обработчик.



ШАГ 7 · ОДНА СТРОКА ВНУТРИ CLICK

Добавьте кнопку и свяжите её с кодом

```
MessageBox.Show("Кнопка работает");
```

ШАГ 8 ИЗ 24

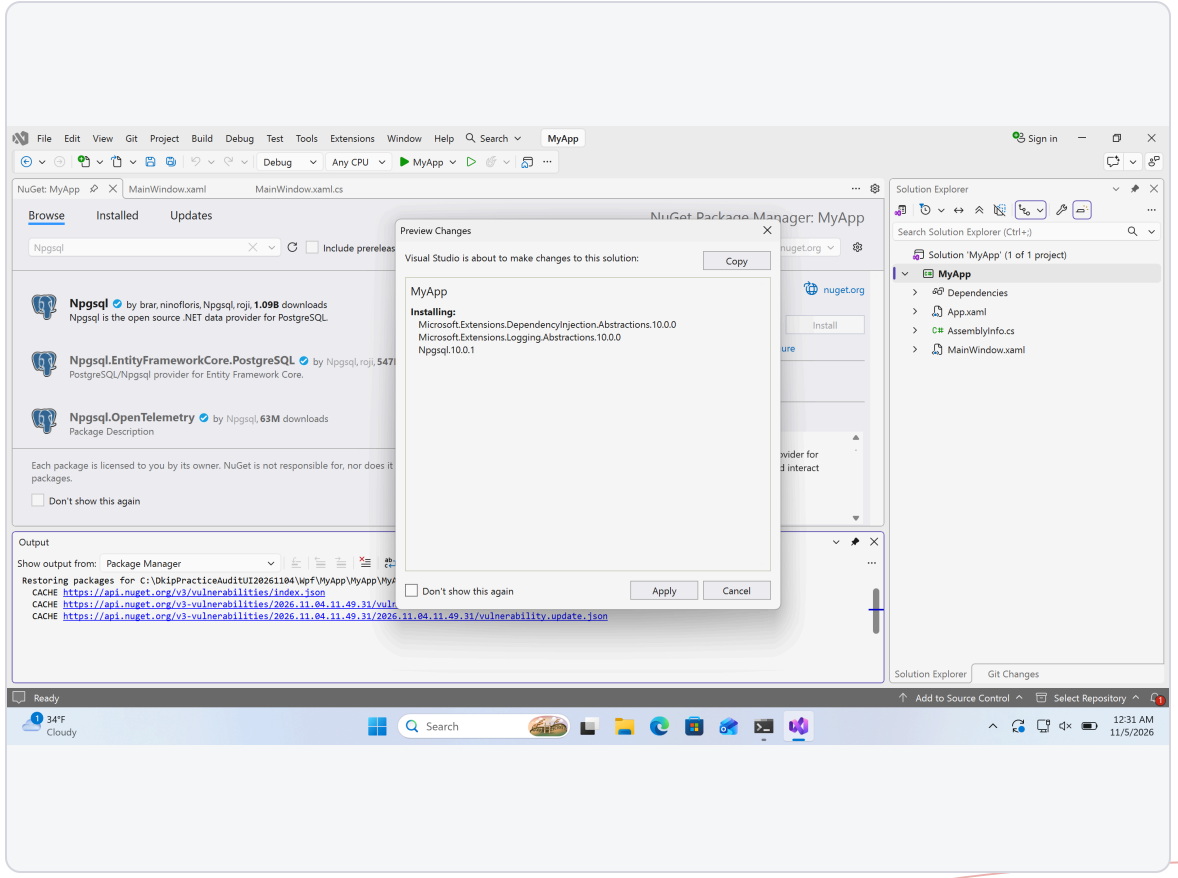
Откройте пакеты MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений нажмите правой кнопкой именно по проекту MyApp → «Управление пакетами NuGet». Перейдите на вкладку «Обзор» (Browse). Введите Npgsql в поиск и нажмите Enter. Дождитесь списка результатов. Выберите пакет с точным названием Npgsql.

ЧТО ПРОВЕРИТЬ

Открыт NuGet → Browse; найден пакет провайдера с точным названием.



Окно Preview Changes для Npgsql 10.0.1. Apply подтверждает установку показанного пакета и зависимостей.

ШАГ 9 ИЗ 24

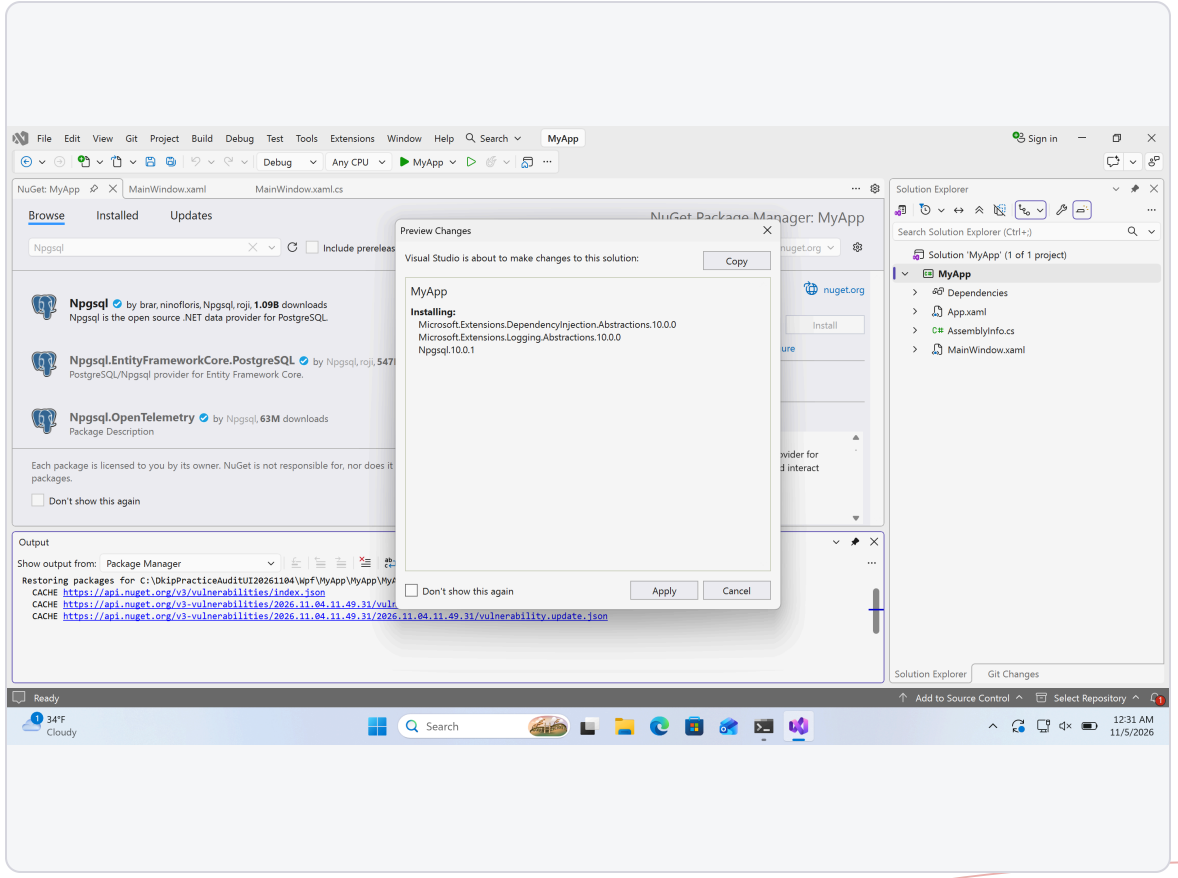
Выберите версию MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. **NuGet** устанавливает её зависимость в конкретный проект.

1. В правой панели **NuGet** найдите **Version**. Нажмите стрелку списка и выберите готовый пункт **10.0.1**. Выбор из списка связывает версию с пакетом; простое изменение текста в поле может оставить кнопку **Install** недоступной.

ЧТО ПРОВЕРИТЬ

В **Version** выбрана нужная версия; кнопка **Install** доступна.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 10 ИЗ 24

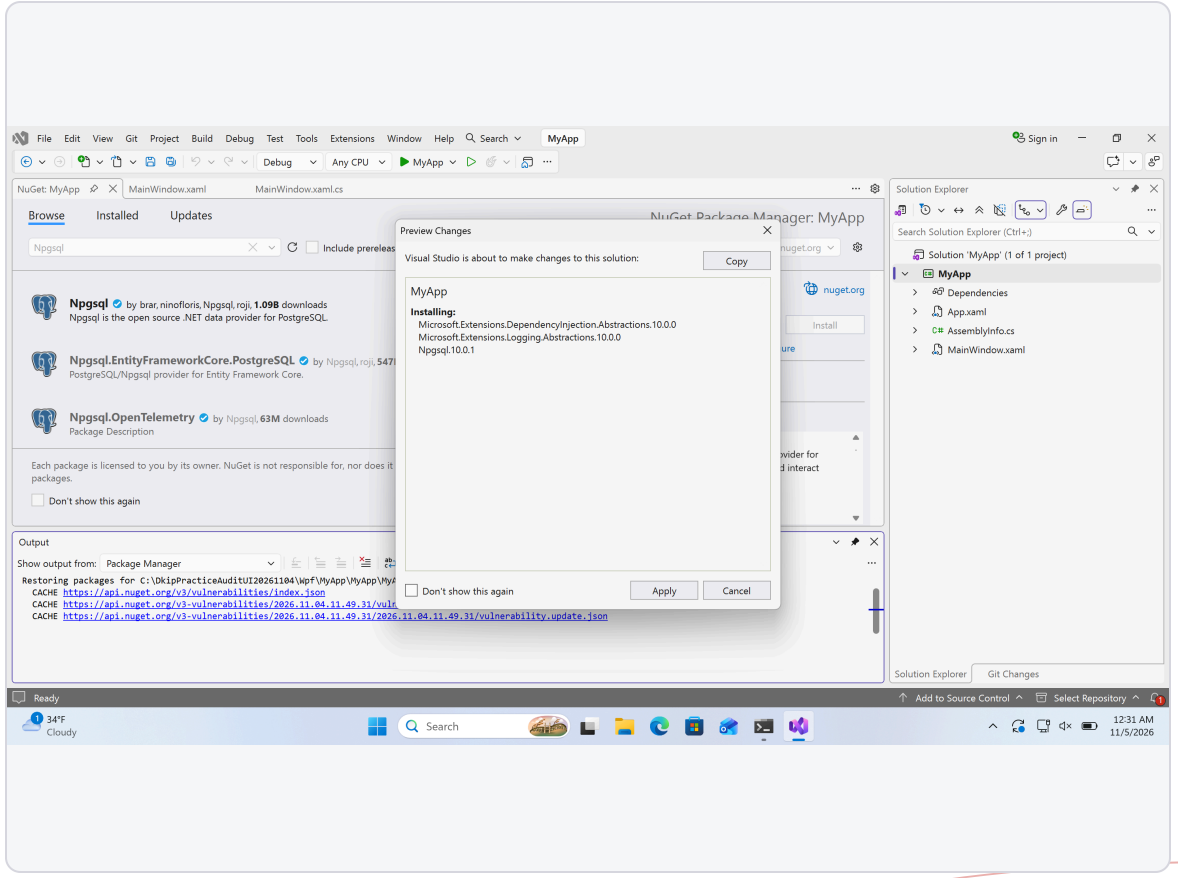
Установите провайдер MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. **NuGet** устанавливает её зависимость в конкретный проект.

1. Нажмите **Install** («Установить»). Если появилось окно **Preview Changes** («Просмотр изменений»), сверяйте название пакета и версию, затем нажмите **Apply** («Применить»). Дождитесь окончания установки. Не закрывайте **Visual Studio** во время загрузки.

ЧТО ПРОВЕРИТЬ

Установка завершилась; в карточке пакета показана установленная версия.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 11 ИЗ 24

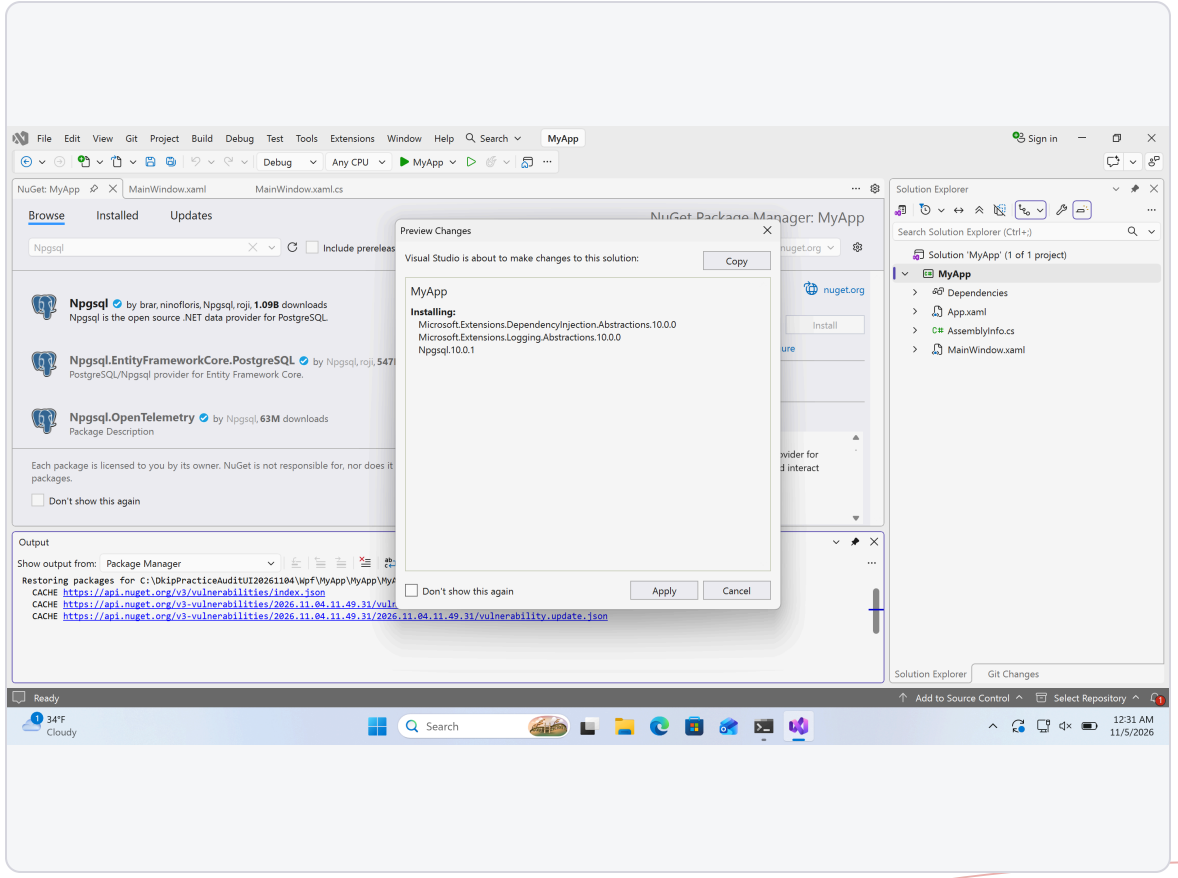
Проверьте провайдер MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений раскройте **Dependencies** («Зависимости») → **Packages** («Пакеты»). Должен появиться **Npgsql** выбранной версии. Нажмите **Build** («Сборка») → **Build Solution** («Собрать решение»). В нижнем окне не должно быть ошибок.

ЧТО ПРОВЕРИТЬ

Провайдер появился в **Dependencies** → **Packages**; сборка решения завершилась без ошибок.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 12 ИЗ 24

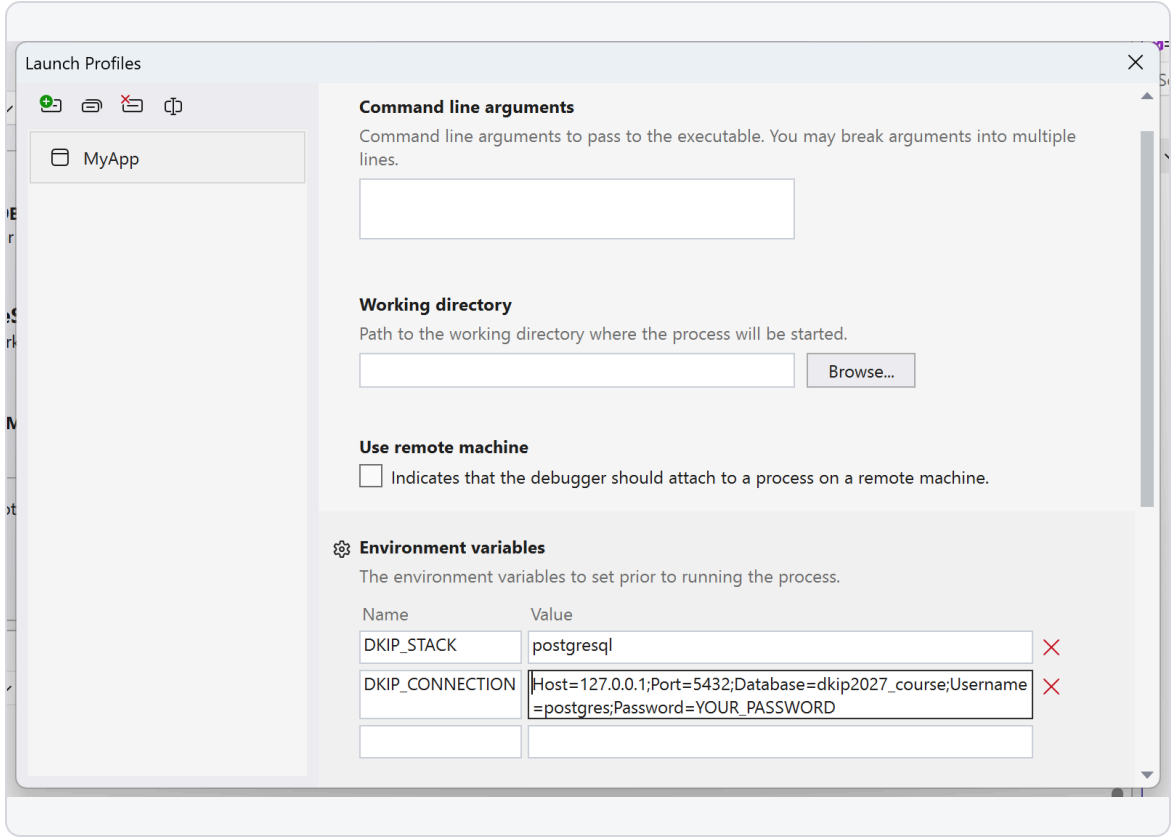
Откройте профиль подключения MyApp

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Выделите проект MyApp в Обозревателе решений. Выберите меню «Отладка» → «Свойства отладки MyApp». Другой путь: свойства проекта → «Отладка» → «Открыть профили запуска отладки». Выберите профиль с запуском самого проекта.

ЧТО ПРОВЕРИТЬ

Открыто окно профилей запуска именно нужного проекта; видна таблица **Environment variables**.



Профиль запуска Visual Studio: DKIP_STACK=postgresql и DKIP_CONNECTION в Environment variables. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 13 ИЗ 24 · 1/2

Введите параметры подключения MyApp

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Прокрутите окно профиля до **Environment variables**. В первой пустой строке нажмите поле **Name**, введите **DKIP_CONNECTION** и нажмите **Tab**. Скопируйте строку ниже в поле **Value**. Замените имя базы, порт и учётные данные значениями своей учебной установки. Подставьте пароль пользователя **PostgreSQL** вместо ВАШ_ПАРОЛЬ.

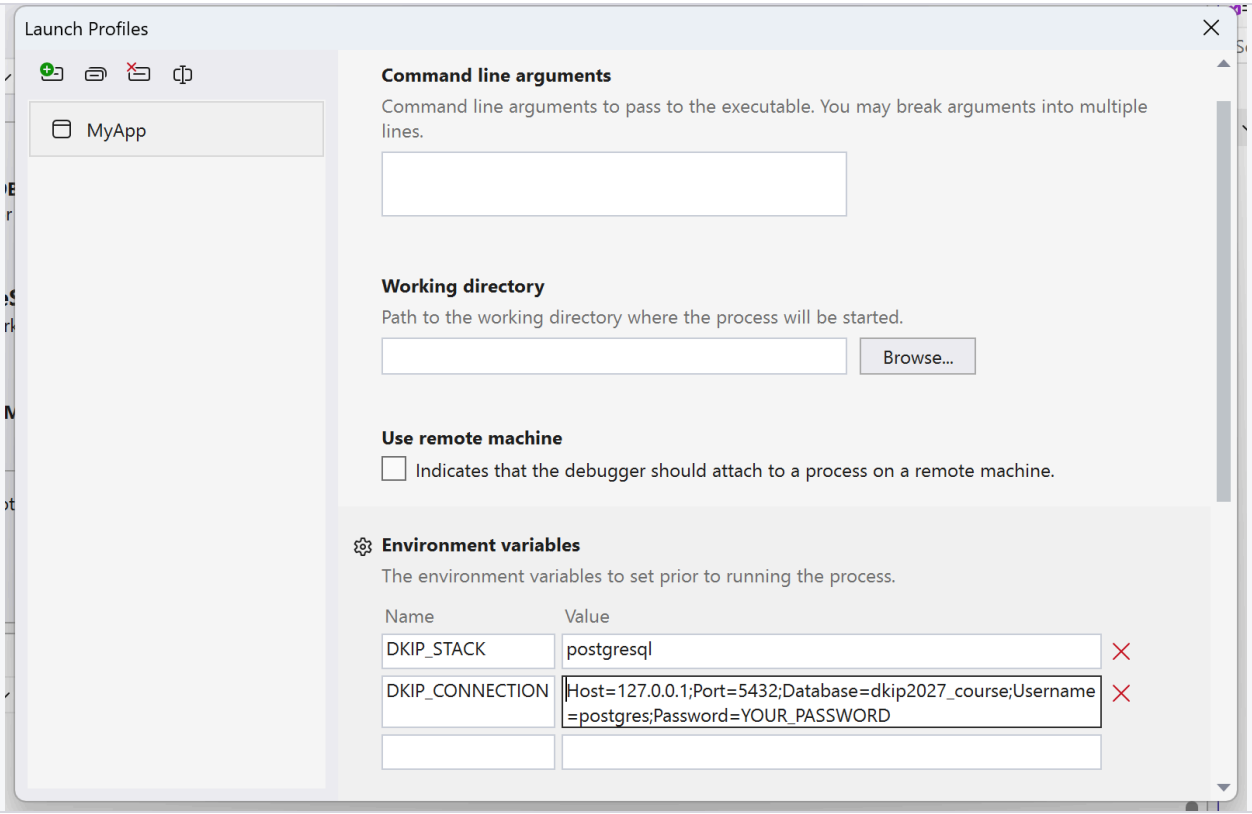
ЧТО ПРОВЕРИТЬ

В **DKIP_CONNECTION** указаны параметры своей учебной базы, без незаменённых заглушек.

ШАГ 13 ИЗ 24 · 2/2

Введите параметры подключения MyApp

Покажите на экране



Профиль запуска **Visual Studio**: **DKIP_STACK=postgresql** и **DKIP_CONNECTION** в **Environment variables**. **YOUR_PASSWORD** — заглушка, замените её паролем своего сервера.

ШАГ 13 · КОД ДЛЯ ЭТОГО ШАГА

Введите параметры подключения MyApp

```
Host=127.0.0.1;Port=5432;Database=dkip2027_course;Username=postgres;Password=ВАШ_ПАРОЛЬ
```

ШАГ 14 ИЗ 24

Подключите проект к своей базе

Перед проверкой входа убедитесь, что приложение видит таблицу **users** из задания 2.

1. Правой кнопкой по **MyApp** → «Добавить» → «Класс». Введите **Database.cs**. Вставьте полный код из кнопки «Открыть Database.cs», заменив содержимое созданного файла. Проверьте **namespace MyApp** в начале файла.
2. Дважды щёлкните «Войти» в конструкторе. Внутри **Click** замените старую строку кодом ниже. Нажмите **Ctrl+S** и **Ctrl+F5**. Сравните число с количеством строк **users** в редакторе базы.

ЧТО ПРОВЕРИТЬ

По нажатию кнопки выводится число строк таблицы **users**, включая **0** для пустой таблицы.

ШАГ 14 · ВРЕМЕННАЯ ПРОВЕРКА ПОДКЛЮЧЕНИЯ

Подключите проект к своей базе

```
MessageBox.Show(Database.Scalar("SELECT COUNT(*) FROM users")?.ToString());
```



ШАГ 14 · DATABASE.CS · 1/4

Подключите проект к своей базе

```
using System.Data;
using System.Data.Common;
using Npgsql;
namespace MyApp;

public static class Database
{
    public static DbConnection Open()
    {
        // Профиль запуска передаёт строку подключения при запуске приложения.
        var connectionString = Environment.GetEnvironmentVariable("DKIP_CONNECTION");
        if (string.IsNullOrWhiteSpace(connectionString))
            throw new InvalidOperationException("Укажите DKIP_CONNECTION в профиле запуска.");

        var connection = new NpgsqlConnection(connectionString);
        try
```



ШАГ 14 · DATABASE.CS · 2/4

Подключите проект к своей базе

```
        {
            connection.Open();
            return connection;
        }
        catch { connection.Dispose(); throw; }
    }
    public static DbCommand Command(DbConnection connection, string sql, params object?[] values)
    {
        var cmd = connection.CreateCommand();
        cmd.CommandText = sql;
        for (int i = 0; i < values.Length; i++)
        {
            var p = cmd.CreateParameter();
            p.ParameterName = "p" + i;
            p.Value = values[i] ?? DBNull.Value;
```



ШАГ 14 · DATABASE.CS · 3/4

Подключите проект к своей базе

```
            cmd.Parameters.Add(p);
        }
        return cmd;
    }
    public static int Execute(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        return cmd.ExecuteNonQuery();
    }
    public static DataTable Query(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        using var reader = cmd.ExecuteReader();
        var result = new DataTable();
```



ШАГ 14 · DATABASE.CS · 4/4

Подключите проект к своей базе

```
        result.Load(reader);
        return result;
    }
    public static object? Scalar(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        return cmd.ExecuteScalar();
    }
}
```

ШАГ 15 ИЗ 24

Заполните пустую таблицу двумя учебными записями

Для проверки нужны обе роли. Две строки можно добавить в редакторе базы и сразу приступить к окнам.

1. В pgAdmin выберите свою учебную базу, затем «Инструменты» → Query Tool.
2. Если users уже содержит подходящие тестовые записи с обеими ролями, используйте их. Для пустой users вставьте SQL из блока и выполните его один раз.
3. Проверьте строки в users. Логины для окна «Вход» — admin и student. Для обеих записей учебный пароль Demo2027!.

ЧТО ПРОВЕРИТЬ

В users есть admin с ролью admin и student с ролью user. Счётчики равны нулю, обе записи разблокированы.

ШАГ 15 · SQL ДЛЯ ПУСТОЙ УЧЕБНОЙ ТАБЛИЦЫ USERS

Заполните пустую таблицу двумя учебными записями

```
INSERT INTO users (login, password_hash, role, failed_attempts, is_locked)
VALUES
    ('admin', 'ZGtpcC1kZW1vLWFkbWluIQ==:razCURk4WRm93+4n6JRjHafFWVGLT9mE/zK6yM6vFq8=', 'admin', 0, FALSE),
    ('student', 'ZGtpcC1kZW1vLXN0dWQuIQ==:oGrwAbgIg57bmM8lZOkiV+Um0GPIyCSS4+sb9YweW+A=', 'user', 0, FALSE);
```

ШАГ 16 ИЗ 24

Проверьте логин и пароль

Отработайте поиск записи и проверку пароля отдельно. Тогда ошибку сборки пазла будет проще отличить от ошибки входа.

1. Правой кнопкой по **MyApp** → «Добавить» → «Класс» → **Accounts.cs**. Замените всё содержимое кодом из «Открыть Accounts.cs». В строке **namespace Dkip** замените **Dkip** на **MyApp**.
2. Замените временный вывод числа строк блоком ниже. Временно передайте **true** вместо результата пазла. На следующем этапе это значение заменится настоящей проверкой.
3. Проверьте пустые поля, один неверный пароль и правильный пароль для **student**. После правильного входа предыдущая ошибка должна сброситься.

ЧТО ПРОВЕРИТЬ

Кнопка выводит логин при верных данных. При пустых полях и неверном пароле видны разные сообщения.



ШАГ 16 · ВНУТРИ ОБРАБОТЧИКА CLICK

Проверьте логин и пароль

```
try
{
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, true);
    MessageBox.Show(user.Login);
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message);
}
catch (System.Data.Common.DbException)
{
    MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
}
```

ШАГ 16 · ACCOUNTS.CS · 1/9

Проверьте логин и пароль

```
using System.Security.Cryptography;
namespace Dkip;

public record Account(int Id, string Login, string Role);
public static class Accounts
{
    public const string Wrong = "Вы ввели неверный логин или пароль. Пожалуйста проверьте ещё раз введенные данные";
    public const string Locked = "Вы заблокированы. Обратитесь к администратору";
    const int MaxAttempts = 3;

    // Соль хранится рядом с хешем. Сам пароль в БД не записываем.
    public static string Hash(string password)
    {
        var salt = RandomNumberGenerator.GetBytes(16);
        var hash = DeriveHash(password, salt);
```

ШАГ 16 · ACCOUNTS.CS · 2/9

Проверьте логин и пароль

```
        return Convert.ToBase64String(salt) + ":" + Convert.ToBase64String(hash);
    }
    public static bool Verify(string password, string hash)
    {
        try
        {
            var parts = hash.Split(':');
            if (parts.Length != 2)
                return false;

            var salt = Convert.FromBase64String(parts[0]);
            var savedHash = Convert.FromBase64String(parts[1]);
            return CryptographicOperations.FixedTimeEquals(savedHash, DeriveHash(password, salt));
        }
        catch (FormatException)
```



ШАГ 16 · ACCOUNTS.CS · 3/9

Проверьте логин и пароль

```
        {
            return false;
        }
    }

    static byte[] DeriveHash(string password, byte[] salt) =>
        Rfc2898DeriveBytes.Pbkdf2(password, salt, 100000, HashAlgorithmName.SHA256, 32);

    public static Account SignIn(string login, string password, bool puzzleSolved)
    {
        login = login.Trim().ToLowerInvariant();
        if (login.Length == 0 || password.Length == 0)
            throw new InvalidOperationException("Заполните логин и пароль.");
        // Пока проверяем вход, другая попытка не может изменить эту строку.
        using var connection = Database.Open();
        using var transaction = connection.BeginTransaction();
```



ШАГ 16 · ACCOUNTS.CS · 4/9

Проверьте логин и пароль

```
        using var findUser = Database.Command(connection,
            "SELECT id, password_hash, role, failed_attempts, is_locked FROM users WHERE login=@p0 FOR UPDATE", login);
        findUser.Transaction = transaction;

        int id;
        int attempts;
        string savedHash;
        string role;
        using (var reader = findUser.ExecuteReader())
        {
            if (!reader.Read())
                throw new InvalidOperationException(Wrong);
            if (Convert.ToBoolean(reader["is_locked"]))
                throw new InvalidOperationException(Locked);
```



ШАГ 16 · ACCOUNTS.CS · 5/9

Проверьте логин и пароль

```
            id = Convert.ToInt32(reader["id"]);
            savedHash = (string)reader["password_hash"];
            role = (string)reader["role"];
            attempts = Convert.ToInt32(reader["failed_attempts"]);
        }

        bool passwordCorrect = Verify(password, savedHash);
        bool accepted = passwordCorrect && puzzleSolved;

        // Пароль и пазл делят один счётчик. Успешный вход его обнуляет.
        if (accepted)
            attempts = 0;
        else
            attempts++;

        bool blocked = attempts >= MaxAttempts;
```

ШАГ 16 · ACCOUNTS.CS · 6/9

Проверьте логин и пароль

```
        using var update = Database.Command(connection,
            "UPDATE users SET failed_attempts=@p0, is_locked=@p1 WHERE id=@p2", attempts, blocked, id);
        update.Transaction = transaction;
        update.ExecuteNonQuery();
        // Сохраняем результат даже при ошибке, иначе попытка потеряется.
        transaction.Commit();

        if (blocked)
            throw new InvalidOperationException(Locked);
        if (!passwordCorrect)
            throw new InvalidOperationException(Wrong);
        if (!puzzleSolved)
            throw new InvalidOperationException("Пазл собран неверно. Осталось попыток: " + (MaxAttempts - attempts));
```



ШАГ 16 · ACCOUNTS.CS · 7/9

Проверьте логин и пароль

```
        return new Account(id, login, role);
    }

    public static void RequireAdmin(Account actor)
    {
        if (!Equals(Database.Scalar("SELECT role FROM users WHERE id=@p0 AND is_locked=FALSE", actor.Id), "admin"))
            throw new InvalidOperationException("Действие доступно только администратору.");
    }
    public static void Save(Account actor, int? id, string login, string password, string role, bool unlock)
    {
        RequireAdmin(actor);
        login = login.Trim().ToLowerInvariant();
        if (login.Length is < 1 or > 64 || role is not ("admin" or "user"))
```

ШАГ 16 · ACCOUNTS.CS · 8/9

Проверьте логин и пароль

```
        throw new InvalidOperationException("Введите логин до 64 символов и выберите роль.");
    if (id is null && password.Length == 0)
        throw new InvalidOperationException("Введите пароль нового пользователя.");
    if (id == actor.Id && role != "admin")
        throw new InvalidOperationException("Нельзя снять собственную роль администратора.");
    if (Convert.ToInt32(Database.Scalar("SELECT COUNT(*) FROM users WHERE login=@p0 AND id<>@p1", login, id ?? -1)) > 0)
        throw new InvalidOperationException("Пользователь с указанным логином уже существует.");
    if (id is null)
    {
        Database.Execute(
            "INSERT INTO users(login, password_hash, role, failed_attempts, is_locked) VALUES(@p0,@p1,@p2,0,FALSE)",
```

ШАГ 16 · ACCOUNTS.CS · 9/9

Проверьте логин и пароль

```
                login, Hash(password), role);
            return;
        }

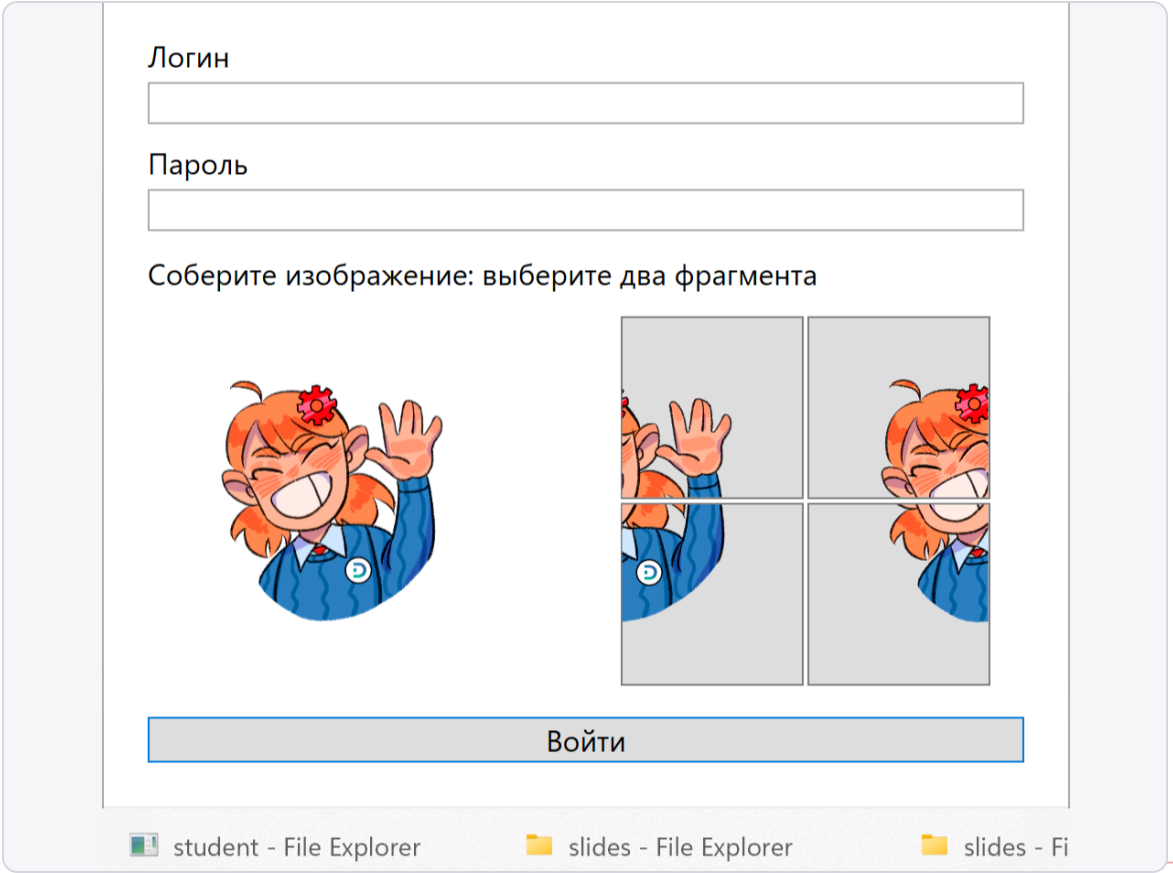
        // Пустой новый пароль сохраняет старый хеш. При разблокировке сбрасываем оба поля.
        string? newHash = password.Length > 0 ? Hash(password) : null;
        Database.Execute("""
            UPDATE users SET login=@p0, role=@p1, password_hash=COALESCE(@p2, password_hash),
                failed_attempts=CASE WHEN @p3 THEN 0 ELSE failed_attempts END,
                is_locked=CASE WHEN @p3 THEN FALSE ELSE is_locked END
            WHERE id=@p4
            """, login, role, newHash, unlock, id);
    }
}
```

ШАГ 17 ИЗ 24 · 1/2

Покажите фрагменты пазла

Четыре исходных PNG уже являются частями одной картинки. Правильный порядок нужен и для образца, и для проверки.

1. В Обозревателе решений нажмите правой кнопкой по MyApp → «Добавить» → «Новая папка», назовите Assets. Правой кнопкой по Assets → «Добавить» → «Существующий элемент». Выберите 1.png–4.png из исходных файлов задания 4.
2. Выделите добавленные PNG и нажмите F4. В «Копировать в выходной каталог» выберите «Копировать, если новее». Так программа найдёт PNG рядом с EXE.



WPF. Окно «Вход»: поля логина и пароля, образец пазла слева и четыре фрагмента справа.

ШАГ 17 ИЗ 24 · 2/2

Покажите фрагменты пазла

1. Откройте **LoginWindow.xaml**. Во вкладке **XAML** найдите тег **Button**. Вставьте перед ним разметку первого блока, внутри **StackPanel**. У кнопки укажите **Click="SignInButton_Click"**.
2. В Обозревателе решений раскройте **LoginWindow.xaml** и откройте **LoginWindow.xaml.cs**. Замените всё содержимое вторым блоком. В нём уже есть прежняя проверка пароля и готовые методы пазла.
3. Запустите приложение. Слева сравните образец с файлами. Справа проверьте, что все четыре части видны и расположены в другом порядке.

ЧТО ПРОВЕРИТЬ

Окно показывает исходное изображение и перемешанную сетку 2 × 2.

ШАГ 17 · 1. РАЗМЕТКА ПАЗЛА В LOGINWINDOW.XAML

Покажите фрагменты пазла

```
<Grid Height="204">
    <Grid.ColumnDefinitions>
        <ColumnDefinition/>
        <ColumnDefinition/>
    </Grid.ColumnDefinitions>
    <Image x:Name="Original" Width="200" Height="200"/>
    <UniformGrid x:Name="Puzzle" Grid.Column="1" Rows="2" Columns="2"/>
</Grid>
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 1/8

Покажите фрагменты пазла

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace MyApp;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 2/8

Покажите фрагменты пазла

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 3/8

Покажите фрагменты пазла

```
        var visual = new DrawingVisual();
        using (var drawing = visual.RenderOpen())
        {
            for (int i = 0; i < 4; i++)
            {
                var image = new BitmapImage();
                image.BeginInit();
                image.CacheOption = BitmapCacheOption.OnLoad;
                image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
                image.EndInit();
                image.Freeze();
                pieces[i] = image;
                drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
            }
        }
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 4/8

Покажите фрагменты пазла

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 5/8

Покажите фрагменты пазла

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```



ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 6/8

Покажите фрагменты пазла

```
bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

void RenderPuzzle()
{
    for (int i = 0; i < 4; i++)
    {
        tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
        tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
        tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
    }
}
void Shuffle()
{
    do
    {
        Random.Shared.Shuffle(order);
```



ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 7/8

Покажите фрагменты пазла

```
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignInButton_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, true);
            MessageBox.Show(user.Login);
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message);
        }
        catch (System.Data.Common.DbException)
```

ШАГ 17 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 8/8

Покажите фрагменты пазла

```
        {
            MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
        }
    }
}
```

ШАГ 18 ИЗ 24

Поменяйте местами две части

Первое нажатие запоминает ячейку. Второе меняет местами две части и обновляет картинки.

1. Найдите в коде метод **SelectTile** и массив **order**. Массив хранит номера частей в порядке их показа.
2. Щёлкните по двум соседним ячейкам. После первого щелчка видна рамка. После второго части меняются местами, а рамка исчезает.
3. Соберите рисунок по образцу. Метод **IsSolved** должен вернуть **true**, когда порядок стал 0, 1, 2, 3.

ЧТО ПРОВЕРИТЬ

Первая часть выделяется, вторая меняется с ней местами. Пазл можно собрать полностью.

ШАГ 18 · ОБМЕН ДВУХ ЭЛЕМЕНТОВ В SELECTTILE

Поменяйте местами две части

```
int fragment = order[selected];
order[selected] = order[position];
order[position] = fragment;
selected = -1;
```

ШАГ 19 ИЗ 24

Свяжите пазл с входом и проверьте блокировку

В методе SignIn неправильный пароль и неверный пазл учитываются одинаково. Состояние хранится в users.

1. Дважды щёлкните «Войти» в конструкторе. Замените содержимое обработчика блоком ниже. В третьем аргументе теперь проверяется пазл. После неудачного входа он перемешивается.
2. На тестовой записи student дважды введите неверный пароль. В третьей попытке введите правильный пароль, но оставьте пазл несобранным.
3. Закройте и снова запустите приложение. Повторите вход с верными данными и собранным пазлом. В users проверьте failed_attempts и is_locked.

ЧТО ПРОВЕРИТЬ

На третьей ошибке и при повторном входе появляется сообщение о блокировке. В users осталось 3 и признак блокировки.

ШАГ 19 · ГОТОВЫЙ КОД ВНУТРИ CLICK С ПРОВЕРКОЙ ПАЗЛА

Свяжите пазл с входом и проверьте блокировку

```
try
{
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
    MessageBox.Show(user.Login);
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message);
    Shuffle();
}
catch (System.Data.Common.DbException)
{
    MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
}
```

ШАГ 20 ИЗ 24 · 1/4

Соберите рабочее окно

Откройте конструктор рабочего окна

Рабочее окно показывает логин и роль. Таблица и поля управления стоят на отдельной панели администратора.

1. Откройте **MainWindow.xaml**, созданный вместе с проектом. Из **Toolbox** добавьте **StackPanel**, затем поместите в него **TextBlock**. В **Name** укажите **RoleLabel**.
2. Ниже подписи поместите второй **StackPanel** внутрь первого. В **Name** введите **AdminPanel**. Все следующие элементы добавляйте внутрь него.

ШАГ 20 ИЗ 24 · 2/4

Соберите рабочее окно

Поставьте таблицу и поля

1. Добавьте в AdminPanel DataGrid. В свойствах укажите имя UsersGrid и включите IsReadOnly.
2. Под таблицей добавьте подписи «Логин», «Новый пароль» и «Роль». Для логина перетащите TextBox с именем UserLogin. Для пароля добавьте PasswordBox с именем NewPassword.
3. Добавьте ComboBox с именем UserRole. Откройте Items в свойствах. Добавьте сначала «Пользователь», потом «Администратор». Установите SelectedIndex=0.

ШАГ 20 ИЗ 24 · 3/4

Соберите рабочее окно

Поставьте таблицу и поля

1. Если редактор Items недоступен, во вкладке XAML добавьте внутрь ComboBox два тега ComboBoxItem. Укажите Content="Пользователь", затем Content="Администратор".

ШАГ 20 ИЗ 24 · 4/4

Соберите рабочее окно

Поставьте флажок и кнопки

1. Из **Toolbox** перетащите **CheckBox**. Задайте имя **Unlock** и подпись «Снять блокировку» в **Content**.
2. Добавьте два **Button**. Для первого задайте имя **NewButton** и подпись «Новый». Для второго — **SaveButton** и подпись «Сохранить». Нажмите **Ctrl+S**.

ЧТО ПРОВЕРИТЬ

В конструкторе видны подпись, таблица, три поля, флажок и две кнопки. Их имена совпадают с инструкцией.

ШАГ 21 ИЗ 24 · 1/2

Подключите таблицу и кнопки администратора

Вставьте готовый код рабочего окна

SelectUser читает выбранную строку, NewUser очищает поля, SaveUser вызывает запись в базу.

1. Раскройте MainWindow.xaml в Обозревателе решений. Откройте MainWindow.xaml.cs и замените его кодом из кнопки ниже. Разметку MainWindow.xaml оставьте своей.
2. Замените namespace Dkip на MyApp. Сохраните. Код использует имена, которые вы задали в свойствах.

ШАГ 21 ИЗ 24 · 2/2

Подключите таблицу и кнопки администратора

Назначьте события в свойствах

1. Выделите **UsersGrid**. Нажмите **F4**, затем значок молнии. В строке **SelectionChanged** выберите **SelectUser**.
2. Выделите **NewButton**. В событии **Click** выберите **NewUser**. Для **SaveButton** в событии **Click** выберите **SaveUser**.
3. Нажмите «Сборка» → «Собрать решение». Если имя не найдено, сверьте **Name** элемента и название обработчика.

ЧТО ПРОВЕРИТЬ

Решение собирается. Список и обе кнопки связаны со своими обработчиками.



ШАГ 21 · MAINWINDOW.XAML.CS · 1/7

Подключите таблицу и кнопки администратора

```
using System.Data;
using System.Data.Common;
using System.Windows;
using System.Windows.Controls;

namespace Dkip;

public partial class MainWindow : Window
{
    private readonly Account actor = null!;
    private int? editingId;

    public MainWindow()
    {
        InitializeComponent();
    }
```

ШАГ 21 · MAINWINDOW.XAML.CS · 2/7

Подключите таблицу и кнопки администратора

```
public MainWindow(Account user) : this()
{
    actor = user;
    RoleLabel.Text = user.Login + " · " + (user.Role == "admin" ? "Администратор" : "Пользователь");
    AdminPanel.Visibility = user.Role == "admin" ? Visibility.Visible : Visibility.Collapsed;
    if (user.Role == "admin")
        LoadUsers();
}

private void LoadUsers()
{
    try
    {
```

ШАГ 21 · MAINWINDOW.XAML.CS · 3/7

Подключите таблицу и кнопки администратора

```
            Accounts.RequireAdmin(actor);
            UsersGrid.ItemsSource = Database.Query(
                "SELECT id, login, role, failed_attempts, is_locked FROM users ORDER BY id").DefaultView;
        }
        catch (InvalidOperationException ex)
        {
            ShowWarning(ex.Message);
        }
        catch (DbException)
        {
            ShowDatabaseError();
        }
    }

    private void SelectUser(object sender, SelectionChangedEventArgs e)
```

ШАГ 21 · MAINWINDOW.XAML.CS · 4/7

Подключите таблицу и кнопки администратора

```
{
    if (UsersGrid.SelectedItem is not DataRowView row)
        return;

    editingId = Convert.ToInt32(row["id"]);
    UserLogin.Text = (string)row["login"];
    UserRole.SelectedIndex = (string)row["role"] == "admin" ? 1 : 0;
    NewPassword.Clear();
    Unlock.IsChecked = false;
}

private void NewUser(object sender, RoutedEventArgs e)
{
    // Без выбранного id кнопка «Сохранить» добавит новую запись.
    editingId = null;
    UsersGrid.UnselectAll();
```

ШАГ 21 · MAINWINDOW.XAML.CS · 5/7

Подключите таблицу и кнопки администратора

```
        UserLogin.Clear();
        NewPassword.Clear();
        UserRole.SelectedIndex = 0;
        Unlock.IsChecked = false;
    }

    private void SaveUser(object sender, RoutedEventArgs e)
    {
        try
        {
            string role = UserRole.SelectedIndex == 1 ? "admin" : "user";
            Accounts.Save(actor, editingId, UserLogin.Text, NewPassword.Password, role, Unlock.IsChecked == true);
            NewUser(sender, e);
            LoadUsers();
```

ШАГ 21 · MAINWINDOW.XAML.CS · 6/7

Подключите таблицу и кнопки администратора

```
            MessageBox.Show("Данные сохранены", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);
        }
        catch (InvalidOperationException ex)
        {
            ShowWarning(ex.Message);
        }
        catch (DbException)
        {
            ShowDatabaseError();
        }
    }

    private void ShowWarning(string message) =>
        MessageBox.Show(message, "Проверьте данные", MessageBoxButton.OK, MessageBoxImage.Warning);
```

ШАГ 21 · MAINWINDOW.XAML.CS · 7/7

Подключите таблицу и кнопки администратора

```
    private void ShowDatabaseError() =>
        MessageBox.Show("Проверьте подключение к базе данных и повторите действие.",
            "Ошибка базы данных", MessageBoxButton.OK, MessageBoxImage.Error);
}
```

ШАГ 22 ИЗ 24 · 1/2

Откройте рабочее окно после входа

Обработчик Click считывает поля, вызывает Accounts.SignIn и открывает окно при успехе.

1. Откройте конструктор окна «Вход». Дважды щёлкните «Войти» и выделите весь код между фигурными скобками метода Click.
2. Вставьте код из блока ниже. Название метода и скобки оставьте. В начало файла добавьте строку using System.Data.Common; если её ещё нет.
3. Нажмите Ctrl+S и Ctrl+F5. Войдите под admin и соберите пазл. После сообщения об успехе проверьте рабочее окно.

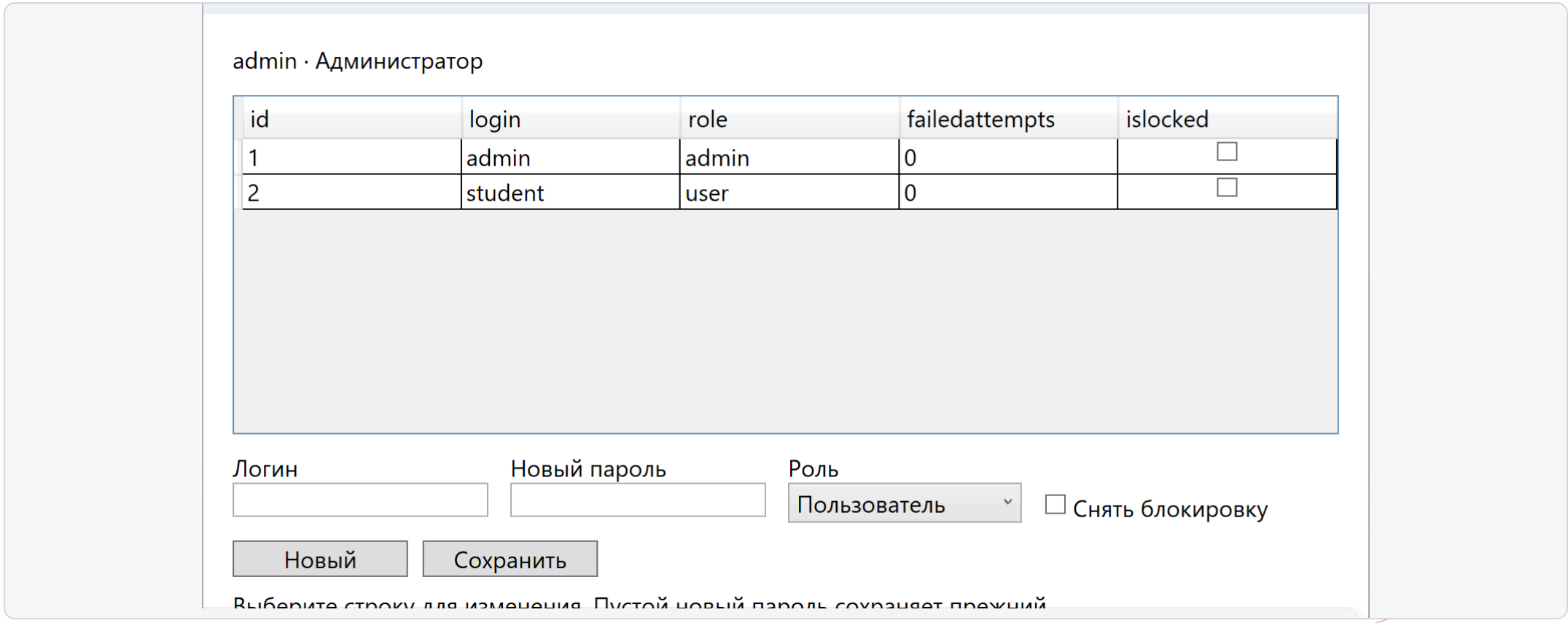
ЧТО ПРОВЕРИТЬ

Под admin открывается таблица пользователей. Для обычной записи видно только имя и роль.

ШАГ 22 ИЗ 24 · 2/2

Откройте рабочее окно после входа

Покажите на экране



WPF. Рабочее окно **admin**: таблица учётных записей, поля редактирования и кнопки «Новый» и «Сохранить».



ШАГ 22 · ГОТОВЫЙ КОД ВНУТРИ CLICK КНОПКИ «ВОЙТИ» · 1/2

Откройте рабочее окно после входа

```
try
{
    // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
    MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);

    var main = new MainWindow(user) { Owner = this };
    main.ShowDialog();
    PasswordInput.Clear();
    Shuffle();
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
    Shuffle();
```

ШАГ 22 · ГОТОВЫЙ КОД ВНУТРИ CLICK КНОПКИ «ВОЙТИ» · 2/2

Откройте рабочее окно после входа

```
}
catch (DbException)
{
    MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
        "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
}
```



ШАГ 22 · LOGINWINDOW.XAML.CS · 1/8

Откройте рабочее окно после входа

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace Dkip;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 2/8

Откройте рабочее окно после входа

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 3/8

Откройте рабочее окно после входа

```
        var visual = new DrawingVisual();
        using (var drawing = visual.RenderOpen())
        {
            for (int i = 0; i < 4; i++)
            {
                var image = new BitmapImage();
                image.BeginInit();
                image.CacheOption = BitmapCacheOption.OnLoad;
                image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
                image.EndInit();
                image.Freeze();
                pieces[i] = image;
                drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
            }
        }
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 4/8

Откройте рабочее окно после входа

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 5/8

Откройте рабочее окно после входа

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 6/8

Откройте рабочее окно после входа

```
bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

void RenderPuzzle()
{
    for (int i = 0; i < 4; i++)
    {
        tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
        tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
        tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
    }
}
void Shuffle()
{
    do
    {
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 7/8

Откройте рабочее окно после входа

```
            Random.Shared.Shuffle(order);
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignIn(object sender, RoutedEventArgs e)
    {
        try
        {
            // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
            MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);

            var main = new MainWindow(user) { Owner = this };
            main.ShowDialog();
```

ШАГ 22 · LOGINWINDOW.XAML.CS · 8/8

Откройте рабочее окно после входа

```
            PasswordInput.Clear();
            Shuffle();
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
            Shuffle();
        }
        catch (DbException)
        {
            MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
                "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

ШАГ 23 ИЗ 24

Проверьте добавление, изменение и разблокировку

Одна кнопка сохраняет оба вида изменений. Если запись не выбрана, добавляем новую. Если выбрана, меняем запись по id.

1. Под **admin** нажмите «Новый». Введите **test_student**, учебный пароль и роль «Пользователь». Нажмите «Сохранить» и найдите строку.
2. Снова нажмите «Новый» и повторите логин. Проверьте сообщение о дубликате.
3. Выберите строку **student**, включите «Снять блокировку» и сохраните. Оставьте новый пароль пустым, чтобы сохранить старый. Закройте рабочее окно и войдите под **student**.

ЧТО ПРОВЕРИТЬ

Новая запись появилась, дубликат отклонён. **student** после разблокировки входит со старым паролем.

ШАГ 24 ИЗ 24 · 1/2

Пройдите короткую проверку перед сдачей

Сценарий проверяет все развилки задания и помогает воспроизвести их по памяти.

1. Войдите как **student** с правильным паролем и пазлом. Убедитесь, что в рабочем окне нет управления пользователями.
2. Проверьте три ошибки подряд на тестовой записи, затем войдите администратором и снимите блокировку. Попробуйте создать дубликат логина.
3. Запустите тестовый профиль с неверным портом БД. Проверьте сообщение об ошибке и верните исходную настройку.

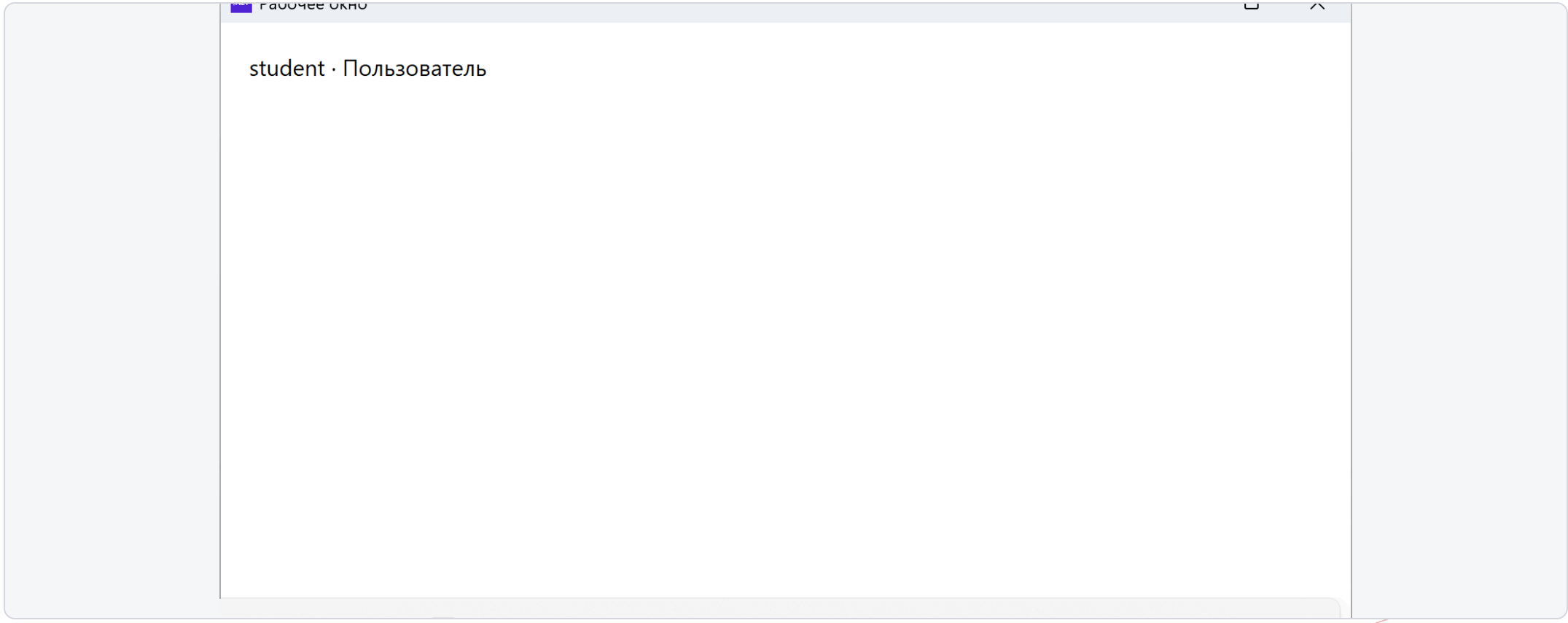
ЧТО ПРОВЕРИТЬ

Приложение выполняет сценарии входа, блокировки, прав и сохранения. При недоступной базе остаётся открытым.

ШАГ 24 ИЗ 24 · 2/2

Пройдите короткую проверку перед сдачей

Покажите на экране



WPF. Рабочее окно **student**: видны логин и роль «Пользователь».



ШАГ 24 · LOGINWINDOW.XAML.CS · 1/8

Пройдите короткую проверку перед сдачей

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace Dkip;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 2/8

Пройдите короткую проверку перед сдачей

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 3/8

Пройдите короткую проверку перед сдачей

```
        var visual = new DrawingVisual();
        using (var drawing = visual.RenderOpen())
        {
            for (int i = 0; i < 4; i++)
            {
                var image = new BitmapImage();
                image.BeginInit();
                image.CacheOption = BitmapCacheOption.OnLoad;
                image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
                image.EndInit();
                image.Freeze();
                pieces[i] = image;
                drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
            }
        }
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 4/8

Пройдите короткую проверку перед сдачей

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 5/8

Пройдите короткую проверку перед сдачей

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 6/8

Пройдите короткую проверку перед сдачей

```
bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

void RenderPuzzle()
{
    for (int i = 0; i < 4; i++)
    {
        tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
        tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
        tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
    }
}
void Shuffle()
{
    do
    {
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 7/8

Пройдите короткую проверку перед сдачей

```
            Random.Shared.Shuffle(order);
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignIn(object sender, RoutedEventArgs e)
    {
        try
        {
            // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
            MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);

            var main = new MainWindow(user) { Owner = this };
            main.ShowDialog();
```

ШАГ 24 · LOGINWINDOW.XAML.CS · 8/8

Пройдите короткую проверку перед сдачей

```
            PasswordInput.Clear();
            Shuffle();
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
            Shuffle();
        }
        catch (DbException)
        {
            MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
                "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Есть проект MyApp с исходниками и снимки входа, администрирования и проверки блокировки.

ЧТО ПРОВЕРИТЬ

Проект **WPF**, работающий с **PostgreSQL**.

ЗАДАНИЕ 5 · 60 МИН

API и проверки

Создадим отдельный HTTP-сервис на ASP.NET Core Minimal APIs. Он читает ту же базу, что и настольное приложение.

ЧТО ПРОВЕРИТЬ

Сохранены проект **Api**, коллекция **Postman** и результаты тестов. Сервис возвращает поля и формат из приложения к КИМ.

УСЛОВИЕ ИЗ КИМ

Интеграция программных модулей

Разработайте API, которое при обращении возвращает данные из базы данных. Данные не должны выдаваться в исходном виде — перед отправкой клиенту они обязательно проходят обработку в соответствии с требованиями, указанными в Приложении 2.

С использованием предоставленного средства автоматизации тестирования API разработайте коллекцию запросов. Внутри коллекции должны быть реализованы тесты, подтверждающие работоспособность решения.

Приложение: Прил_5_ОЗ_КИМ_09.02.07-5-2027.pdf

КИМ · стр. 31–32; Приложение 2 Исходные файлы · ZIP

ШАГ 1 ИЗ 24

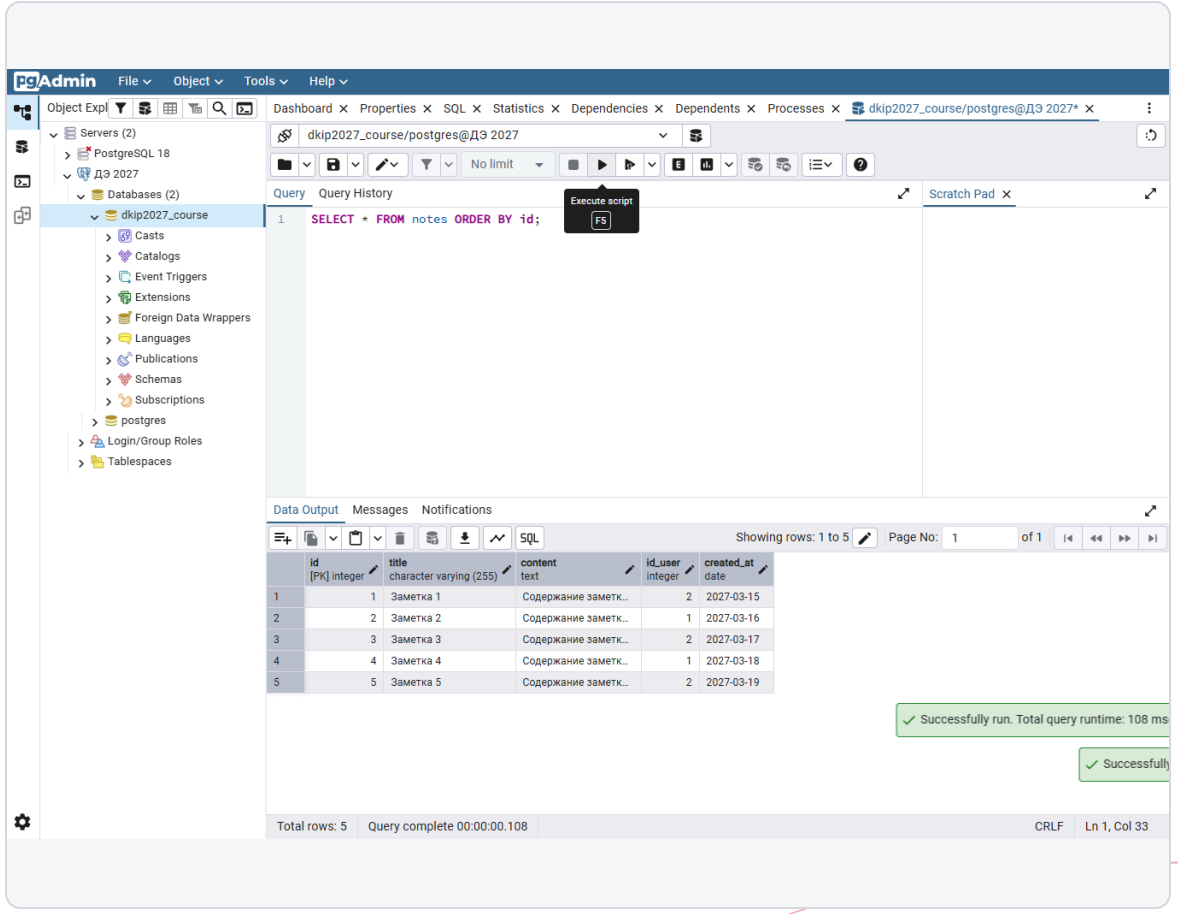
Создайте таблицу заметок

notes хранит исходные данные. Связь с users нужна, чтобы получить логин автора.

1. Напишите CREATE TABLE notes по требованиям приложения к заданию 5.
2. Проверьте id, title, content, id_user и created_at.
3. Проверьте внешний ключ id_user → users.id.

ЧТО ПРОВЕРИТЬ

Тип created_at — DATE, а title_user и formatted_date пока отсутствуют.



Пять исходных записей **notes** в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 ИЗ 24

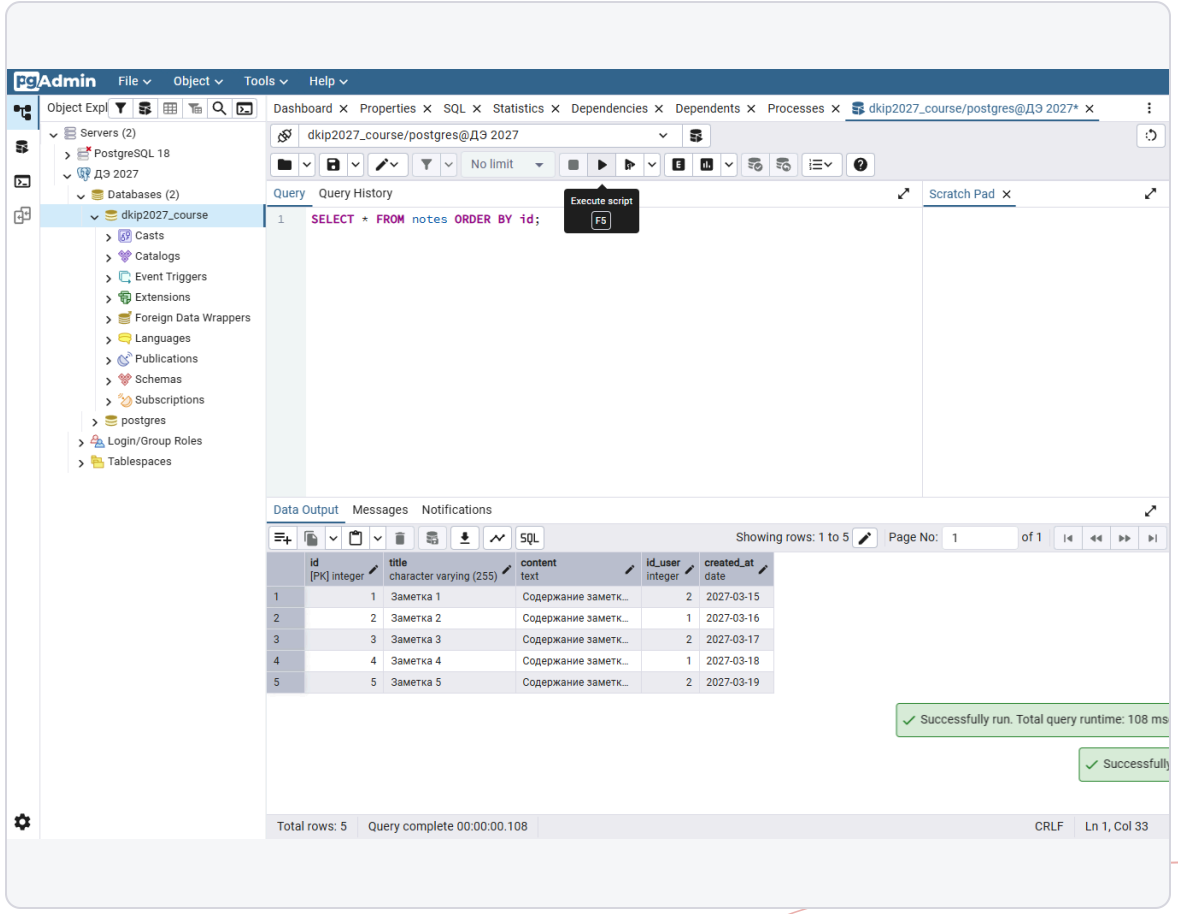
Добавьте тестовые заметки

Тестовые заметки позволяют сразу увидеть преобразование в ответе.

1. Подготовьте 3–5 заметок с разными заголовками, текстами и датами.
2. Запишите их через INSERT, указав id_user существующих пользователей.
3. В pgAdmin выполните SELECT * FROM notes ORDER BY id и сверьте все значения.

ЧТО ПРОВЕРИТЬ

В базе 3–5 тестовых заметок; внешний ключ каждой записи указывает на существующего автора.



Пять исходных записей **notes** в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 · КОД ДЛЯ ЭТОГО ШАГА

Добавьте тестовые заметки

```
SELECT * FROM notes ORDER BY id;
```

ШАГ 3 ИЗ 24

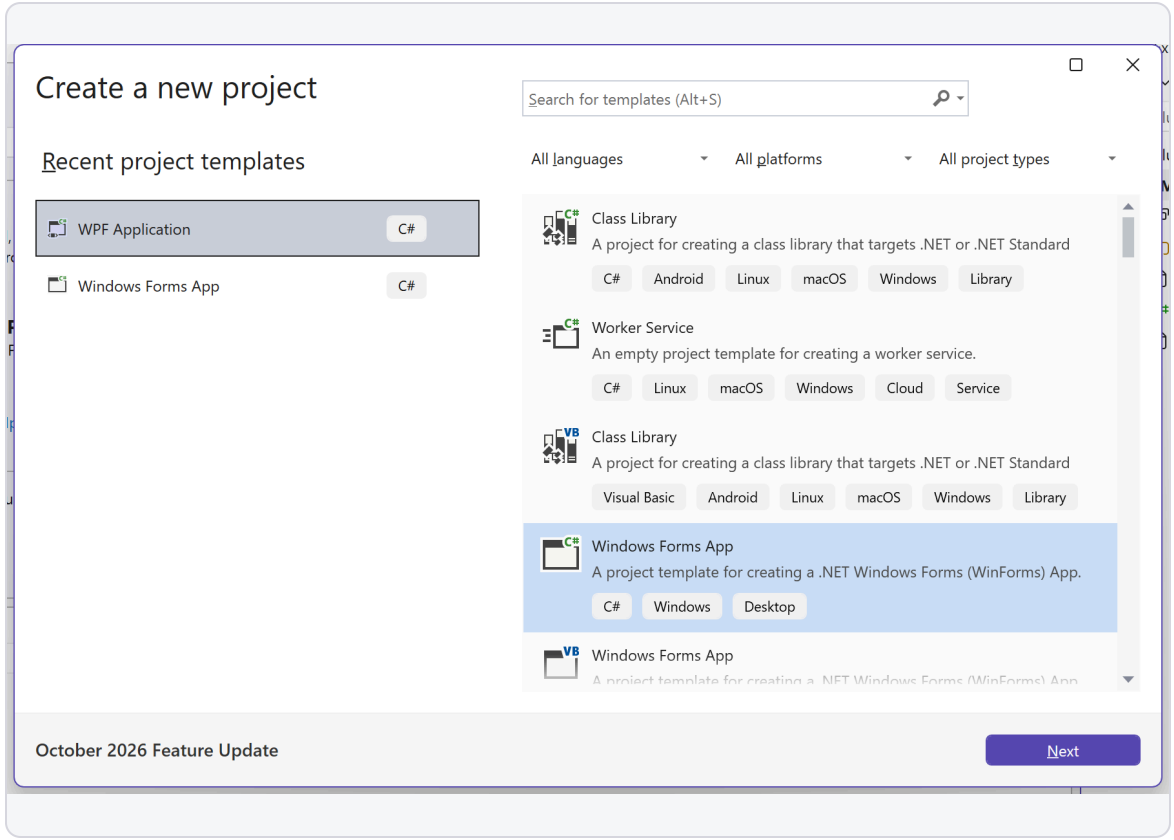
Откройте создание проекта Api

Проект объединяет исходные файлы и настройки сборки. Сначала откройте мастер создания нового проекта в Visual Studio.

1. Если видно стартовое окно Visual Studio, нажмите **Create a new project**. Если проект уже открыт, выберите «Файл» → «Создать» → «Проект» (Ctrl+Shift+N). Откроется окно поиска шаблона. Вводите название шаблона в поле поиска, а не в окно кода.

ЧТО ПРОВЕРИТЬ

Открылось окно **Create a new project** с полем поиска шаблонов.



Мастер **Create a new project** в **Visual Studio**: поле **Search for templates** и кнопка **Next**. Введите шаблон из текущего шага; список **Recent** зависит от ранее открытых проектов.

ШАГ 4 ИЗ 24

Выберите шаблон Api

Шаблон определяет тип приложения и начальные файлы. Названия современного .NET и .NET Framework похожи, но это разные платформы.

1. Найдите «Пустой проект ASP.NET Core (ASP.NET Core Empty)». Проверьте язык C#. Для Windows Forms/WPF выбирайте современный .NET, без надписи .NET Framework. Нажмите «Далее». Если шаблона нет, остановитесь и проверьте компоненты Visual Studio с преподавателем.

ЧТО ПРОВЕРИТЬ

Выбран нужный шаблон C# современного .NET; открыта настройка имени проекта.

ШАГ 5 ИЗ 24

Задайте имя и папку Api

Имя отличает проект в Обозревателе решений, расположение определяет папку исходников на диске, платформа — способ сборки.

1. В поле «Имя проекта» введите Api. В «Расположение» нажмите кнопку обзора и выберите свою рабочую папку на рабочем столе Windows. Нажмите «Далее». Выберите .NET 10 → «Создать». Дождитесь Обозревателя решений с проектом Api.

ЧТО ПРОВЕРИТЬ

В Обозревателе решений появился проект с заданным именем и выбранной платформой .NET 10.

ШАГ 6 ИЗ 24

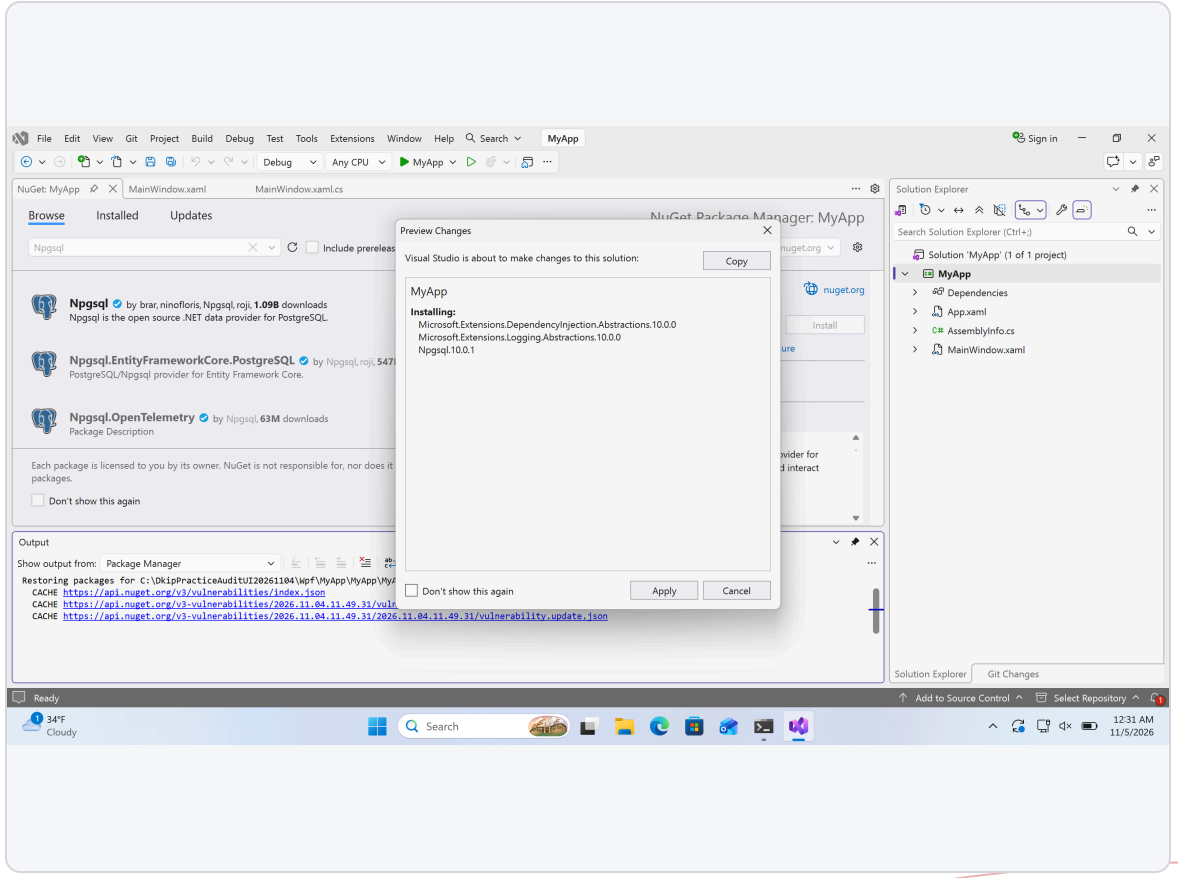
Откройте пакеты Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений нажмите правой кнопкой именно по проекту Api → «Управление пакетами NuGet». Перейдите на вкладку «Обзор» (Browse). Введите Npgsql в поиск и нажмите Enter. Дождитесь списка результатов. Выберите пакет с точным названием Npgsql.

ЧТО ПРОВЕРИТЬ

Открыт NuGet → Browse; найден пакет провайдера с точным названием.



Окно Preview Changes для Npgsql 10.0.1. Apply подтверждает установку показанного пакета и зависимостей.

ШАГ 7 ИЗ 24

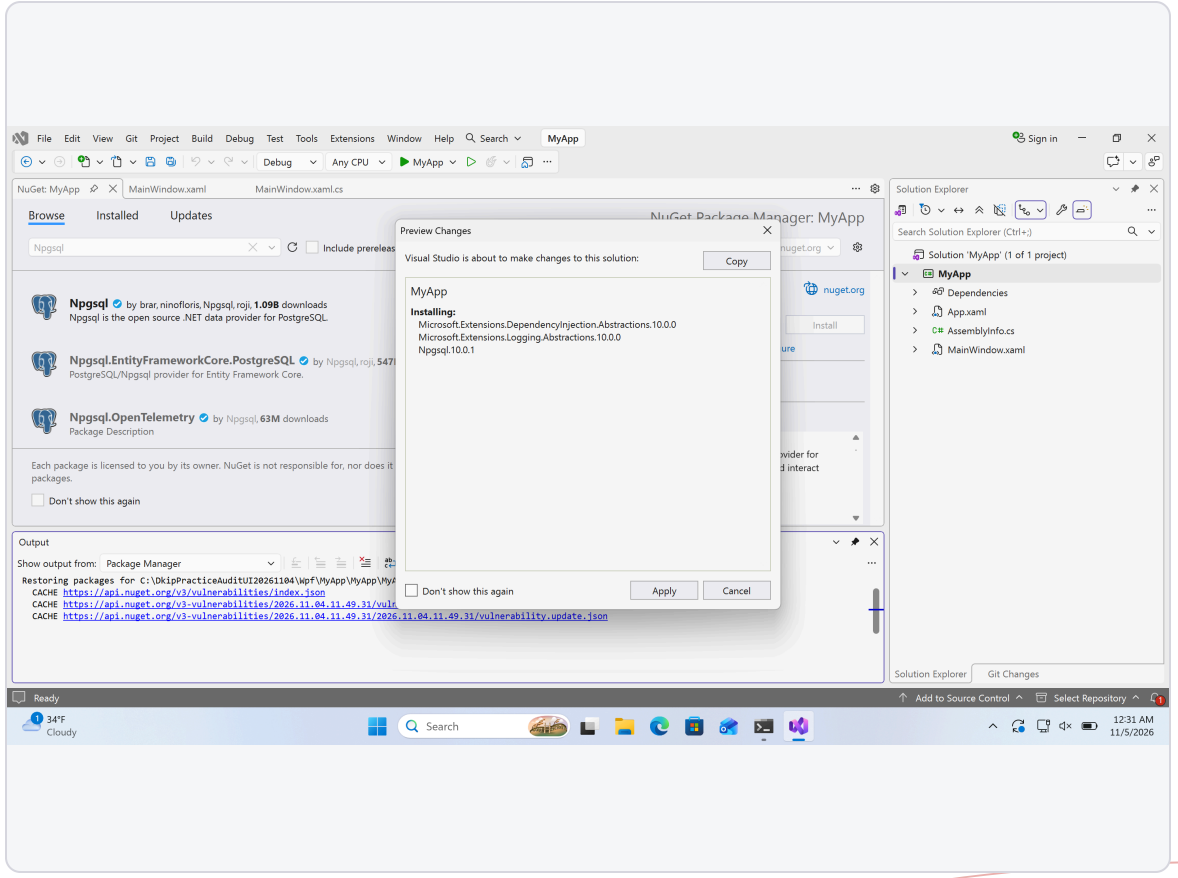
Выберите версию Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В правой панели **NuGet** найдите **Version**. Нажмите стрелку списка и выберите готовый пункт 10.0.1. Выбор из списка связывает версию с пакетом; простое изменение текста в поле может оставить кнопку **Install** недоступной.

ЧТО ПРОВЕРИТЬ

В **Version** выбрана нужная версия; кнопка **Install** доступна.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 8 ИЗ 24

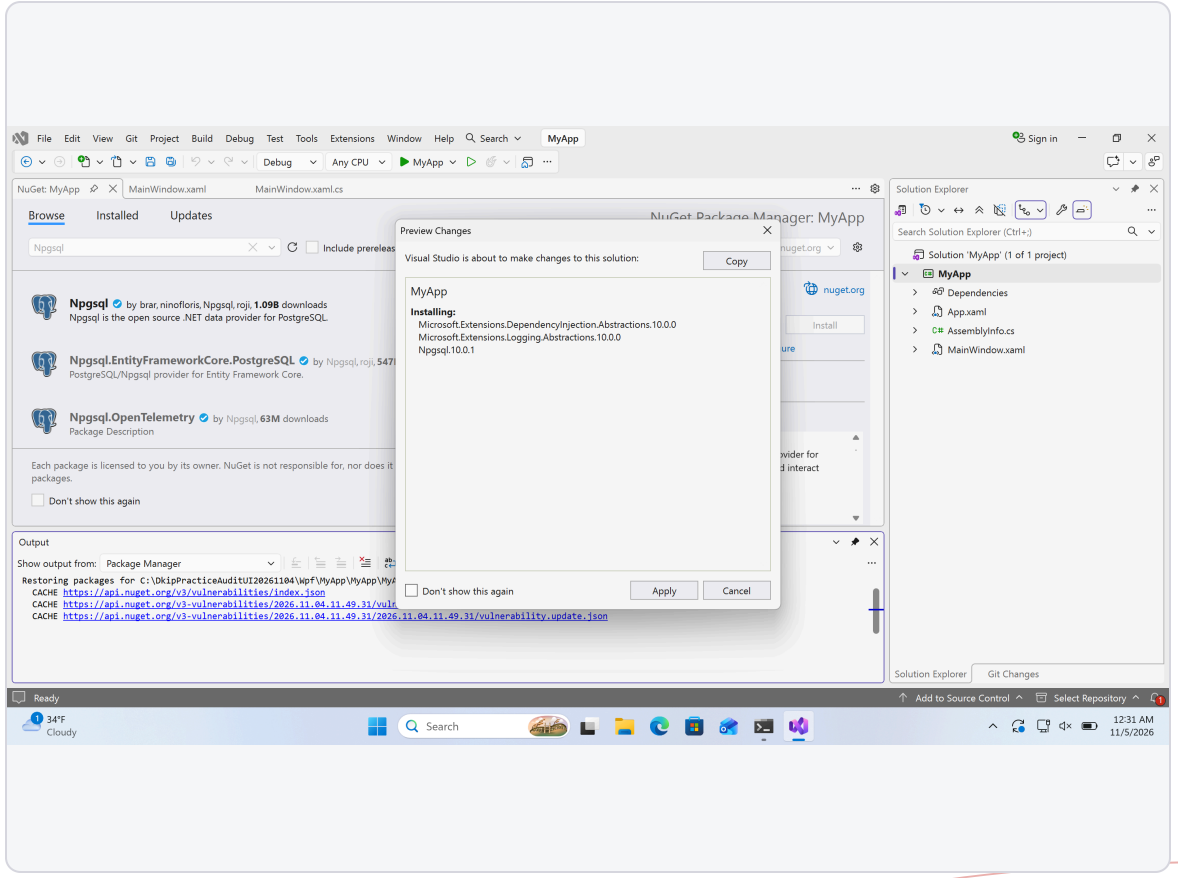
Установите провайдер Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. **NuGet** устанавливает её зависимость в конкретный проект.

1. Нажмите **Install** («Установить»). Если появилось окно **Preview Changes** («Просмотр изменений»), сверяйте название пакета и версию, затем нажмите **Apply** («Применить»). Дождитесь окончания установки. Не закрывайте **Visual Studio** во время загрузки.

ЧТО ПРОВЕРИТЬ

Установка завершилась; в карточке пакета показана установленная версия.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 9 ИЗ 24

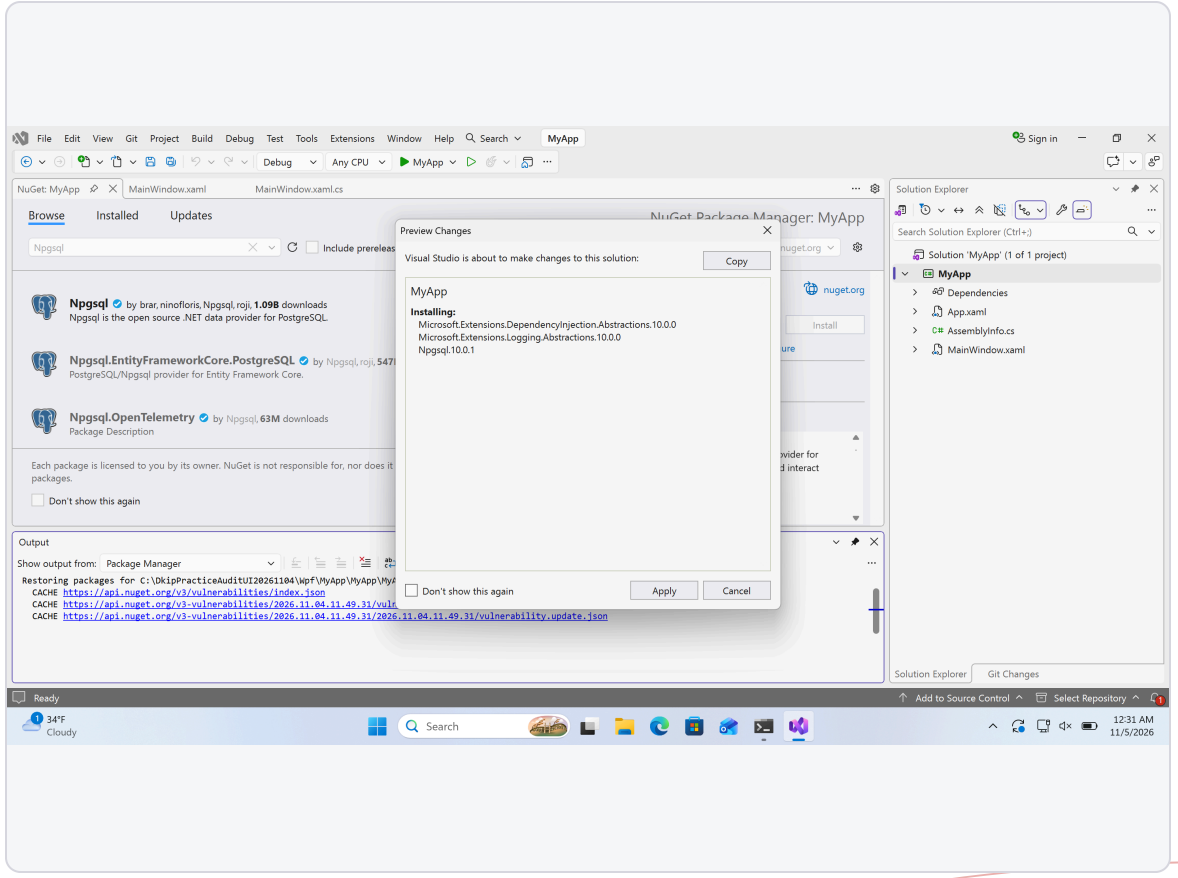
Проверьте провайдер Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. **NuGet** устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений раскройте **Dependencies** («Зависимости») → **Packages** («Пакеты»). Должен появиться **Npgsql** выбранной версии. Нажмите **Build** («Сборка») → **Build Solution** («Собрать решение»). В нижнем окне не должно быть ошибок.

ЧТО ПРОВЕРИТЬ

Провайдер появился в **Dependencies** → **Packages**; сборка решения завершилась без ошибок.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 10 ИЗ 24

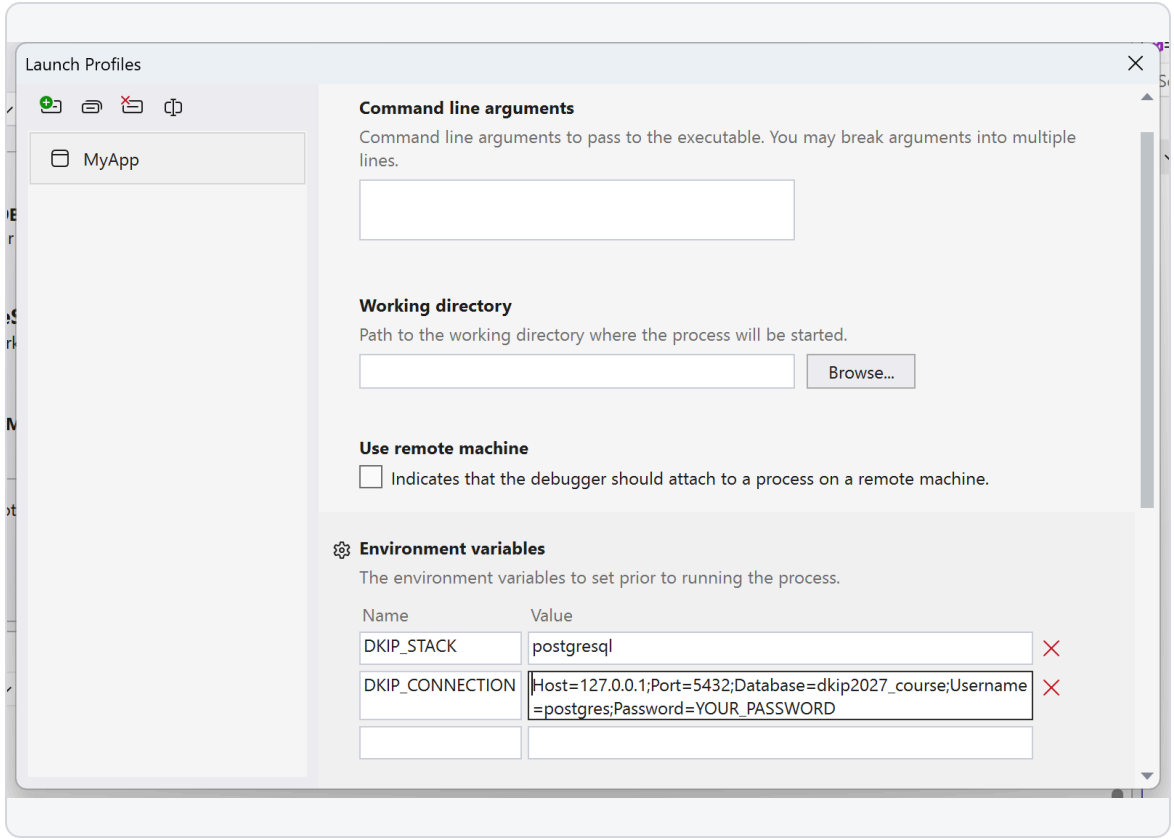
Откройте профиль подключения Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Выделите проект Api в Обозревателе решений. Выберите меню «Отладка» → «Свойства отладки Api». Другой путь: свойства проекта → «Отладка» → «Открыть профили запуска отладки». Выберите профиль с запуском самого проекта.

ЧТО ПРОВЕРИТЬ

Открыто окно профилей запуска именно нужного проекта; видна таблица **Environment variables**.



Профиль запуска Visual Studio: DKIP_STACK=postgresql и DKIP_CONNECTION в Environment variables. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 11 ИЗ 24 · 1/2

Введите параметры подключения Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Прокрутите окно профиля до **Environment variables**. В первой пустой строке нажмите поле **Name**, введите **DKIP_CONNECTION** и нажмите **Tab**. Скопируйте строку ниже в поле **Value**. Замените имя базы, порт и учётные данные значениями своей учебной установки. Подставьте пароль пользователя **PostgreSQL** вместо ВАШ_ПАРОЛЬ.

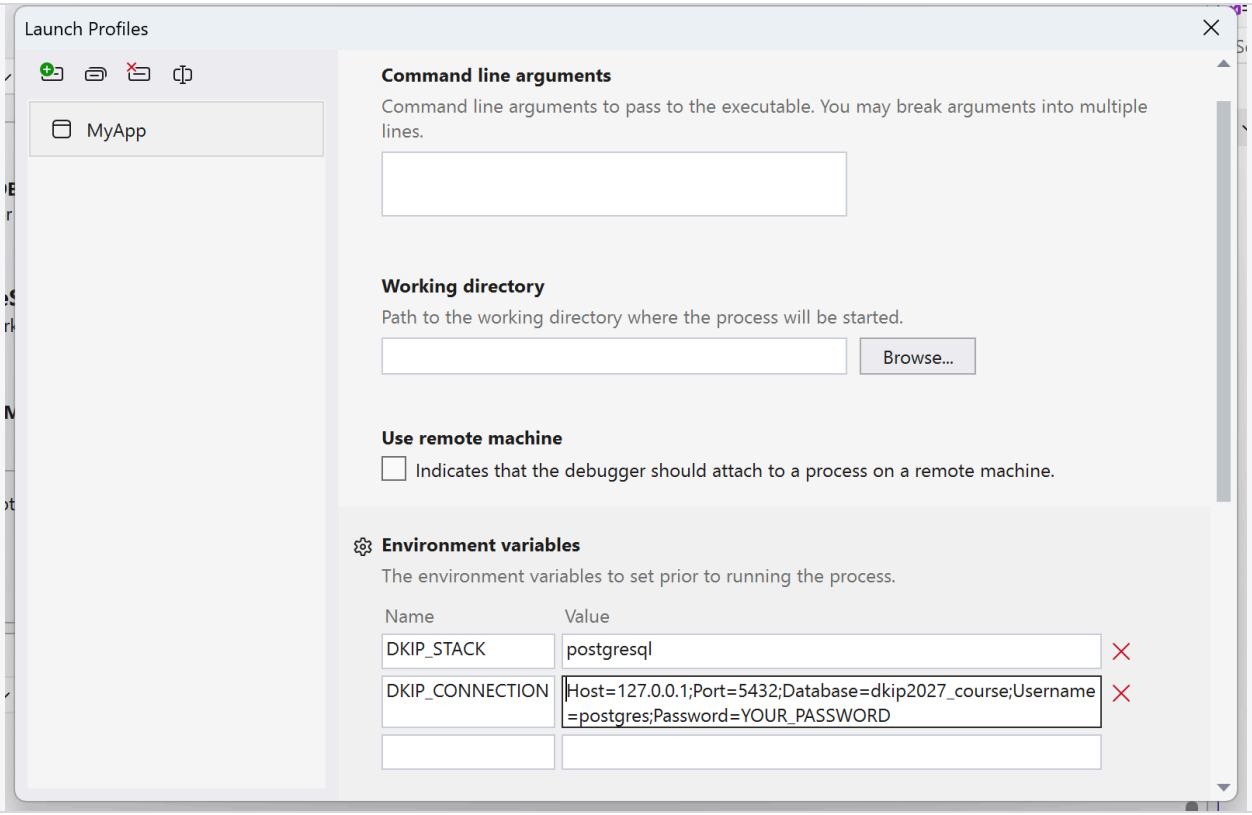
ЧТО ПРОВЕРИТЬ

В **DKIP_CONNECTION** указаны параметры своей учебной базы, без незаменённых заглушек.

ШАГ 11 ИЗ 24 · 2/2

Введите параметры подключения Api

Покажите на экране



Профиль запуска **Visual Studio**: DKIP_STACK=postgresql и DKIP_CONNECTION в **Environment variables**. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 11 · КОД ДЛЯ ЭТОГО ШАГА

Введите параметры подключения Api

```
Host=127.0.0.1;Port=5432;Database=dkip2027_course;Username=postgres;Password=ВАШ_ПАРОЛЬ
```

ШАГ 12 ИЗ 24 · 1/2

Задайте маршрут и сохраните Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. После ввода **Value** нажмите **Tab**. В появившейся пустой строке **Name** введите **DKIP_STACK**; в **Value** — **postgresql**. Нажмите **Tab** и закройте окно **Launch Profiles** крестиком. Значения сохраняются автоматически, отдельной кнопки **Save** нет. Профиль действует только для этого проекта. Если запускаете другой проект, настройте его отдельно. Файл **Properties/launchSettings.json** с настоящим паролем не отправляйте в **Git**; добавьте его в **.gitignore**.

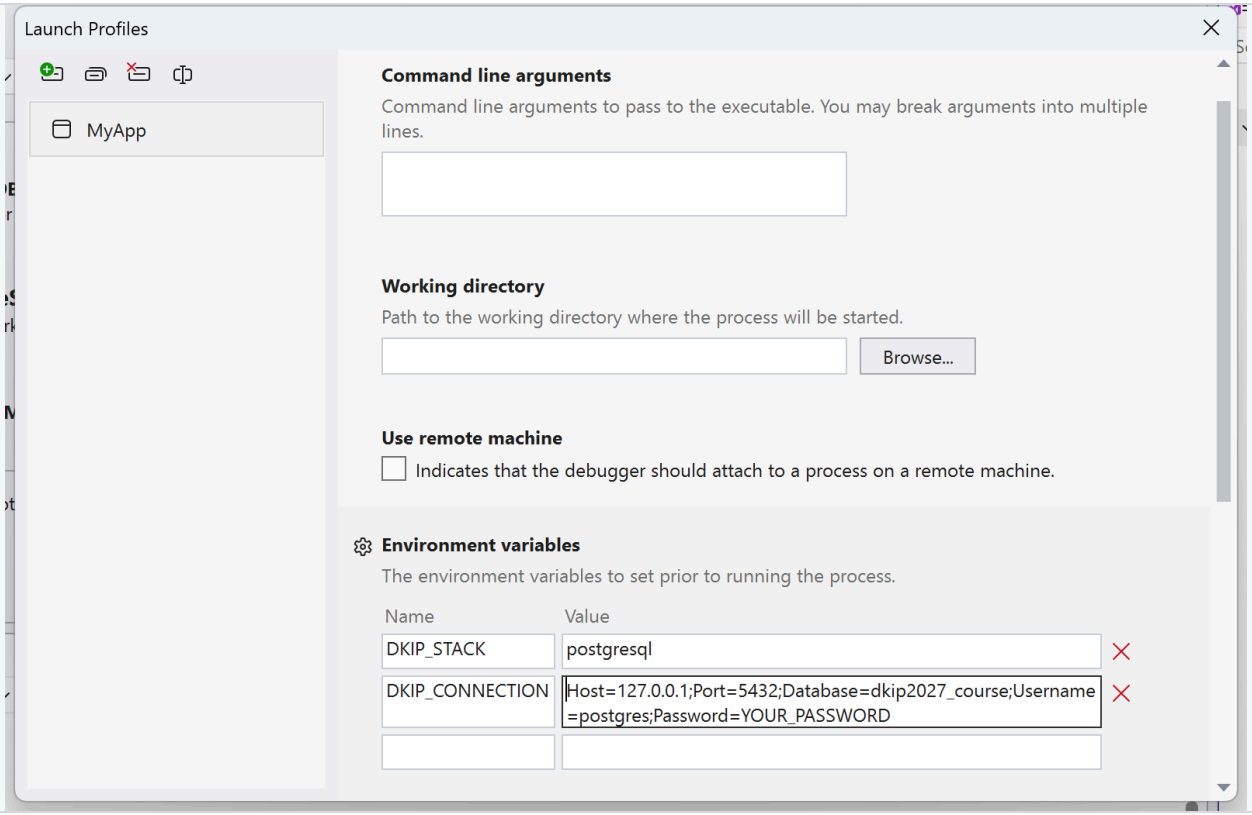
ЧТО ПРОВЕРИТЬ

В профиле сохранён **DKIP_STACK** выбранной СУБД; файл с настоящим паролем исключён из **Git**.

ШАГ 12 ИЗ 24 · 2/2

Задайте маршрут и сохраните Api

Покажите на экране



Профиль запуска **Visual Studio**: DKIP_STACK=postgresql и DKIP_CONNECTION в **Environment variables**. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 13 ИЗ 24

Настройте HTTP-адрес Api

HTTP-запрос идёт в отдельный работающий процесс API. Приложение с окнами, сервер БД и API — три разных процесса, которые могут останавливаться независимо.

1. Откройте «Отладка» → «Свойства отладки Api». Выберите профиль проекта **http**, а не **IIS Express**. В поле «URL-адрес приложения» (App URL) введите **http://127.0.0.1:5272**. Для своего локального учебного запуска используйте этот HTTP-адрес; параметры чужих приложений не меняйте.

ЧТО ПРОВЕРИТЬ

Выбран профиль проекта **http**. **App URL** содержит адрес текущего учебного маршрута.

ШАГ 14 ИЗ 24

Запустите первый HTTP-проект

HTTP-запрос идёт в отдельный работающий процесс API. Приложение с окнами, сервер БД и API — три разных процесса, которые могут останавливаться независимо.

1. Нажмите **Ctrl+F5**. В открывшемся браузере должен появиться **Hello World!** из **app.MapGet("/", ...)**. Это подтверждает запуск HTTP-приложения, но ещё не подключение к БД. Вернитесь в **Visual Studio**. Нажмите «Отладка» → «Остановить отладку»; если запускали без отладки, закройте окно работающего приложения.

ЧТО ПРОВЕРИТЬ

Браузер показывает **Hello World!** от своего API; после проверки процесс остановлен.

ШАГ 15 ИЗ 24

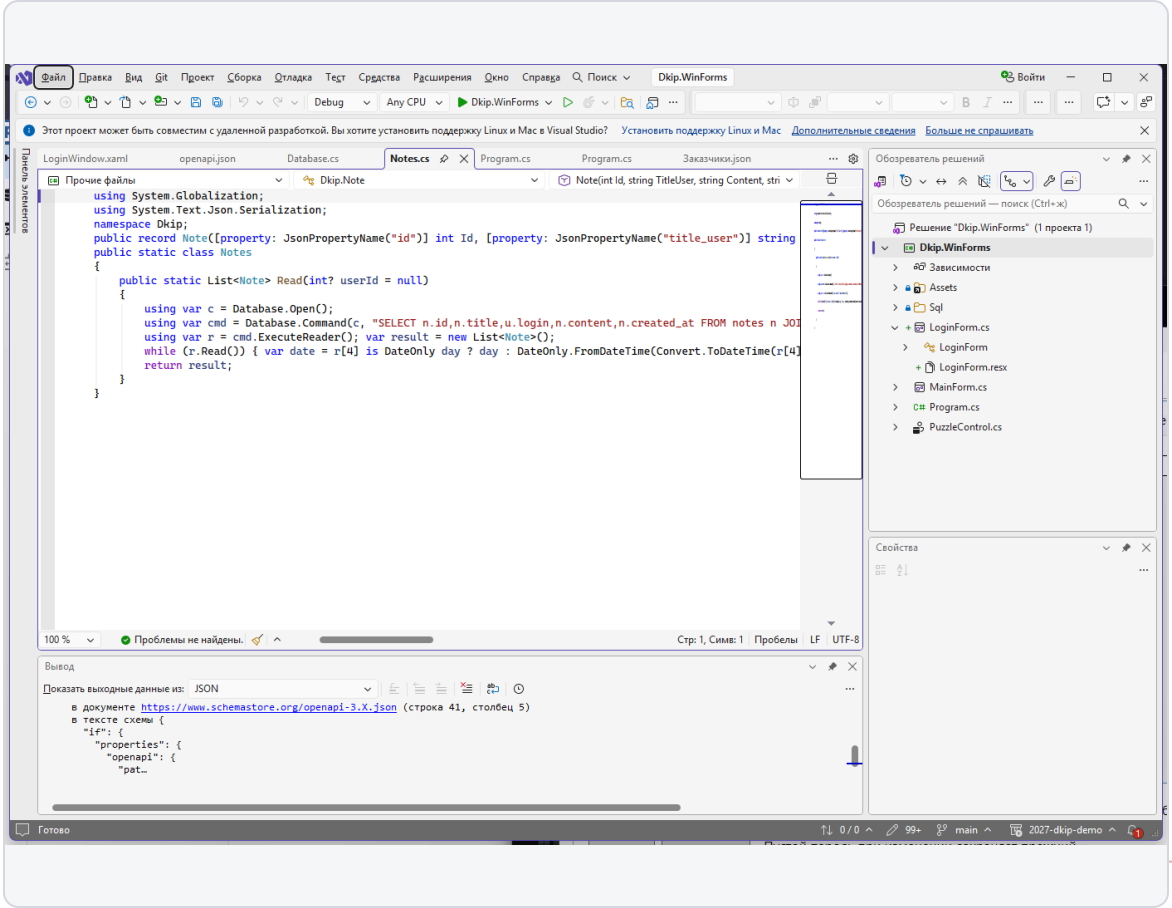
Соберите исходные данные через JOIN

API нужен логин, но он находится в users.

1. В свой проект Api добавьте классы Database.cs и Notes.cs. Database.cs возьмите из заготовки своего стека, Notes.cs — из примера. В обоих файлах задайте namespace Api.
2. Напишите SELECT n.id,n.title,u.login,n.content,n.created_at.
3. Добавьте JOIN users u ON u.id=n.id_user и проверьте результат запроса.

ЧТО ПРОВЕРИТЬ

У каждой строки доступны и заголовок, и логин автора.



Код JOIN и преобразования полей ответа в Notes.cs. Маршрут: WPF / PostgreSQL.

ШАГ 15 · NOTES.CS · 1/2

Соберите исходные данные через JOIN

```
using System.Globalization;
using System.Text.Json.Serialization;
namespace Dkip;

public record Note([property: JsonPropertyName("id")] int Id, [property: JsonPropertyName("title_user")] string TitleUser,
[property: JsonPropertyName("content")] string Content, [property: JsonPropertyName("formatted_date")] string FormattedDate);
public static class Notes
{
    public static List<Note> Read(int? userId = null)
    {
        using var c = Database.Open();
        using var cmd = Database.Command(c, "SELECT n.id,n.title,u.login,n.content,n.created_at FROM notes n JOIN users u ON
u.id=n.id_user" + (userId.HasValue ? " WHERE n.id_user=@p0" : "") + " ORDER BY n.id", userId.HasValue ? new object?[] {
userId.Value } : []);
        using var r = cmd.ExecuteReader();
```

ШАГ 15 · NOTES.CS · 2/2

Соберите исходные данные через JOIN

```
        var result = new List<Note>();
        while (r.Read())
        {
            var date = r[4] is DateOnly day ? day : DateOnly.FromDateTime(Convert.ToDateTime(r[4], CultureInfo.InvariantCulture));
            result.Add(new Note(Convert.ToInt32(r[0]), r.GetString(1) + " - " + r.GetString(2), r.GetString(3),
date.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)));
        }
        return result;
    }
}
```

ШАГ 16 ИЗ 24

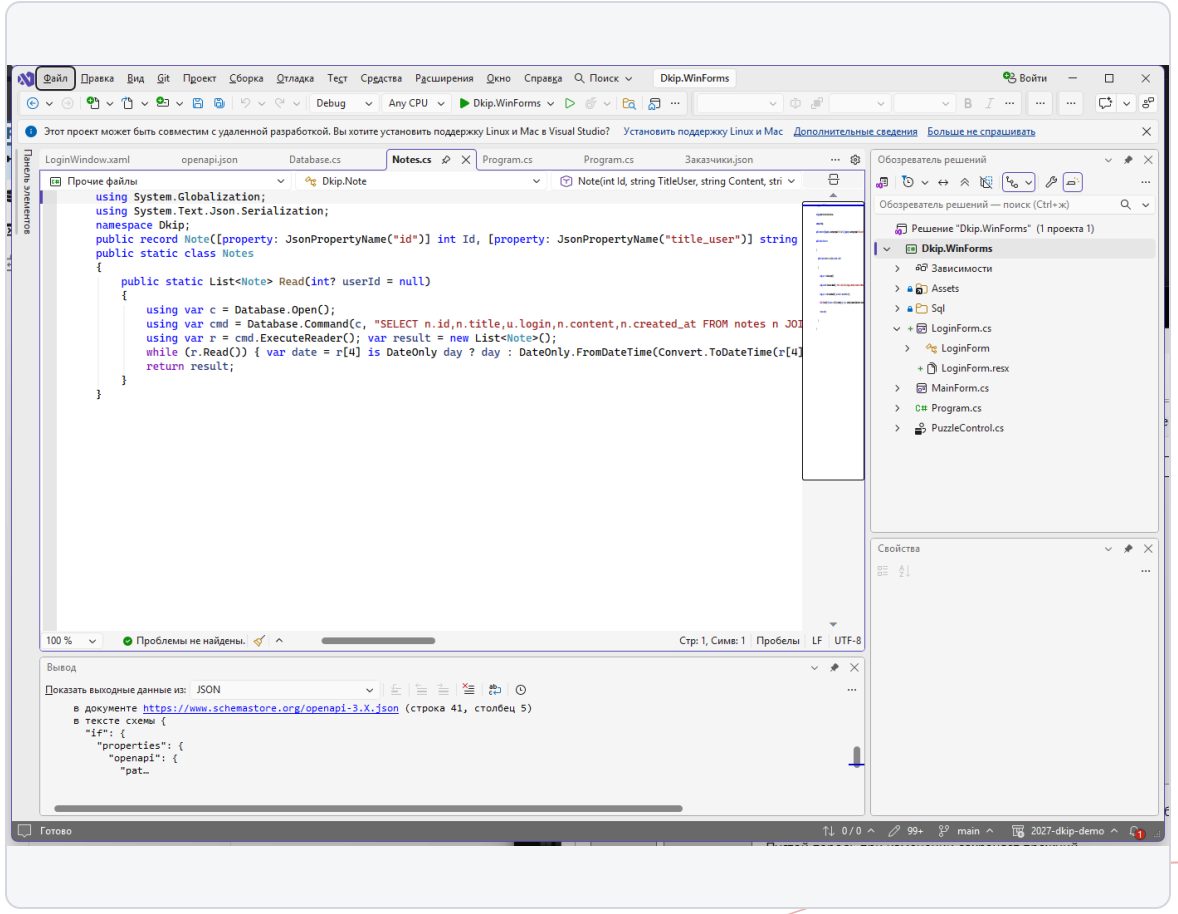
Преобразуйте поля ответа

Контракт определяет ровно четыре поля JSON.

1. Соедините title + " - " + login.
2. Передайте content без изменений.
3. Преобразуйте дату через dd.MM.yyyy и сохраните исходный id.

ЧТО ПРОВЕРИТЬ

Ответ содержит id, title_user, content, formatted_date.



Код JOIN и преобразования полей ответа в Notes.cs. Маршрут: WPF / PostgreSQL.

ШАГ 16 · NOTES.CS · 1/2

Преобразуйте поля ответа

```
using System.Globalization;
using System.Text.Json.Serialization;
namespace Dkip;

public record Note([property: JsonPropertyName("id")] int Id, [property: JsonPropertyName("title_user")] string TitleUser,
[property: JsonPropertyName("content")] string Content, [property: JsonPropertyName("formatted_date")] string FormattedDate);
public static class Notes
{
    public static List<Note> Read(int? userId = null)
    {
        using var c = Database.Open();
        using var cmd = Database.Command(c, "SELECT n.id,n.title,u.login,n.content,n.created_at FROM notes n JOIN users u ON
u.id=n.id_user" + (userId.HasValue ? " WHERE n.id_user=@p0" : "") + " ORDER BY n.id", userId.HasValue ? new object?[] {
userId.Value } : []);
        using var r = cmd.ExecuteReader();
```



ШАГ 16 · NOTES.CS · 2/2

Преобразуйте поля ответа

```
        var result = new List<Note>();
        while (r.Read())
        {
            var date = r[4] is DateOnly day ? day : DateOnly.FromDateTime(Convert.ToDateTime(r[4], CultureInfo.InvariantCulture));
            result.Add(new Note(Convert.ToInt32(r[0]), r.GetString(1) + " - " + r.GetString(2), r.GetString(3),
date.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)));
        }
        return result;
    }
}
```

ШАГ 17 ИЗ 24

Добавьте GET /notes через MapGet

Маршрут определяет, какой код выполнится при запросе адреса.

1. В своём проекте ASP.NET Core Minimal API откройте Program.cs.
2. В Program.cs зарегистрируйте MapGet("/notes", ReadNotes); при необходимости добавьте /api/notes с тем же обработчиком.
3. В ReadNotes вызовите Notes.Read и передайте результат в Results.Ok. Откройте код примера, если нужно сверить обработчик.

ЧТО ПРОВЕРИТЬ

Реализованный путь GET читает реальные строки БД; JSON формируется из результата запроса.

ШАГ 17 · МАРШРУТ В PROGRAM.CS

Добавьте GET /notes через MapGet

```
app.MapGet("/notes", ReadNotes);
```

ШАГ 17 · PROGRAM.CS · 1/3

Добавьте GET /notes через MapGet

```
using Dkip;
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.UseStaticFiles();
IResult ReadNotes(HttpRequest request)
{
    int? userId = null;
    if (request.Query.Keys.Any(k => k != "user_id"))
        return Results.BadRequest(new
        {
            error = "Неизвестный параметр. Допустим user_id."
        });
    if (request.Query.ContainsKey("user_id"))
    {
        if (request.Query["user_id"].Count != 1 || !int.TryParse(request.Query["user_id"], out var parsed) || parsed <= 0)
```



ШАГ 17 · PROGRAM.CS · 2/3

Добавьте GET /notes через MapGet

```
            return Results.BadRequest(new
            {
                error = "user_id должен быть положительным целым числом"
            });
        userId = parsed;
    }
    try
    {
        return Results.Ok(Notes.Read(userId));
    }
    catch (Exception ex)
    {
        // Подробности остаются в журнале сервера. Клиент получает только сообщение.
        app.Logger.LogError(ex, "Ошибка обработки API");
        return Results.Json(
            new { error = "Ошибка подключения к базе данных или обработки запроса" },
```

ШАГ 17 · PROGRAM.CS · 3/3

Добавьте GET /notes через MapGet

```
            statusCode: StatusCodes.Status500InternalServerError);
    }
}
app.MapGet("/notes", ReadNotes);
app.MapGet("/api/notes", ReadNotes);
app.MapGet("/", () => Results.Redirect("/notes"));
app.Run();
```

ШАГ 18 ИЗ 24

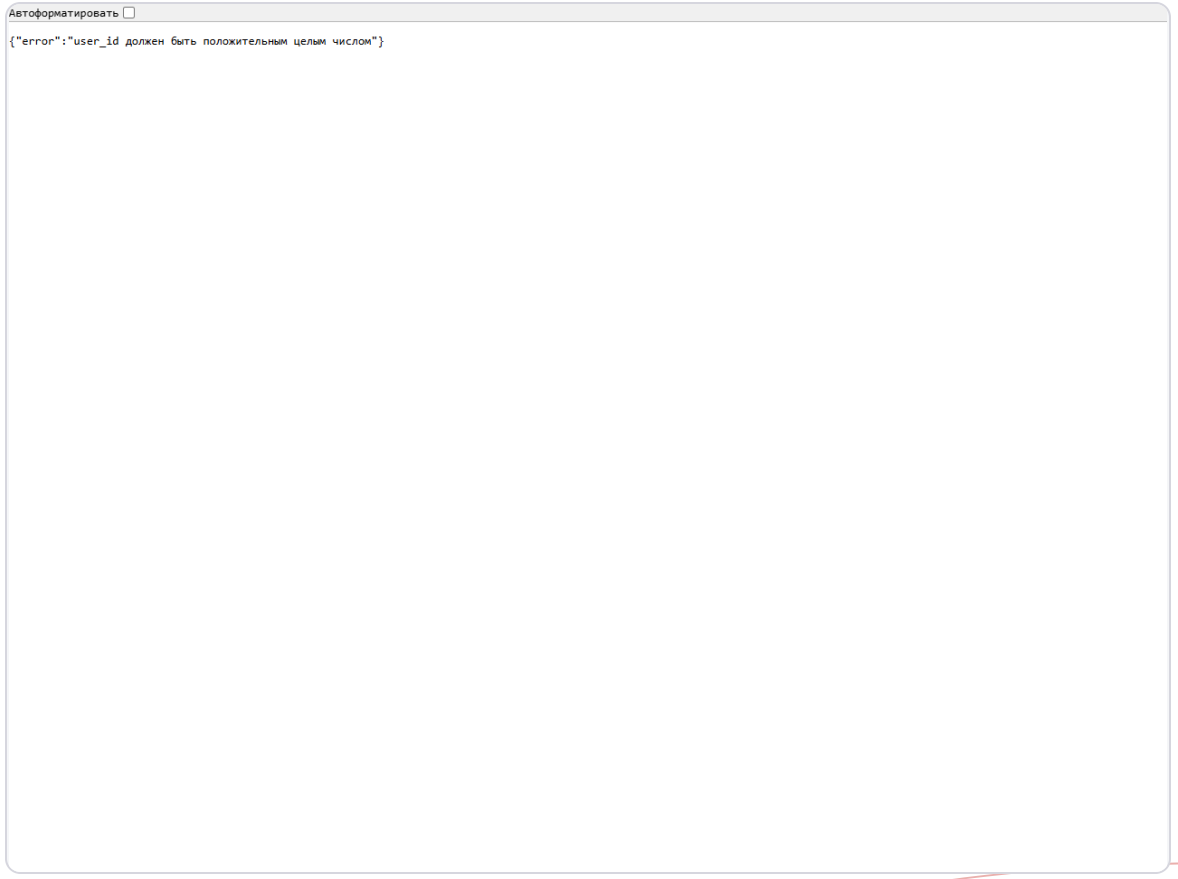
Обработайте неверные параметры

В учебном API необязательный user_id демонстрирует фильтр и ошибку 400.

1. В ReadNotes проверьте целое положительное число.
2. Для abc, нуля, повторного или неизвестного параметра верните 400 и JSON error.
3. Если параметра нет, прочитайте весь список.

ЧТО ПРОВЕРИТЬ

/notes?user_id=abc возвращает 400; без параметров — 200.



Настоящий JSON-ответ на user_id=abc. Статус 400 отдельно подтверждён HTTP-тестами. Маршрут: WPF / PostgreSQL.

ШАГ 19 ИЗ 24

Обработайте пустой результат и сбой

Пустой список является нормальным ответом. Ошибка базы должна иметь отдельный статус.

1. Если строк нет, верните список нулевой длины.
2. Обработчик исключения записывает подробности в консоль сервера.
3. Клиент получает 500 и JSON с error без реквизитов подключения.

ЧТО ПРОВЕРИТЬ

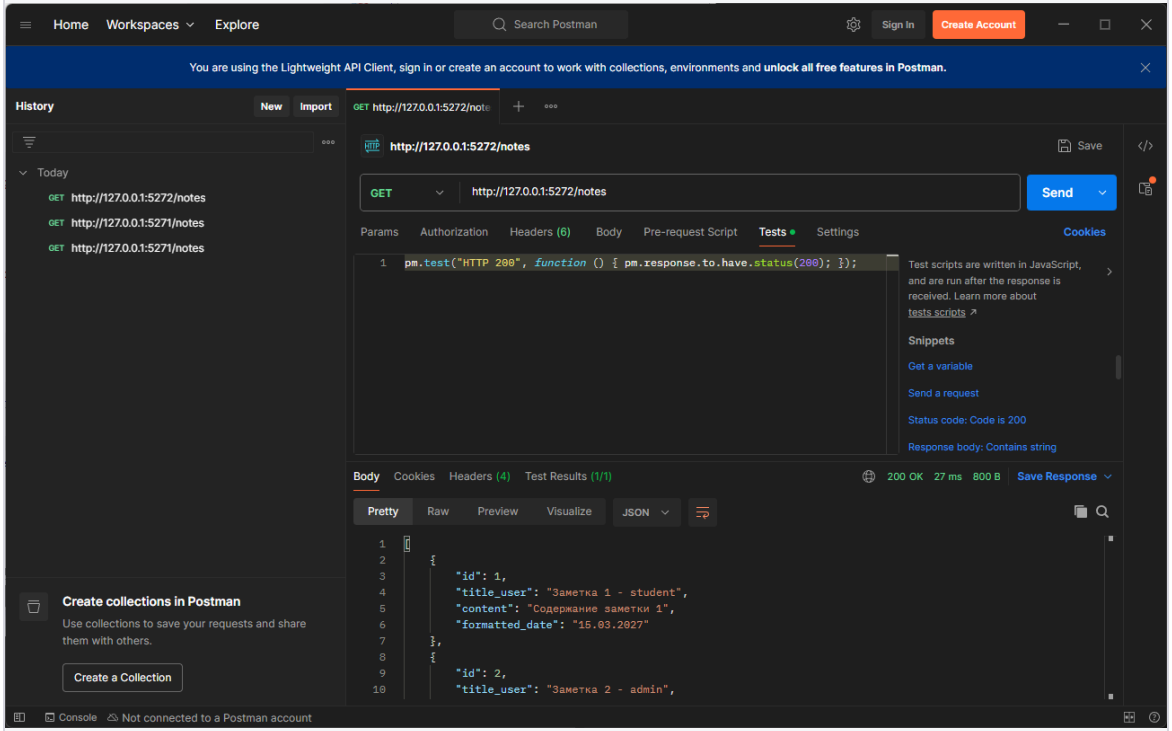
Пустой список: 200 и []; сбой: 500 и объект error.

ШАГ 20 ИЗ 24 · 1/2

Запустите сервер API

API работает отдельным приложением. Пока оно остановлено, браузер и Postman не получат данные.

1. Откройте свой Api.csproj через Файл → Открыть → Проект или решение.
2. Выделите Api → Отладка → Свойства отладки Api.
3. В профиле http задайте DKIP_CONNECTION и DKIP_STACK=postgresql в Переменных среды.



Postman Lightweight: настоящий ответ локального API. Маршрут: WPF / PostgreSQL.

ШАГ 20 ИЗ 24 · 2/2

Запустите сервер API

1. В App URL укажите http://127.0.0.1:5272; сохраните профиль.
2. Выберите этот профиль рядом с зелёной кнопкой запуска → Отладка → Запуск без отладки (Ctrl+F5).
3. В браузере откройте http://127.0.0.1:5272/notes. Оставьте приложение работающим.

ЧТО ПРОВЕРИТЬ

Браузер показывает JSON; последующие запросы выполняются к этому адресу.

ШАГ 21 ИЗ 24 · 1/2

Создайте коллекцию Postman

Коллекция объединяет HTTP-запросы и исполняемые проверки. Она должна подтверждать поведение вашего API при успешной работе, неверном запросе и сбое.

1. В Postman создайте новую коллекцию для своего API.
2. Задайте переменную base_url=http://127.0.0.1:5272; добавьте GET для списка заметок, пустого результата и неверного параметра.
3. Добавьте отдельный запрос к тестовому экземпляру с ошибочным подключением; задайте его адрес в error_url.

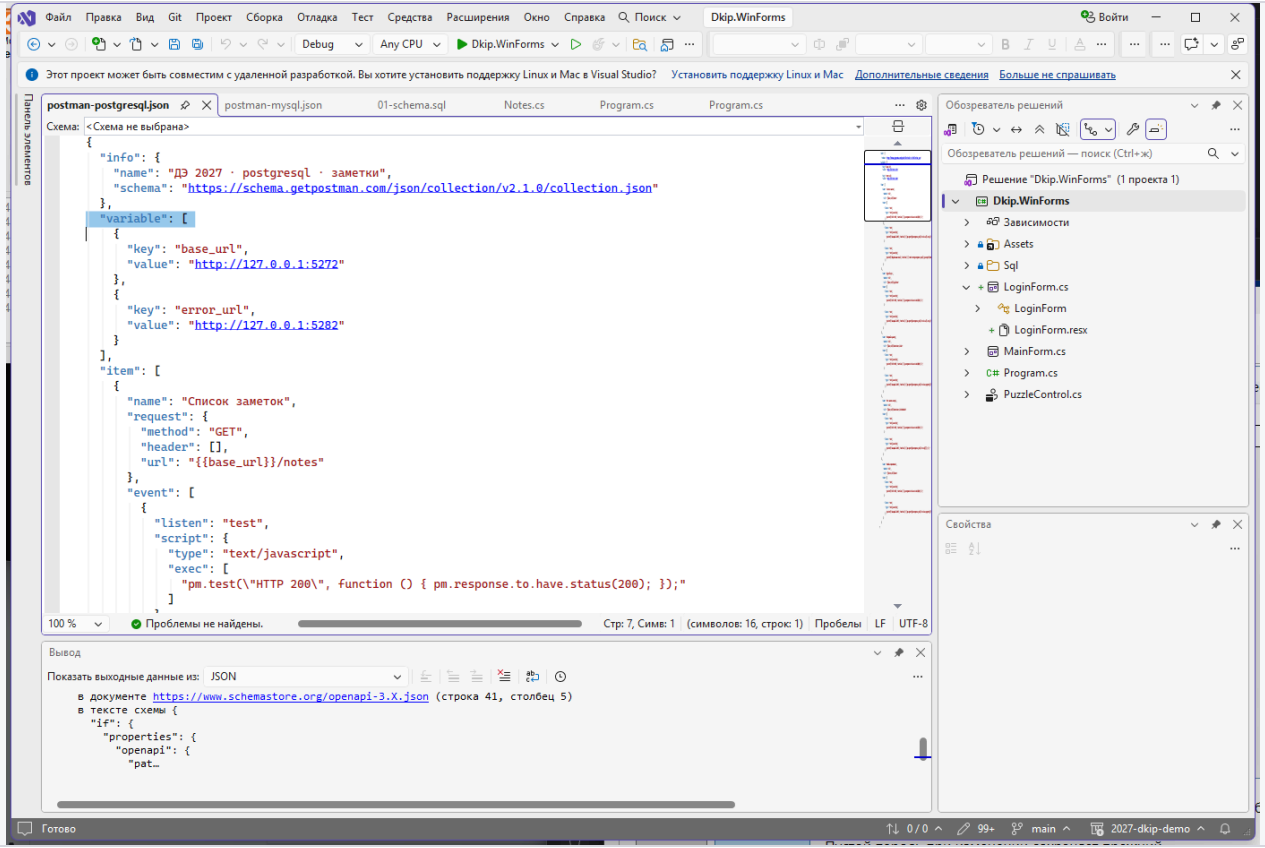
ЧТО ПРОВЕРИТЬ

В собственной коллекции сохранены запросы для успешного, пустого и ошибочных ответов.

ШАГ 21 ИЗ 24 · 2/2

Создайте коллекцию Postman

Покажите на экране



Файл коллекции **Postman**: адреса **base_url** и **error_url** перед импортом. Маршрут: **WPF / PostgreSQL**.

ШАГ 22 ИЗ 24

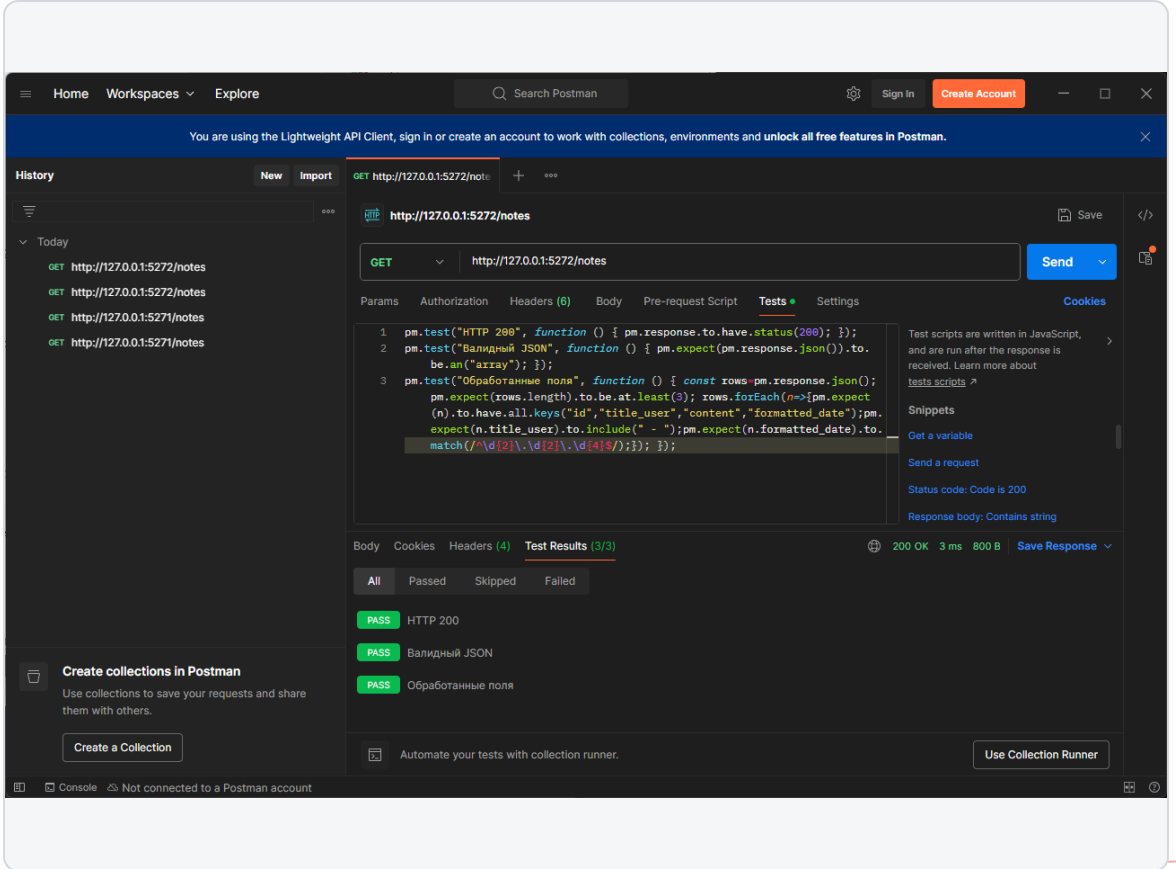
Выполните проверки 200 и JSON

Тест выполняется после получения ответа.

1. В Scripts → Post-response добавьте pm.test для статуса 200 и разбора pm.response.json().
2. Добавьте проверки четырёх полей, разделителя title_user и формата formatted_date.
3. Отправьте запрос списка заметок и проверьте результаты во вкладке Test Results.

ЧТО ПРОВЕРИТЬ

Проверки статуса и формата пройдены; число объектов соответствует вашим тестовым записям.



Postman: выполненные проверки **HTTP 200, JSON** и преобразованных полей. Маршрут: **WPF / PostgreSQL.**

ШАГ 23 ИЗ 24 · 1/2

Воспроизведите 500 на отдельном сервере

Проверяем отдельную тестовую копию API. Основную базу и рабочее подключение сохраняем.

1. Остановите тестовую копию Api, если она запущена.
2. В её свойствах отладки создайте отдельный профиль **DkipError** с запуском проекта.
3. В **DKIP_CONNECTION** этого профиля задайте неверный порт БД 1 и короткий **timeout**; имя и данные рабочей базы не меняйте.
4. В **App URL** задайте **http://127.0.0.1:5282**; сохраните профиль.

ШАГ 23 ИЗ 24 · 2/2

Воспроизведите 500 на отдельном сервере

1. Выберите DkipError рядом с кнопкой запуска и нажмите Ctrl+F5.
2. В Postman задайте error_url равным адресу тестовой копии → откройте запрос Ошибка подключения → Send.
3. Проверьте статус 500 и JSON error. Завершите только тестовую копию приложения.

ЧТО ПРОВЕРИТЬ

500 получен при реальном отказе подключения; основной профиль не изменён.

ШАГ 24 ИЗ 24

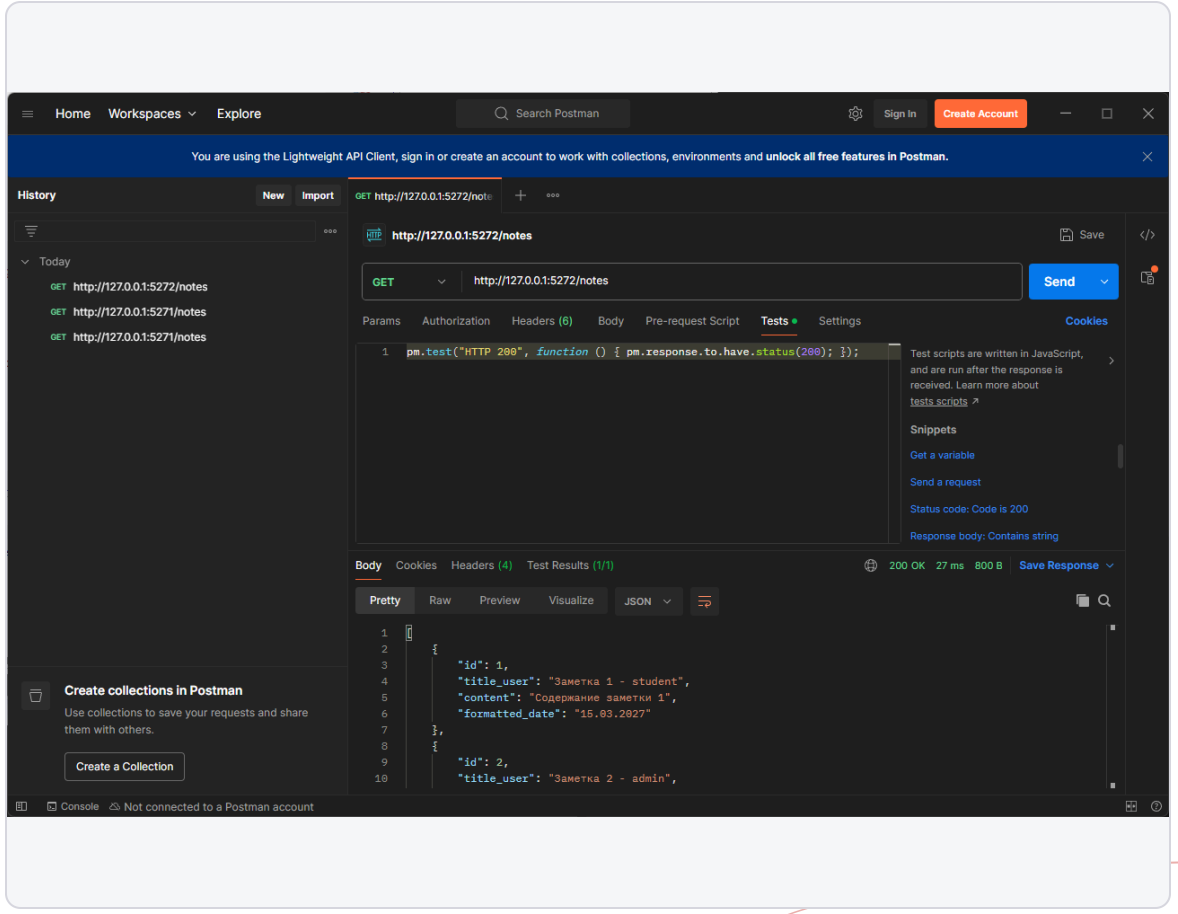
Сохраните результаты тестирования

Коллекция должна содержать и запросы, и исполняемые тесты.

1. В Postman экспортируйте коллекцию v2.1.
2. Сохраните результаты запуска 200, 400, пустого ответа и 500.
3. Переходите к документации с уже работающим API.

ЧТО ПРОВЕРИТЬ

Другой человек может импортировать коллекцию и повторить проверки.



Postman Lightweight: настоящий ответ локального API. Маршрут: WPF / PostgreSQL.

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Сохранены проект Api, коллекция Postman и результаты тестов. Сервис возвращает поля и формат из приложения к КИМ.

ЧТО ПРОВЕРИТЬ

HTTP-сервис и коллекция Postman с тестами.

ЗАДАНИЕ 6 · 20 МИН

Документация API

Документацию пишем по работающему сервису. Точный запрос и фактический ответ берём из **Postman**.

ЧТО ПРОВЕРИТЬ

Готов документ Прил_6_ОЗ_КИМ_**09.02.07-5-2027.docx**, заполненный по шаблону КИМ.

УСЛОВИЕ ИЗ КИМ

Разработка проектной документации

Разработать документацию на реализованный API в точном соответствии с шаблоном из Приложения 3.

В документации описать все доступные методы, параметры к ним и привести примеры запросов/ответов.

Приложение: Прил_6_О3_КИМ_09.02.07-5-2027.docx

КИМ · стр. 32; Приложение 3 Исходные файлы · ZIP

ШАГ 1 ИЗ 7

Откройте исходный шаблон Word

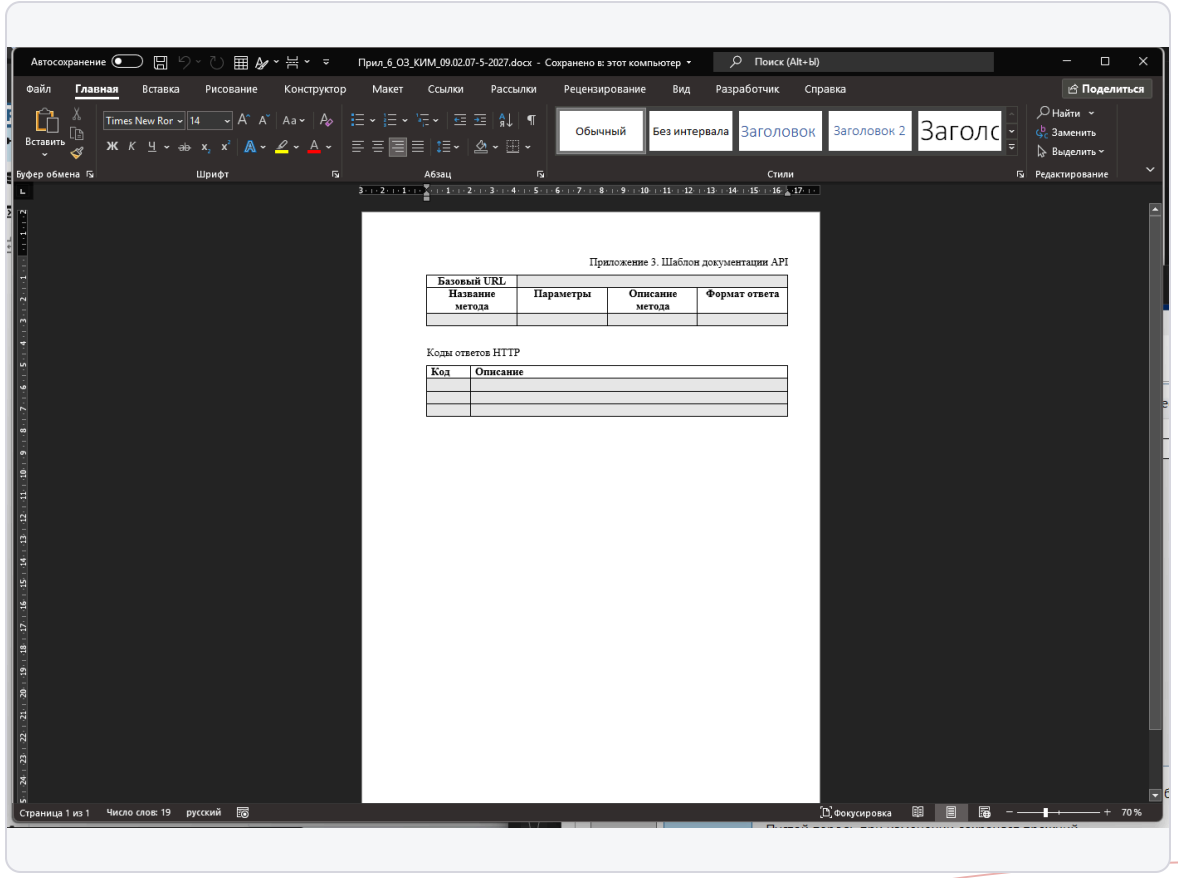
Задание 6 описывает ASP.NET Core Minimal API, который вы уже написали в задании 5.

1. Скачайте Приложение 3 к заданию 6. Сделайте копию DOCX и откройте её в Word.
2. Найдите поля базового URL, методов, параметров, описания и формата ответа. Ниже найдите таблицу кодов HTTP.

ЧТО ПРОВЕРИТЬ

Открыта копия исходного шаблона с таблицами для методов и кодов HTTP.

КИМ, стр. 32; Приложение 3



Исходный шаблон Приложения 3 открыт в Microsoft Word. Маршрут: WPF / PostgreSQL.

ШАГ 2 ИЗ 7

Найдите методы в Program.cs

Строка **MapGet** связывает путь и обработчик. По ней можно определить, что написать в колонке «Название метода».

1. Откройте **Program.cs** в своём проекте Api. Найдите строки **app.MapGet** для заметок.
2. Для каждой строки выпишите **GET** и путь. Если один обработчик привязан к **/notes** и **/api/notes**, укажите оба пути.

ЧТО ПРОВЕРИТЬ

Список методов в черновике совпадает с маршрутами **MapGet** в вашем проекте.

ШАГ 2 · PROGRAM.CS · 1/3

Найдите методы в Program.cs

```
using Dkip;
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.UseStaticFiles();
IResult ReadNotes(HttpRequest request)
{
    int? userId = null;
    if (request.Query.Keys.Any(k => k != "user_id"))
        return Results.BadRequest(new
        {
            error = "Неизвестный параметр. Допустим user_id."
        });
    if (request.Query.ContainsKey("user_id"))
    {
        if (request.Query["user_id"].Count != 1 || !int.TryParse(request.Query["user_id"], out var parsed) || parsed <= 0)
```

ШАГ 2 · PROGRAM.CS · 2/3

Найдите методы в Program.cs

```
            return Results.BadRequest(new
            {
                error = "user_id должен быть положительным целым числом"
            });
        userId = parsed;
    }
    try
    {
        return Results.Ok(Notes.Read(userId));
    }
    catch (Exception ex)
    {
        // Подробности остаются в журнале сервера. Клиент получает только сообщение.
        app.Logger.LogError(ex, "Ошибка обработки API");
        return Results.Json(
            new { error = "Ошибка подключения к базе данных или обработки запроса" },
```

ШАГ 2 · PROGRAM.CS · 3/3

Найдите методы в Program.cs

```
            statusCode: StatusCodes.Status500InternalServerError);
    }
}
app.MapGet("/notes", ReadNotes);
app.MapGet("/api/notes", ReadNotes);
app.MapGet("/", () => Results.Redirect("/notes"));
app.Run();
```

ШАГ 3 ИЗ 7

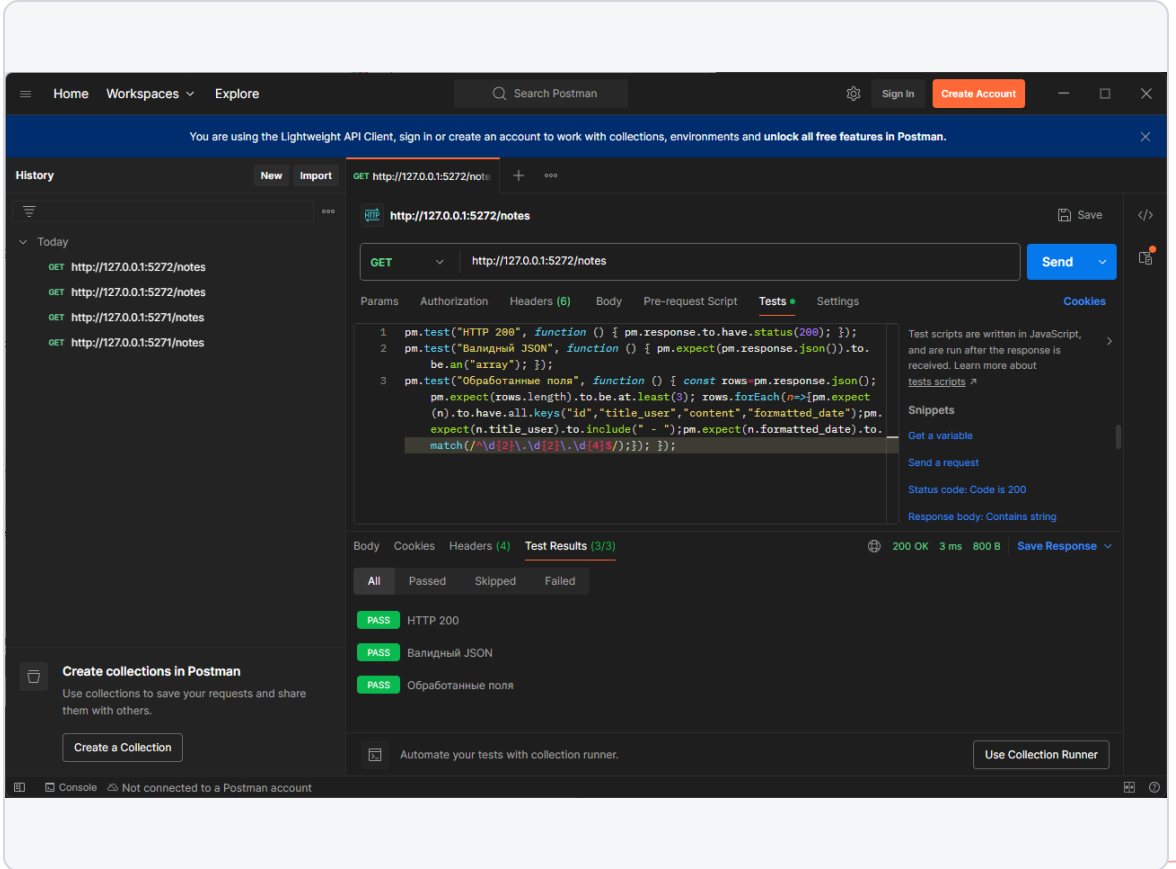
Получите успешный ответ в Postman

Документация должна показывать ответ работающего сервиса с вашими данными.

1. Запустите свой Api из профиля **http** в **Visual Studio**. Проверьте, что выбран адрес **http://127.0.0.1:5272**.
2. В **Postman** откройте коллекцию из задания 5. Выберите запрос списка заметок и нажмите **Send**.
3. Запишите адрес, статус и **JSON** из **Body**. Для примера можно привести одну заметку с подписью «Один элемент ответа».

ЧТО ПРОВЕРИТЬ

Postman показывает статус **200 OK**. В **JSON** видны **id**, **title_user**, **content** и **formatted_date**.



Postman: выполненные проверки **HTTP 200**, **JSON** и преобразованных полей. Маршрут: **WPF / PostgreSQL**.

ШАГ 4 ИЗ 7

Уточните параметр и пустой результат

Параметр user_id фильтрует заметки в учебном примере. Его следует описать, если он есть в вашем обработчике.

1. В Postman сравните GET http://127.0.0.1:5272/notes и тот же запрос с ?user_id=2. Если ваш обработчик принимает другой параметр, используйте его.
2. Для пустого результата укажите id автора, которого нет в базе. Нажмите Send. В примере это ?user_id=2147483647.
3. Запишите тип параметра, его обязательность и пример пустого ответа.

ЧТО ПРОВЕРИТЬ

Пустой список зафиксирован как HTTP 200 с JSON [], в соответствии с проверкой в Postman.

ШАГ 5 ИЗ 7

Возьмите примеры ответов 400 и 500

Статус 400 относится к неверному запросу. Статус 500 относится к сбою работающего сервиса.

1. Откройте в коллекции запрос с неверным значением **user_id** и нажмите **Send**. Сохраните статус и объект **error**.
2. Откройте запрос к отдельному экземпляру из задания 5. Укажите **error_url** для своего тестового профиля с неверным портом БД.
3. Сохраните точный запрос, условие воспроизведения, статус и JSON ответа.

ЧТО ПРОВЕРИТЬ

В **Postman** получены ответы **400** и **500** с полем **error**. Адрес и условие теста записаны.

ШАГ 6 ИЗ 7 · 1/2

Перенесите методы, параметры и ответы в DOCX

Читатель сможет повторить запрос, если в шаблоне указан точный адрес и объяснено каждое поле.

1. Впишите свой базовый URL. Для этого маршрута используется http://127.0.0.1:5272.
2. Укажите методы и параметры. Опишите JSON-массив заметок и четыре поля, включая разделитель в title_user и формат даты.
3. В таблицу HTTP внесите 200 с заметками и с [], 400 и 500. Добавьте к каждому случаю фактический пример из Postman.

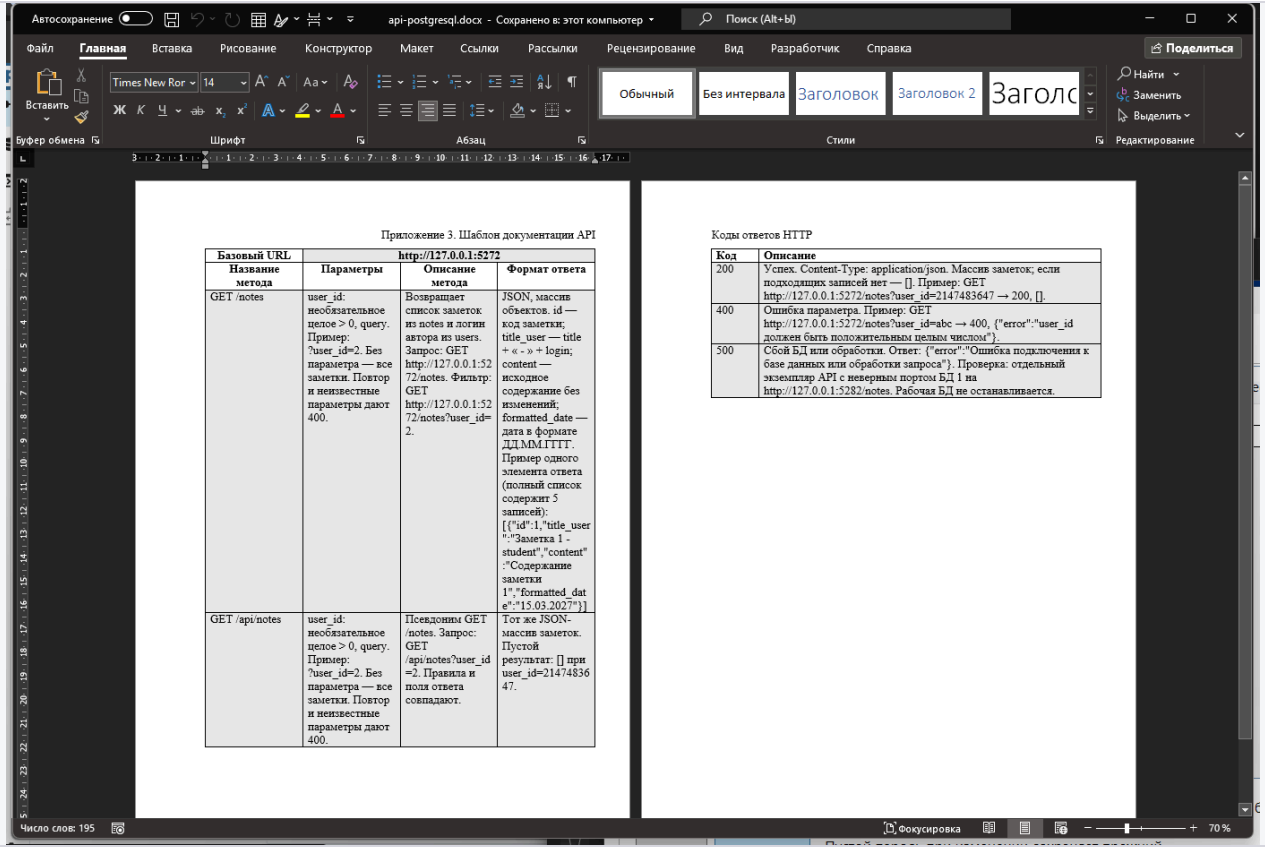
ЧТО ПРОВЕРИТЬ

DOCX содержит рабочий адрес, все реализованные методы, типы параметров и фактические ответы.

ШАГ 6 ИЗ 7 · 2/2

Перенесите методы, параметры и ответы в DOCX

Покажите на экране



Приложение 3. Шаблон документации API

Базовый URL	http://127.0.0.1:5272		
Название метода	**Параметры**	**Описание метода**	**Формат ответа**
GET /notes	user_id: необязательное целое > 0, query. Пример: ?user_id=2. Без параметра — все заметки. Повтор и неизвестные параметры дают 400.	Возвращает список заметок из notes и логин автора из users. Запрос: GET http://127.0.0.1:5272/notes. Фильтр: GET http://127.0.0.1:5272/notes?user_id=2.	JSON, массив объектов. id — код заметки; title_user — title + « - » + login; content — исходное содержание без изменений; formatted_date — дата в формате ДД.ММ.ГГГГ. Пример одного элемента ответа (полный список содержит 5 записей): [{"id":1,"title_user":"Заметка 1 - student","content":"Содержание заметки 1","formatted_date":"15.03.2027"}]
GET /api/notes	user_id: необязательное целое > 0, query. Пример: ?user_id=2. Без параметра — все заметки. Повтор и неизвестные параметры дают 400.	Псевдоним GET /notes. Запрос: GET /api/notes?user_id=2. Правила и поля ответа совпадают.	Тот же JSON-массив заметок. Пустой результат: [] при user_id=2147483647.

Коды ответов HTTP

Код	Описание
200	Успех. Content-Type: application/json. Массив заметок; если подходящих записей нет — []. Пример: GET http://127.0.0.1:5272/notes?user_id=2147483647 → 200, [].
400	Ошибка параметра. Пример: GET http://127.0.0.1:5272/notes?user_id=abc → 400, {"error":"user_id должен быть положительным целым числом"}.
500	Сбой БД или обработки. Ответ: {"error":"Ошибка подключения к базе данных или обработки запроса"}. Проверка: отдельный экземпляр API с неверным портом БД 1 на http://127.0.0.1:5282/notes. Рабочая БД не останавливается.

Заполненный исходный шаблон документации API в **Microsoft Word**. Маршрут: **WPF / PostgreSQL**.

ШАГ 7 ИЗ 7

Сверьте и сохраните документацию

Проверка по готовому документу выявит неточный адрес, пропущенный параметр или пример из другого проекта.

1. Выполните обычный запрос в **Postman** по адресу из **DOCX** и сравните ответ с документом.
2. Если рядом с проектом есть **openapi.json**, сверьте в нём пути, параметры и коды ответов с **Program.cs** и **DOCX**.
3. Сохраните заполненный **DOCX** в «Результаты». Откройте его снова и проверьте, что примеры и таблицы читаются полностью.

ЧТО ПРОВЕРИТЬ

По сохранённому документу можно выполнить запрос к своему **ASP.NET Core Minimal API**.

ШАГ 7 · OPENAPI.JSON · 1/14

Сверьте и сохраните документацию

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "ДЭ 2027 — API заметок",
    "version": "1.0.0",
    "description": "КИМ 09.02.07-5-2027, ГИА БУ, задания 5 и 6. Учебный сервер работает локально."
  },
  "servers": [
    {
      "url": "/",
      "description": "Текущий локальный сервер"
    }
  ],
  "paths": {
    "/notes": {
```

ШАГ 7 · OPENAPI.JSON · 2/14

Сверьте и сохраните документацию

```
      "get": {
        "operationId": "getNotes",
        "summary": "Получить обработанные заметки",
        "description": "Читает notes и users. Объединяет title и login, сохраняет content, форматирует created_at. user_id —
необязательное расширение учебного примера.",
        "parameters": [
          {
            "name": "user_id",
            "in": "query",
            "required": false,
            "description": "Положительный целочисленный идентификатор автора. Если автора нет, возвращается [].",
            "schema": {
              "type": "integer",
              "format": "int32",
              "minimum": 1,
```

ШАГ 7 · OPENAPI.JSON · 3/14

Сверьте и сохраните документацию

```
          "maximum": 2147483647
        }
      }
    ],
    "responses": {
      "200": {
        "description": "Массив заметок, в том числе пустой",
        "content": {
          "application/json": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Note"
              }
            },
            "examples": {
```

ШАГ 7 · OPENAPI.JSON · 4/14

Сверьте и сохраните документацию

```
            "filled": {
              "value": [
                {
                  "id": 1,
                  "title_user": "Заметка 1 - student",
                  "content": "Содержание заметки 1",
                  "formatted_date": "15.03.2027"
                }
              ]
            },
            "empty": {
              "value": []
            }
          }
        }
      }
```

ШАГ 7 · OPENAPI.JSON · 5/14

Сверьте и сохраните документацию

```
},
"400": {
  "description": "Неверный user_id или неизвестный параметр",
  "content": {
    "application/json": {
      "schema": {
        "$ref": "#/components/schemas/Error"
      },
      "example": {
        "error": "user_id должен быть положительным целым числом"
      }
    }
  }
},
"500": {
  "description": "Ошибка БД или обработки",
```



ШАГ 7 · OPENAPI.JSON · 6/14

Сверьте и сохраните документацию

```
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Error"
              },
              "example": {
                "error": "Ошибка подключения к базе данных или обработки запроса"
              }
            }
          }
        }
      }
    },
    "/api/notes": {
      "get": {
```

ШАГ 7 · OPENAPI.JSON · 7/14

Сверьте и сохраните документацию

```
        "operationId": "getApiNotes",
        "summary": "Получить обработанные заметки",
        "description": "Читает notes и users. Объединяет title и login, сохраняет content, форматирует created_at. user_id —
необязательное расширение учебного примера.",
        "parameters": [
          {
            "name": "user_id",
            "in": "query",
            "required": false,
            "description": "Положительный целочисленный идентификатор автора. Если автора нет, возвращается [].",
            "schema": {
              "type": "integer",
              "format": "int32",
              "minimum": 1,
              "maximum": 2147483647
```

ШАГ 7 · OPENAPI.JSON · 8/14

Сверьте и сохраните документацию

```
        }
      }
    ],
    "responses": {
      "200": {
        "description": "Массив заметок, в том числе пустой",
        "content": {
          "application/json": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Note"
              }
            },
            "examples": {
              "filled": {
```

ШАГ 7 · OPENAPI.JSON · 9/14

Сверьте и сохраните документацию

```
                "value": [
                  {
                    "id": 1,
                    "title_user": "Заметка 1 - student",
                    "content": "Содержание заметки 1",
                    "formatted_date": "15.03.2027"
                  }
                ]
              },
              "empty": {
                "value": []
              }
            }
          }
        }
      },
```

ШАГ 7 · OPENAPI.JSON · 10/14

Сверьте и сохраните документацию

```
        "400": {
          "description": "Неверный user_id или неизвестный параметр",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Error"
              },
              "example": {
                "error": "user_id должен быть положительным целым числом"
              }
            }
          }
        },
        "500": {
          "description": "Ошибка БД или обработки",
          "content": {
```

ШАГ 7 · OPENAPI.JSON · 11/14

Сверьте и сохраните документацию

```
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Error"
              },
              "example": {
                "error": "Ошибка подключения к базе данных или обработки запроса"
              }
            }
          }
        }
      }
    }
  },
  "components": {
    "schemas": {
```

ШАГ 7 · OPENAPI.JSON · 12/14

Сверьте и сохраните документацию

```
    "Note": {
      "type": "object",
      "required": [
        "id",
        "title_user",
        "content",
        "formatted_date"
      ],
      "additionalProperties": false,
      "properties": {
        "id": {
          "type": "integer",
          "example": 1
        },
        "title_user": {
          "type": "string",
```

ШАГ 7 · OPENAPI.JSON · 13/14

Сверьте и сохраните документацию

```
          "description": "Заголовок, пробел, дефис, пробел, логин автора",
          "example": "Заметка 1 - student"
        },
        "content": {
          "type": "string",
          "example": "Содержание заметки 1"
        },
        "formatted_date": {
          "type": "string",
          "pattern": "^\\d{2}\\.\\d{2}\\.\\d{4}$",
          "example": "15.03.2027"
        }
      }
    },
    "Error": {
      "type": "object",
```

ШАГ 7 · OPENAPI.JSON · 14/14

Сверьте и сохраните документацию

```
        "required": [
          "error"
        ],
        "properties": {
          "error": {
            "type": "string"
          }
        }
      }
    }
  }
}
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Готов документ Прил_6_ОЗ_КИМ_09.02.07-5-2027.docx, заполненный по шаблону КИМ.

ЧТО ПРОВЕРИТЬ

Заполненный **DOCX** по шаблону КИМ с фактическими примерами ответов.

ГОТОВЫЙ ПРИМЕР

От исходных файлов до работающей системы

WPF + PostgreSQL

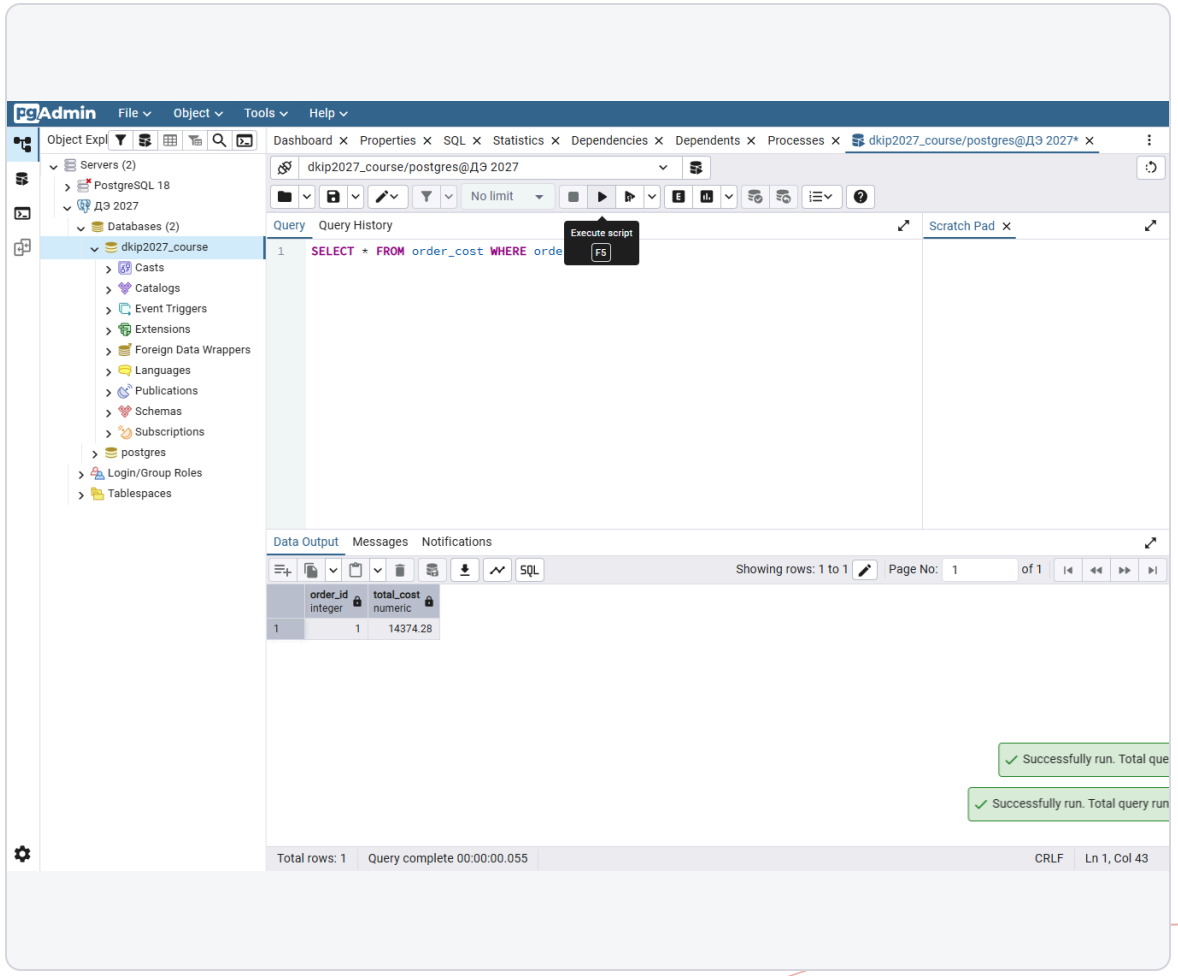
Готовый пример · ZIP

База и расчёт заказа

1. В pgAdmin откройте **counterparty** и сравните первую запись с «Заказчики.json».
2. Откройте представление **order_cost** и найдите заказ с **order_id = 1**.

ЧТО ПРОВЕРИТЬ

Первый код **000000001** сохранился. Нормативная стоимость — **14 374,28**.



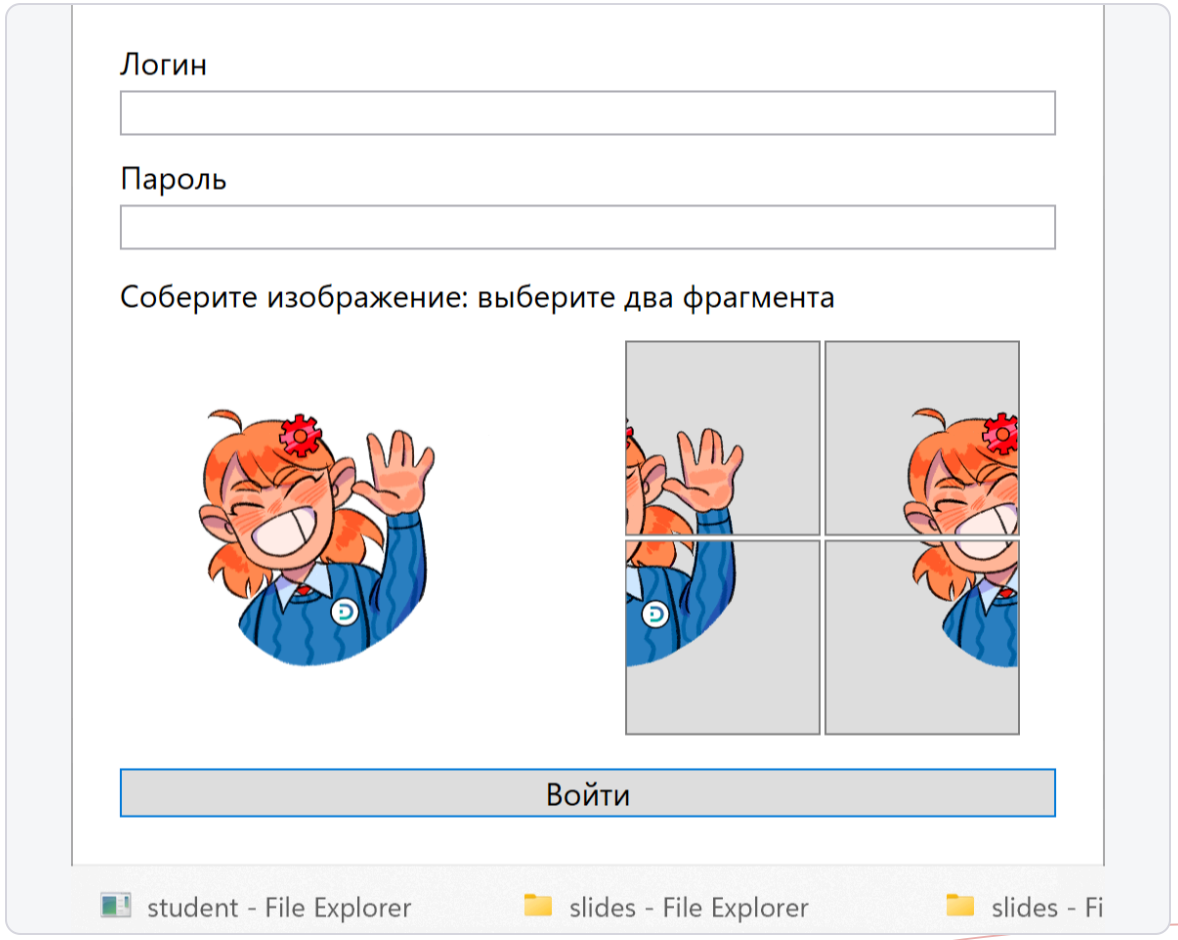
Результат расчёта в СУБД: **14 374,28**. Маршрут: **WPF / PostgreSQL**.

Вход в настольное приложение

1. Запустите WPF. В окне «Вход» введите логин и пароль, затем соберите пазл.
2. Покажите по очереди вход пользователя и администратора. Учебный пароль обеих записей: **Demo2027!**.

ЧТО ПРОВЕРИТЬ

Логин **student** открывает окно пользователя. Логин **admin** — управление учётными записями.



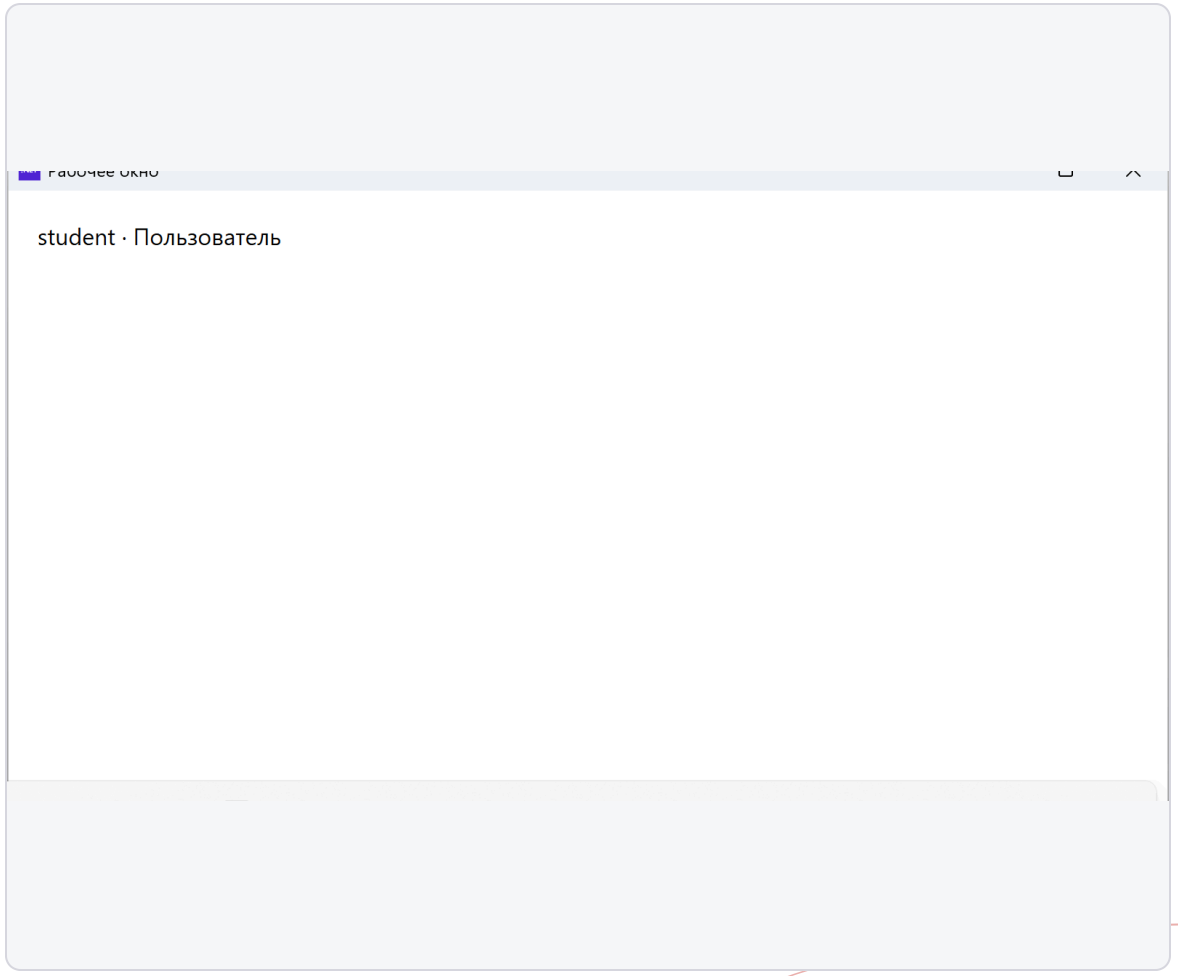
WPF. Окно «Вход»: поля логина и пароля, образец пазла слева и четыре фрагмента справа.

Что видит пользователь

1. Войдите как **student**. Найдите имя и роль в рабочем окне.

ЧТО ПРОВЕРИТЬ

Есть роль «Пользователь». Управление учётными записями недоступно.



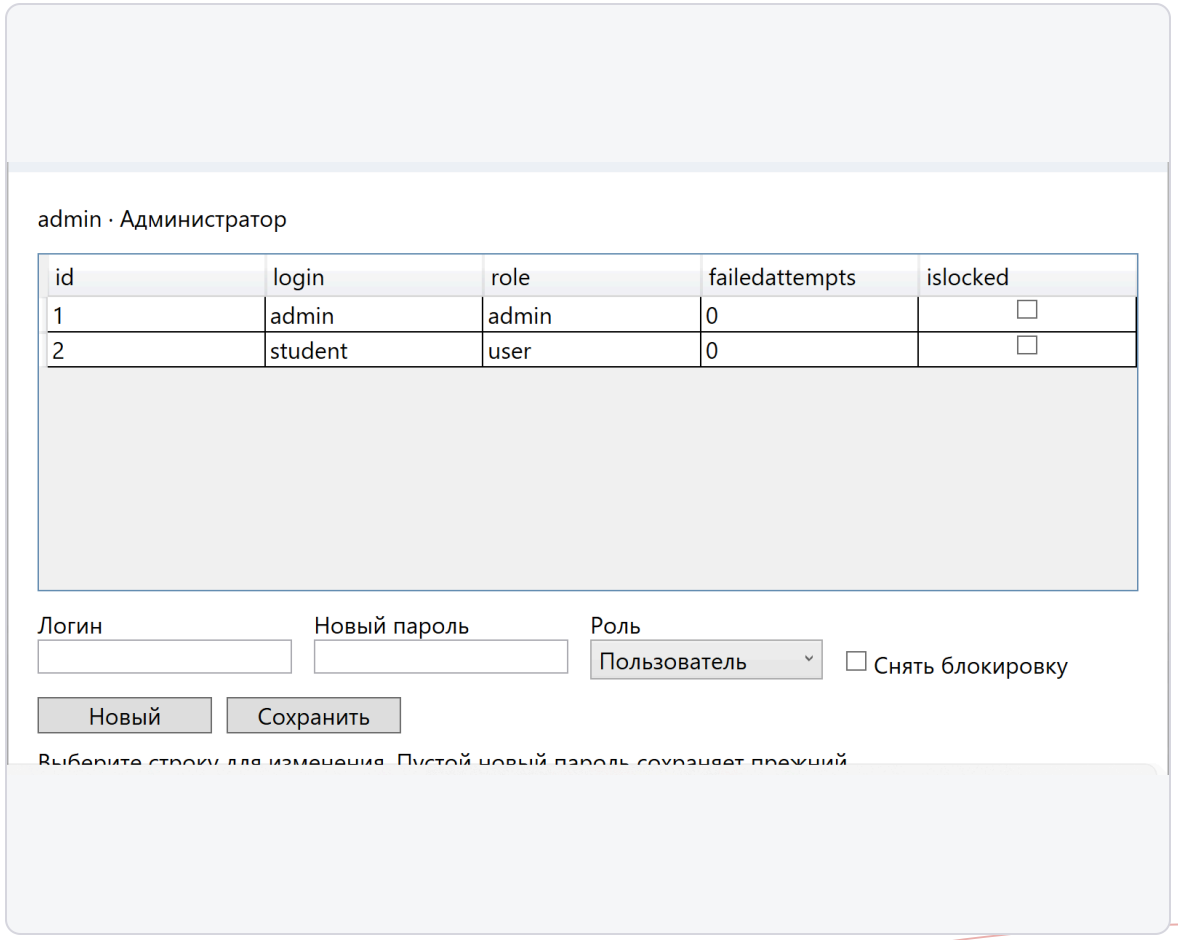
WPF. Рабочее окно **student:** видны логин и роль «Пользователь».

Что видит администратор

1. Войдите как **admin**. Покажите список, поля учётной записи, флажок блокировки и кнопки.
2. Создайте тестовую запись, проверьте блокировку и снимите её из окна администратора.

ЧТО ПРОВЕРИТЬ

Администратор может добавить, изменить и разблокировать учётную запись.

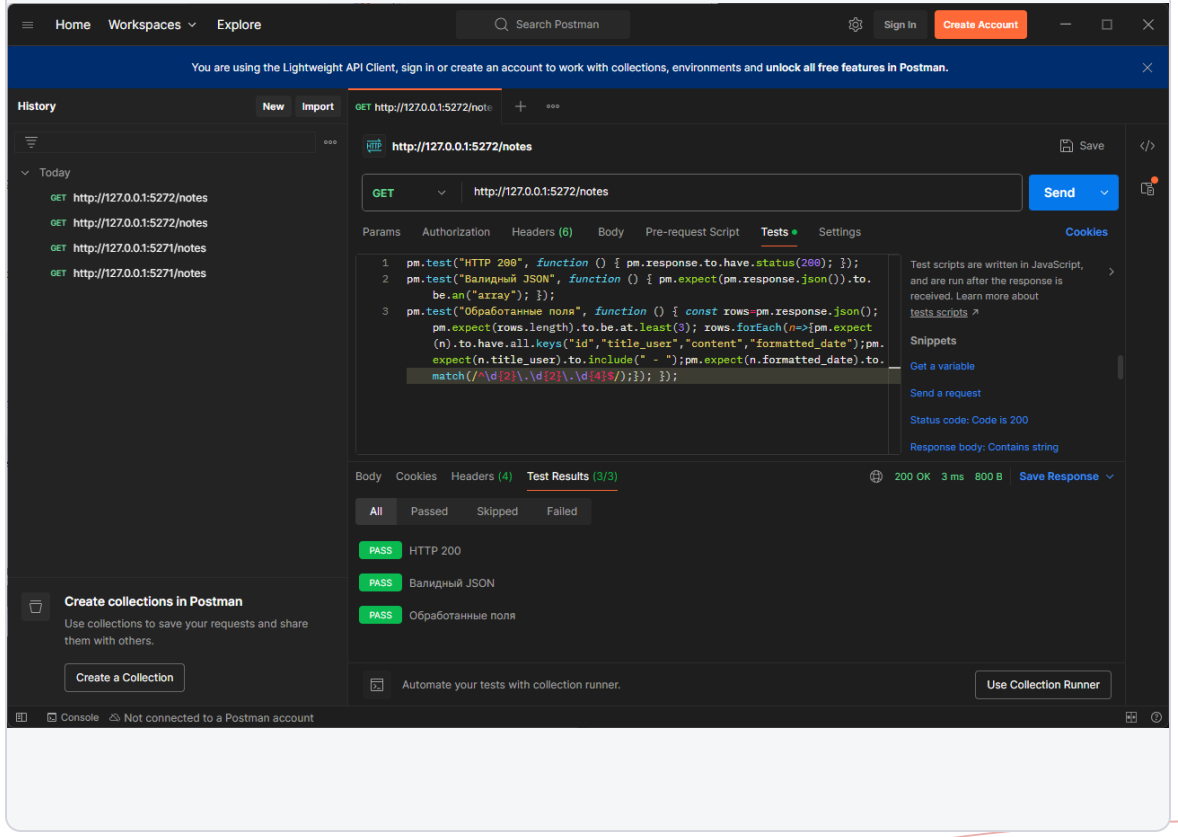


WPF. Рабочее окно **admin**: таблица учётных записей, поля редактирования и кнопки «Новый» и «Сохранить».

Запрос, тесты и документ Word

1. В Postman отправьте GET /notes. Покажите обработанный ответ и запустите тесты коллекции.
2. Сравните пример ответа в заполненном DOCX с результатом этого запроса.

Коллекция Postman Заполненный DOCX



Postman: выполненные проверки **HTTP 200, JSON** и преобразованных полей. Маршрут: **WPF / PostgreSQL.**

КИМ · ТАБЛИЦА 7

За что начисляют баллы

Критерий	Баллы
Осуществление администрирования отдельных компонент серверов	6
Выявление технических проблем при эксплуатации баз данных и серверов	11
Сбор исходных данных для проектной документации	6
Разработка модулей информационной системы по техническому заданию	10
Разработка подсистем безопасности	8
Разработка проектной документации по требованиям заказчика	6
Поиск, анализ и интерпретация информации	3
Разработка тестовых наборов и сценариев	9
Интеграция модулей в программное обеспечение	15
Отладка программного модуля	1

Открыть КИМ · стр. 13